

bok25

**基
礎**

班、考研数据結構



外部排序



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成

构造初始归并段->多路归并



外部排序

49	38	65	97	76	13	27
----	----	----	----	----	----	----



外部排序



长度为1的初始归并段



外部排序

49	38	65	97	76	13	27
----	----	----	----	----	----	----



外部排序

长度为3的初始归并段

49	38	65	97	76	13	27
----	----	----	----	----	----	----

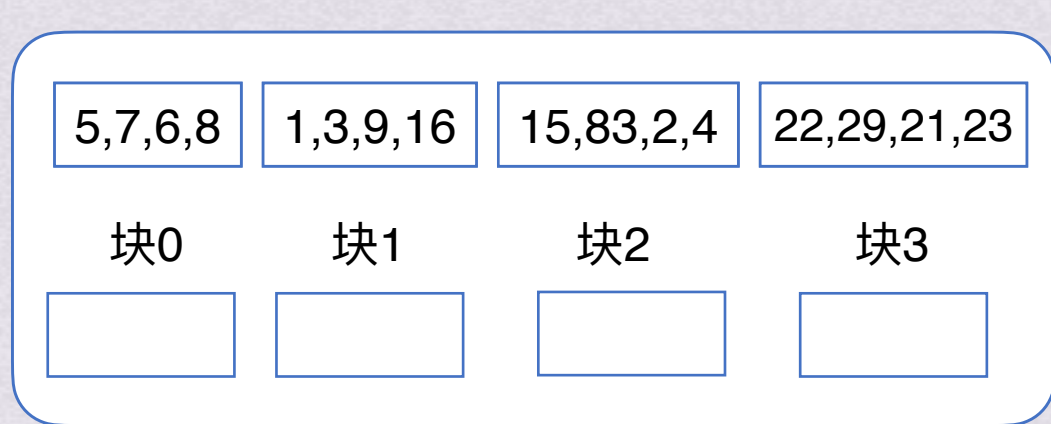
使初始归并段内的记录数增加，即可减少后续归并的次数



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



块4

块5

块6

块7

拥有8个块的磁盘



只能装下12个数字的
小的可怜的内存



能比较大小的cpu

- 1.cpu只能访问直接访问内存，不能访问外存
- 2.内存只能装得下12个数字
- 3.每次访问外存都需要大量时间

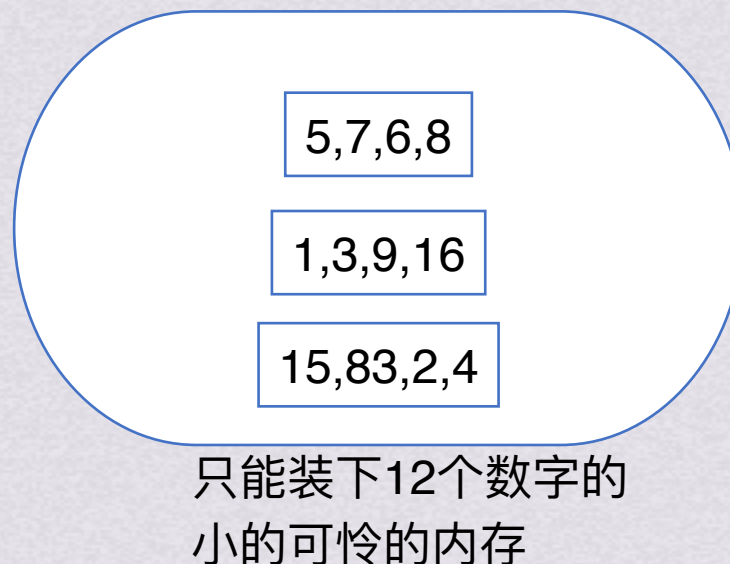
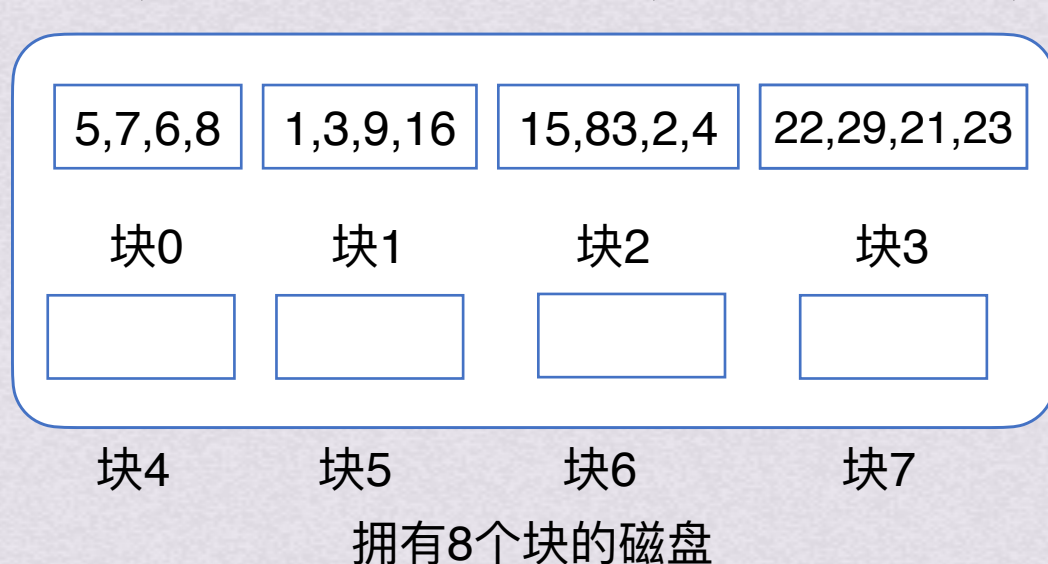
这种情况下，该如何对16个数字排序？



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



能比较大小的cpu

- 1.cpu只能访问直接访问内存，不能访问外存
- 2.内存只能装得下12个数字
- 3.每次访问外存都需要大量时间

1.把块放入内存

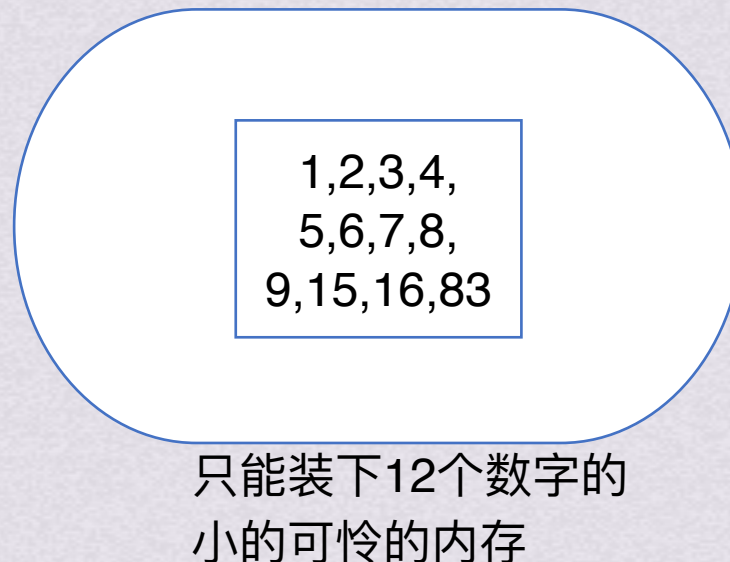
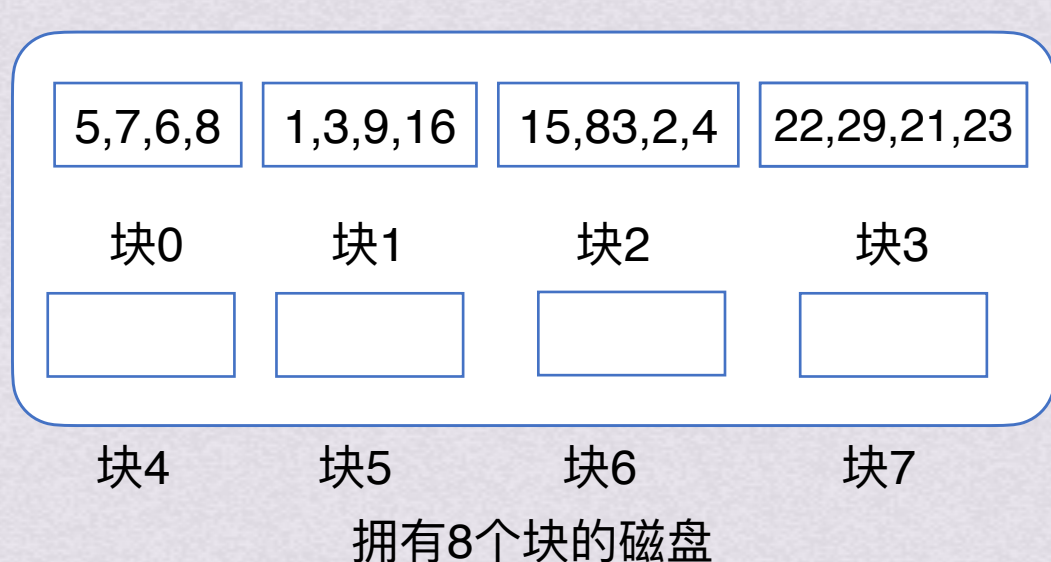
这种情况下，该如何对16个数字排序？



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

1.把块放入内存

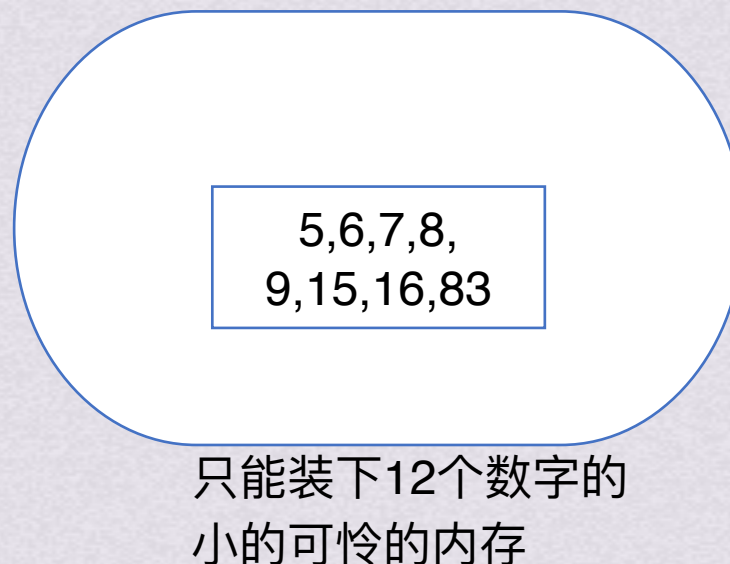
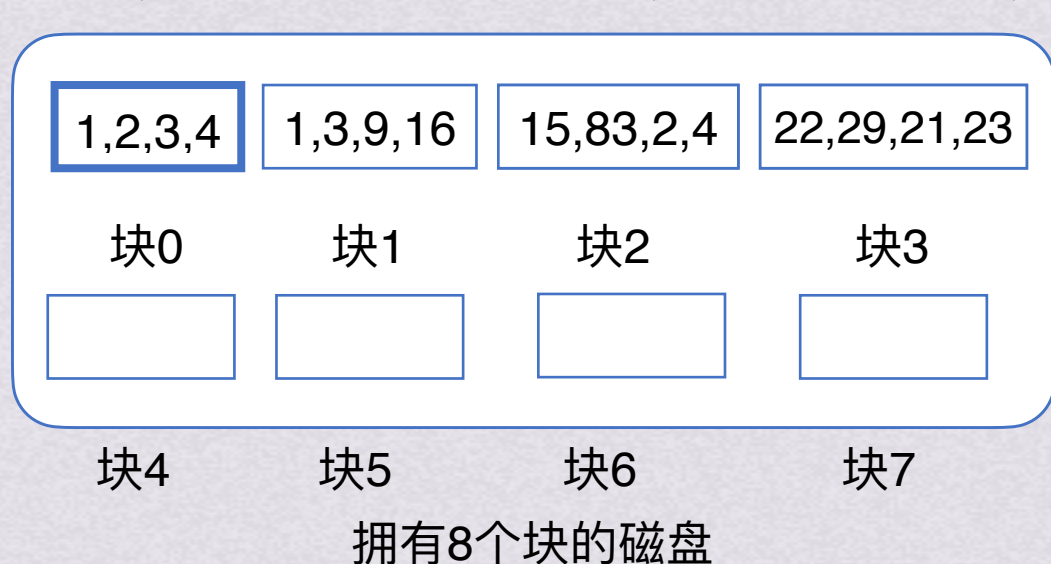
2.用之前学过的内部排序算法排序好



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

1.把块放入内存

2.用之前学过的内部排序算法排序好

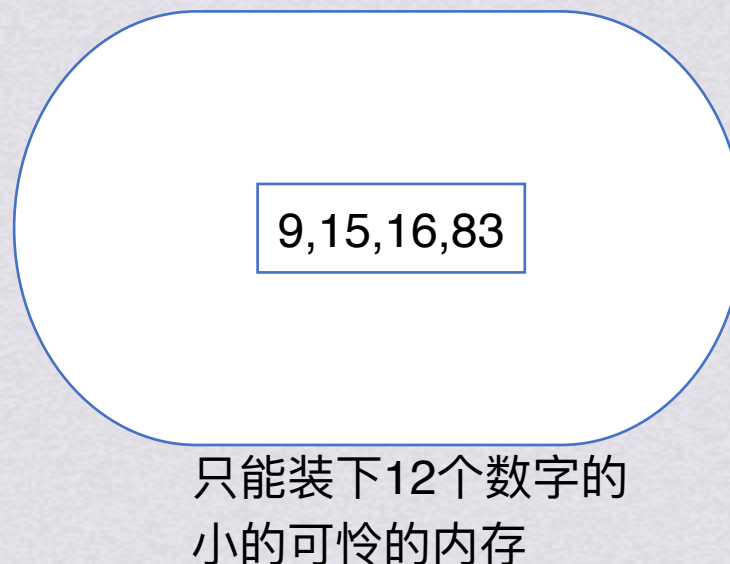
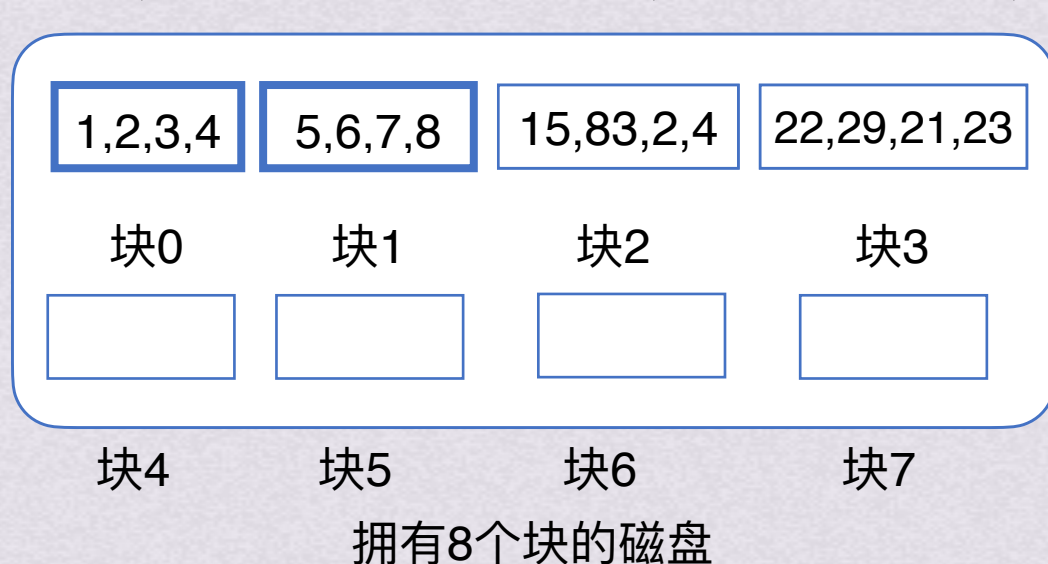
3.写回外存



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

1.把块放入内存

2.用之前学过的内部排序算法排序好

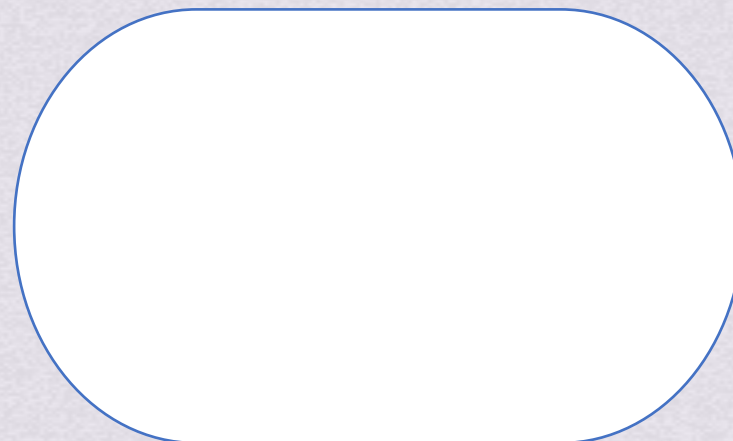
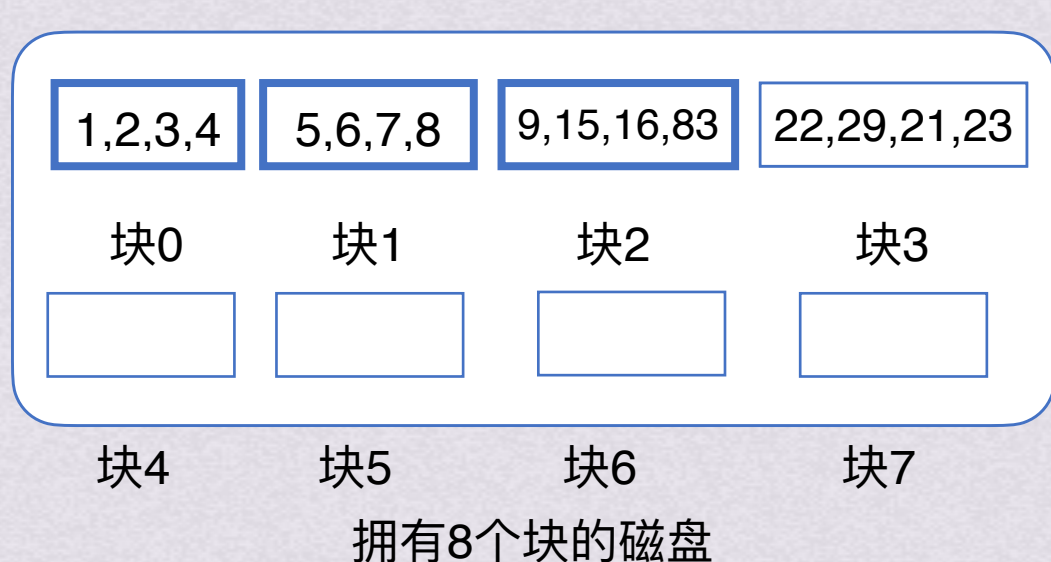
3.写回外存



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



只能装下12个数字的
小的可怜的内存



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

1.把块放入内存

2.用之前学过的内部排序算法排序好

3.写回外存

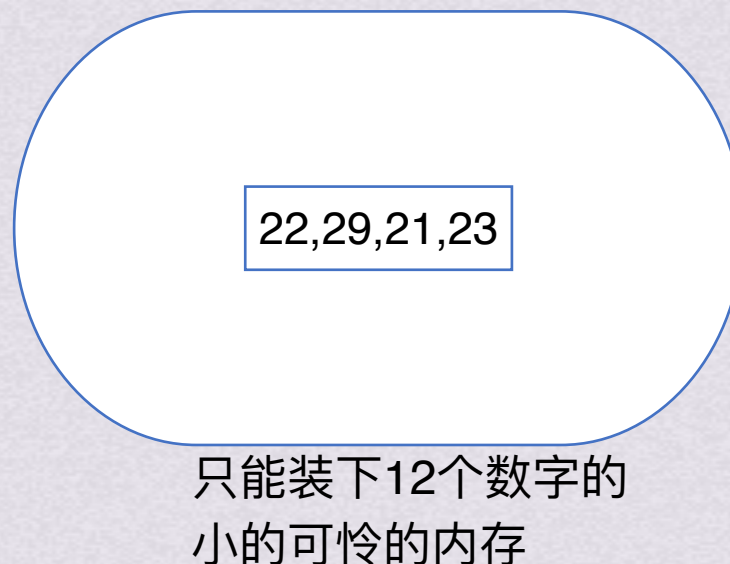
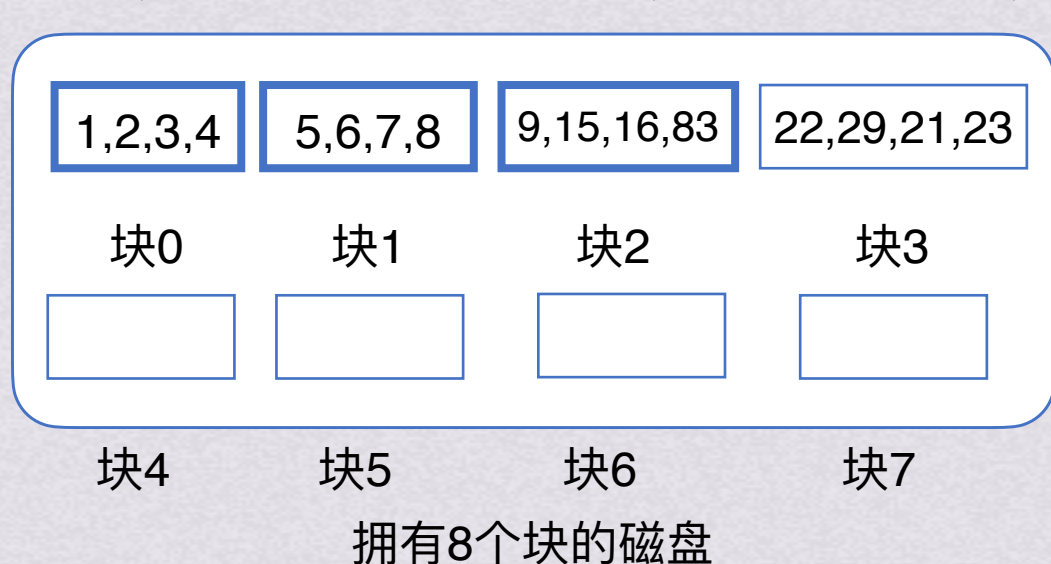
4.对其余块一样处理



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

1.把块放入内存

2.用之前学过的内部排序算法排序好

3.写回外存

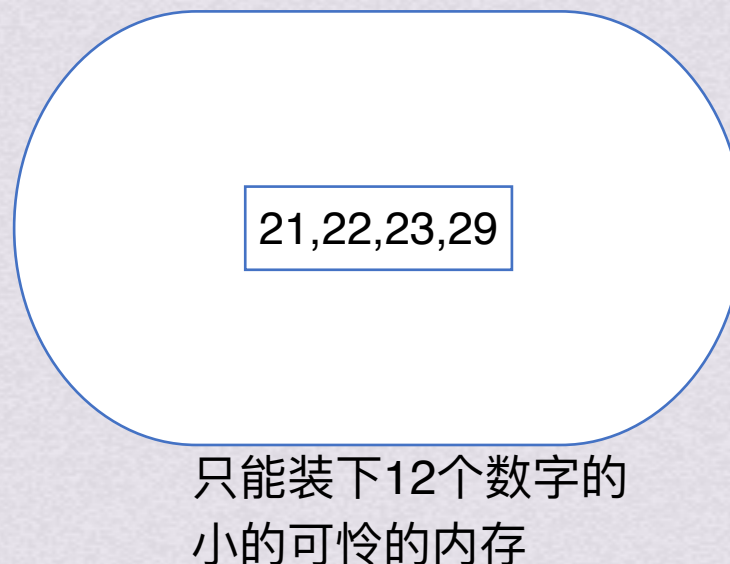
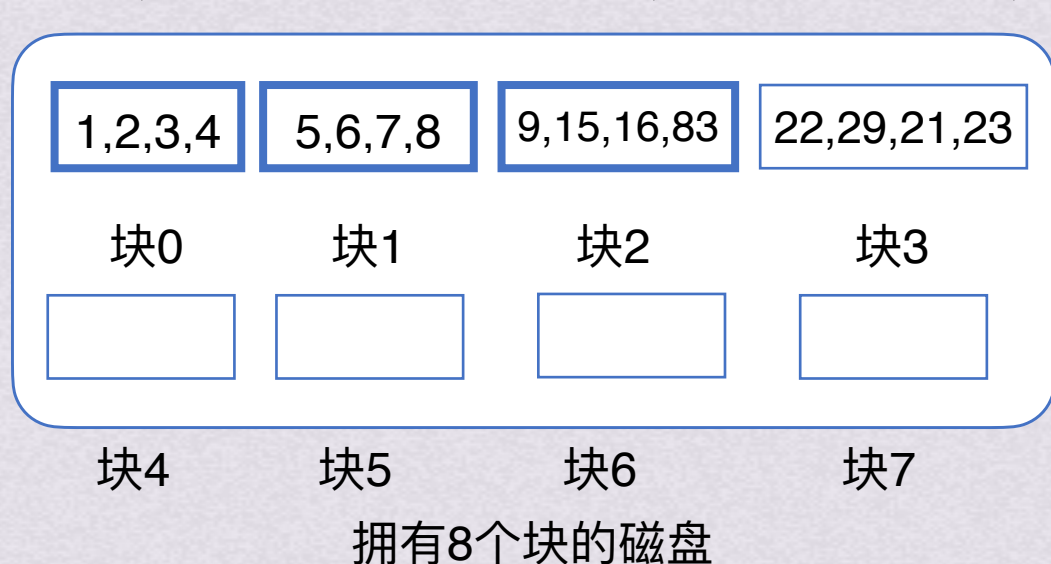
4.对其余块一样处理



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

1.把块放入内存

2.用之前学过的内部排序算法排序好

3.写回外存

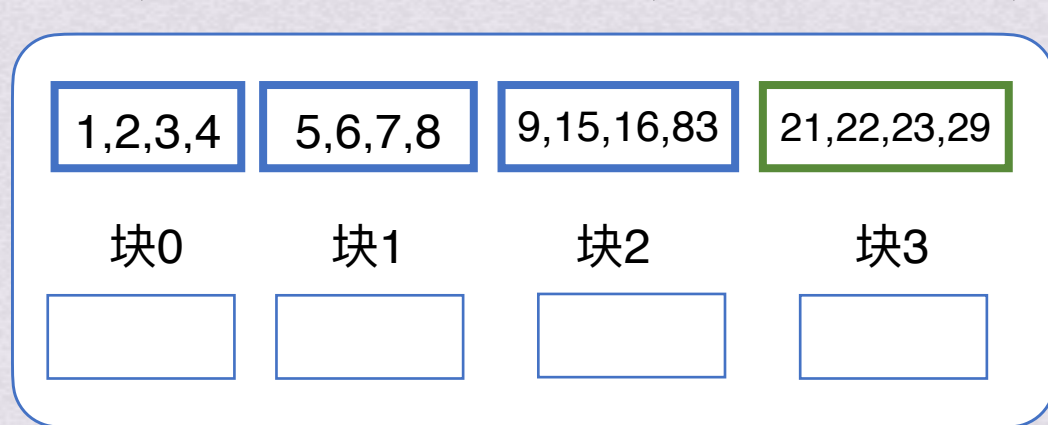
4.对其余块一样处理



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



块4

块5

块6

块7

拥有8个块的磁盘



只能装下12个数字的
小的可怜的内存



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

1.把块放入内存

2.用之前学过的内部排序算法排序好

3.写回外存

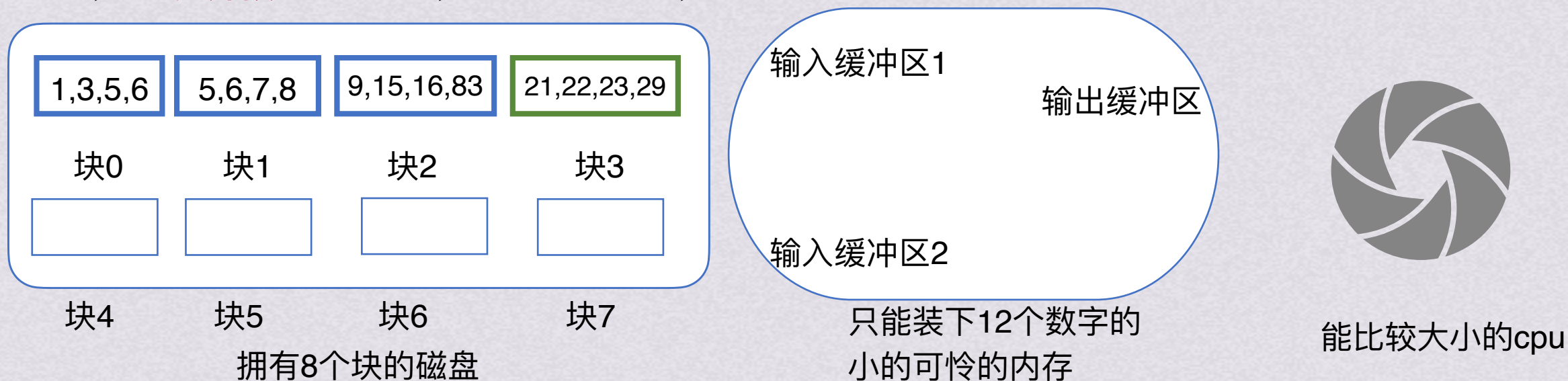
4.对其余块一样处理



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段



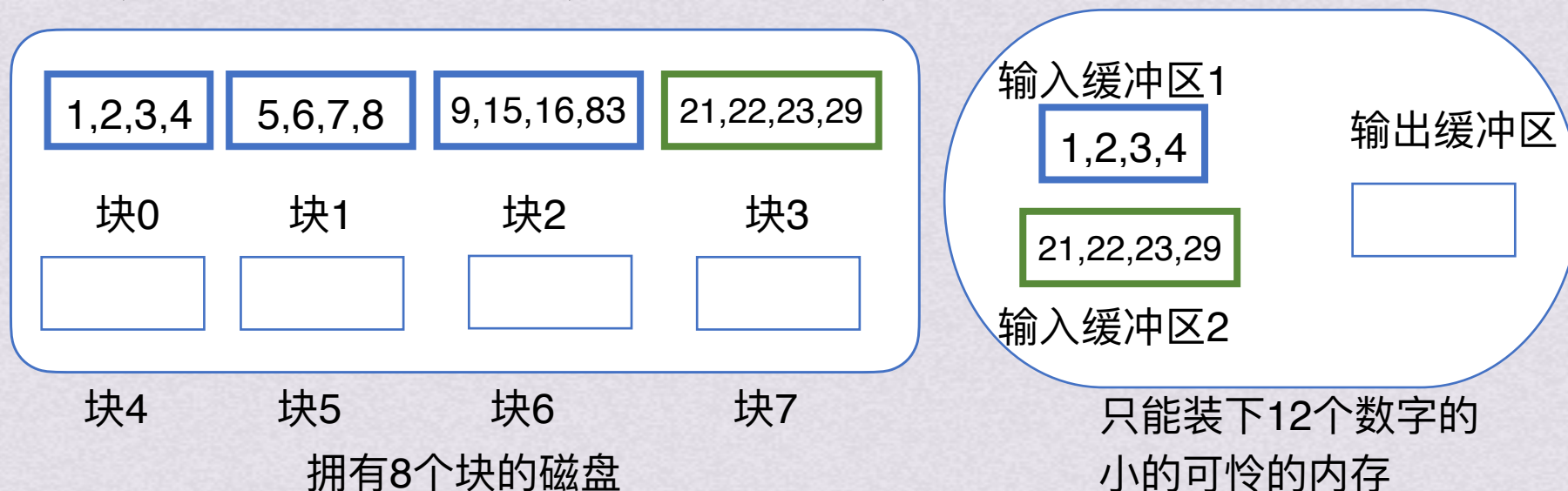
归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段

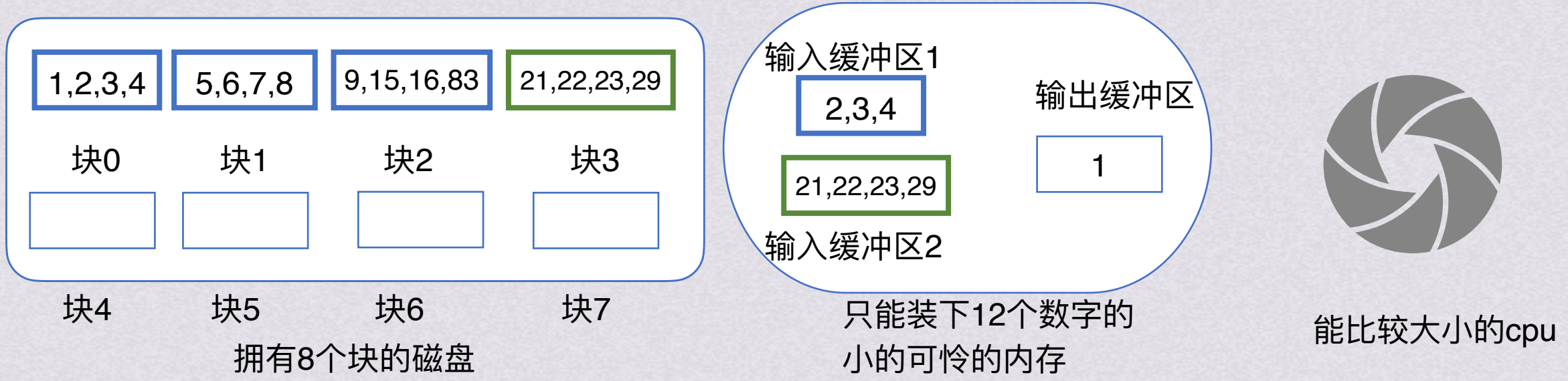


归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

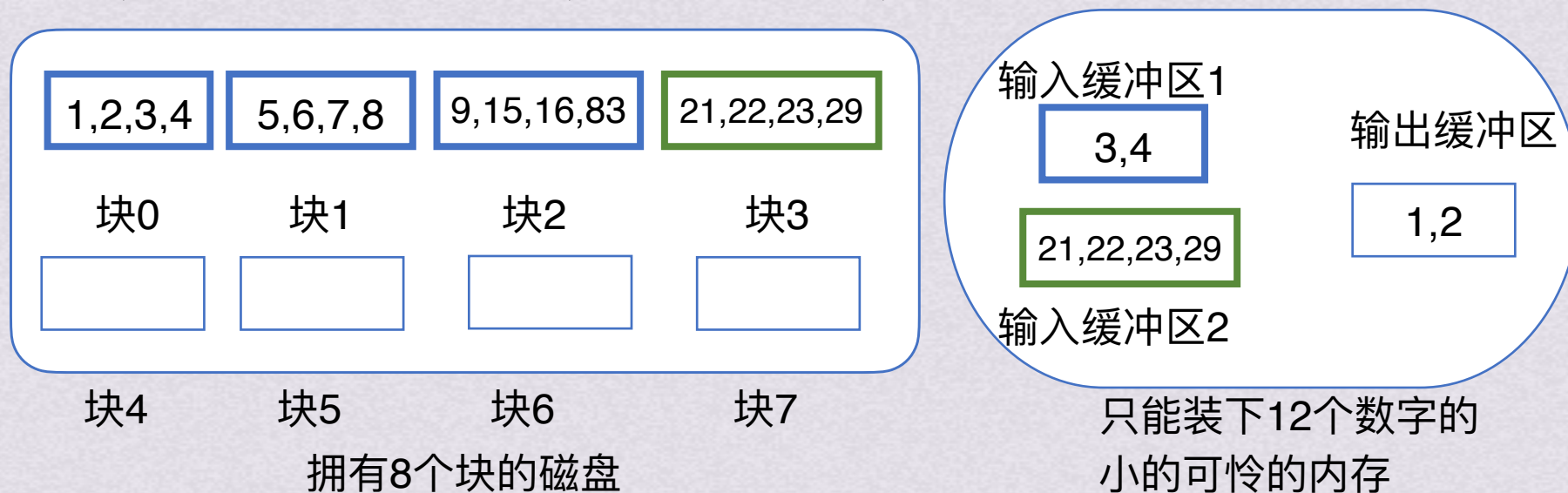
生成初始归并段 → 归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段

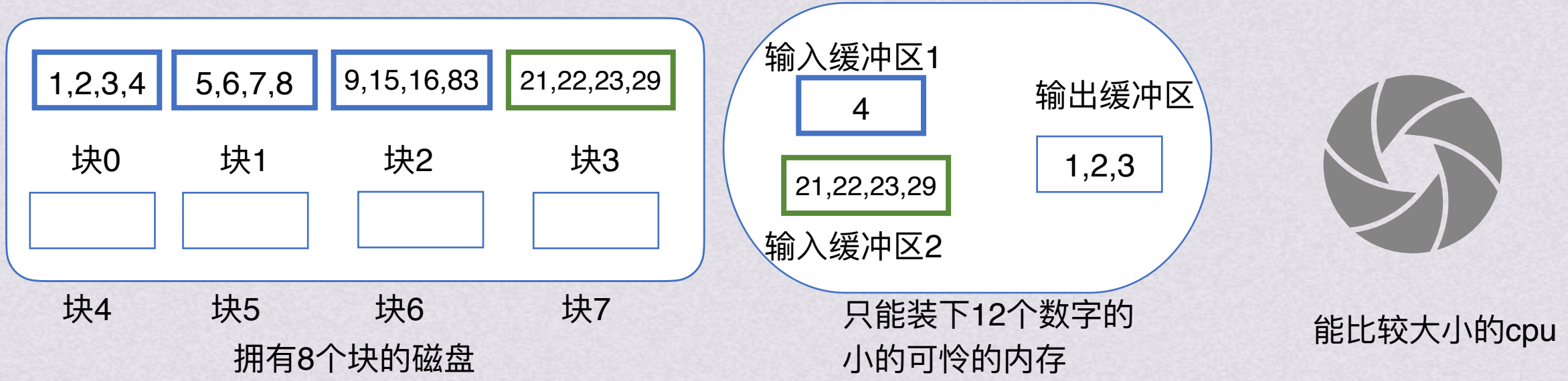


归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

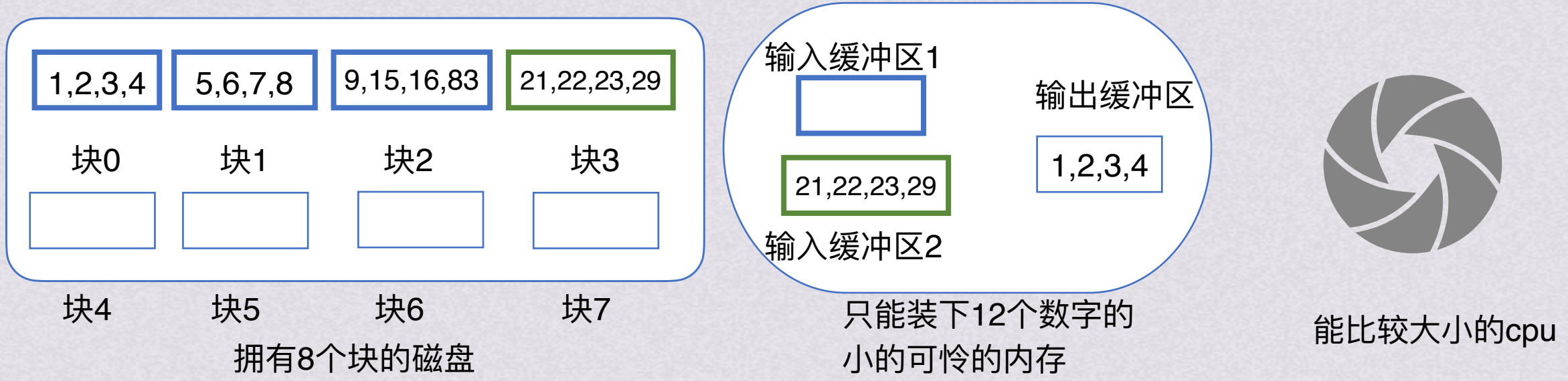
- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段 → 归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

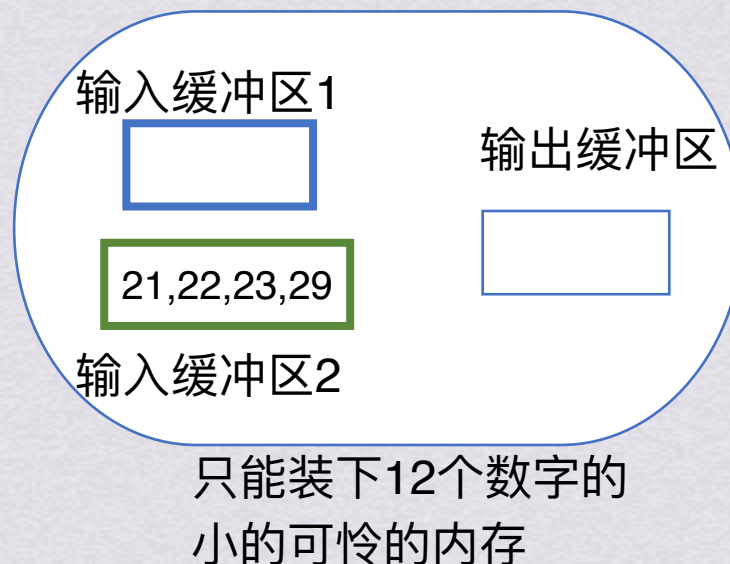
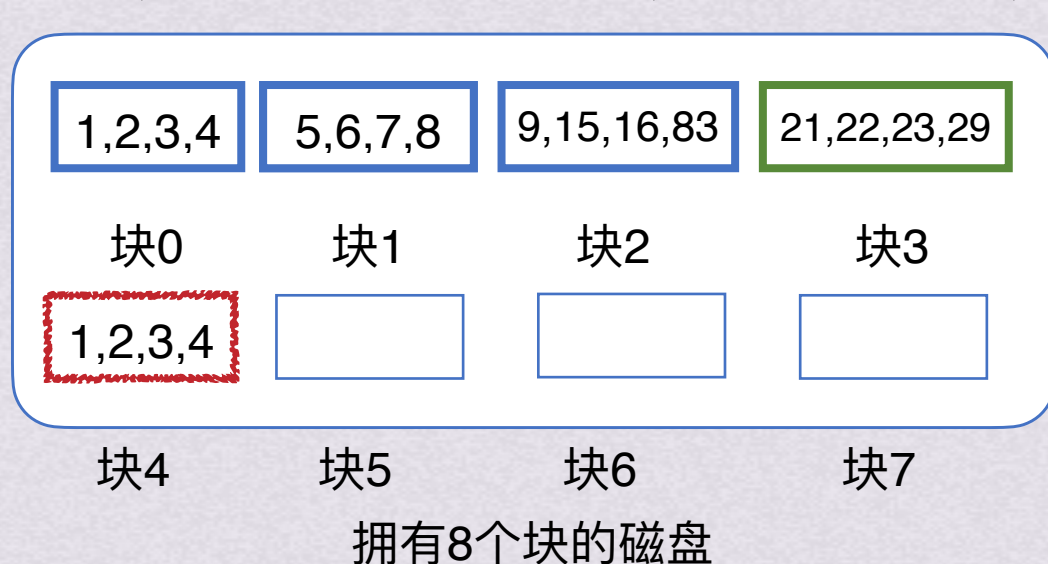
生成初始归并段 → 归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段

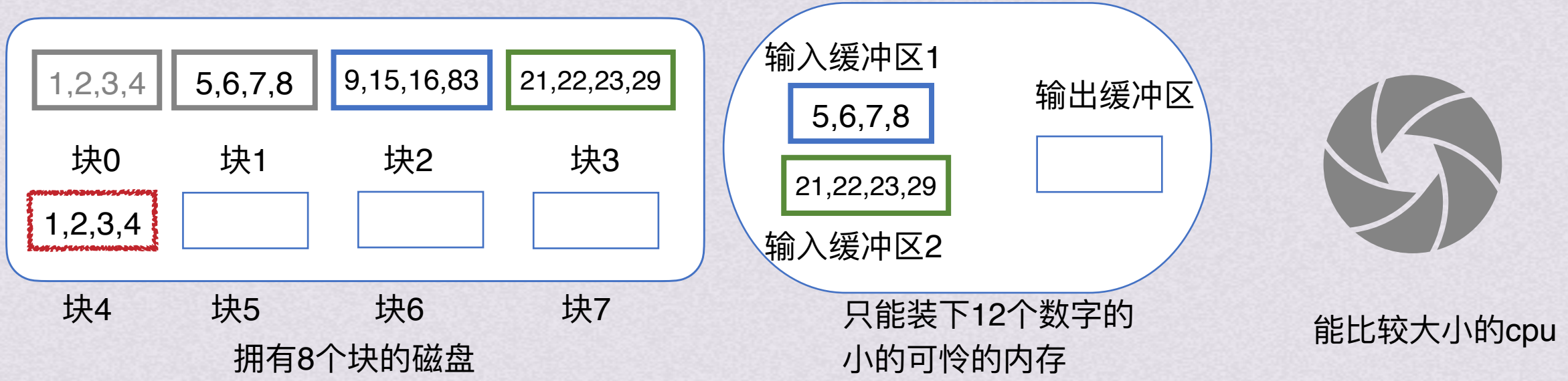


归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

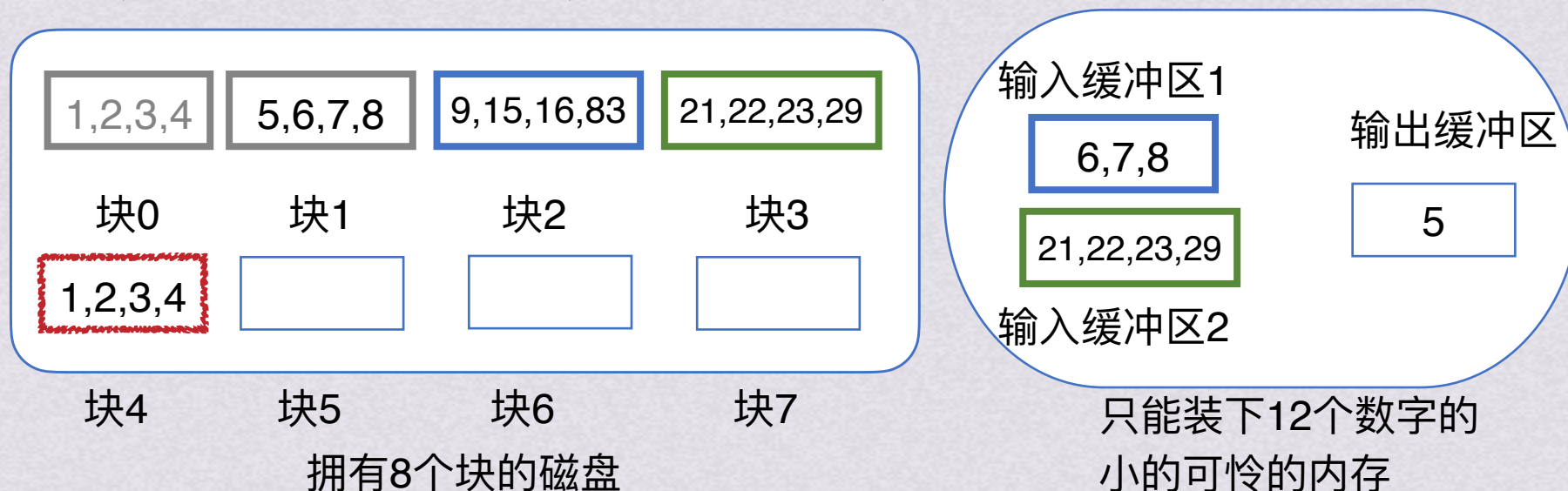
生成初始归并段 → 归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段

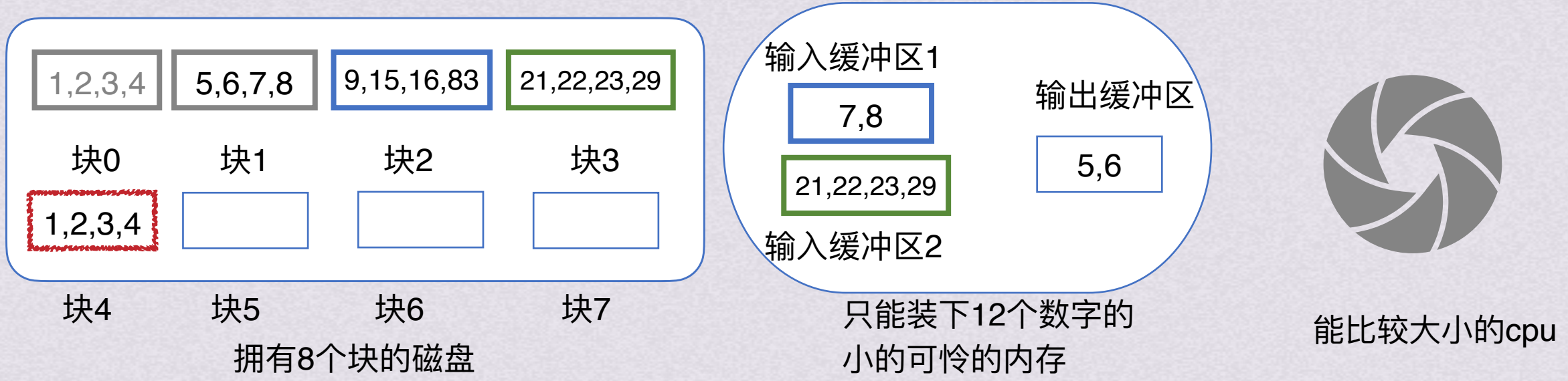


归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

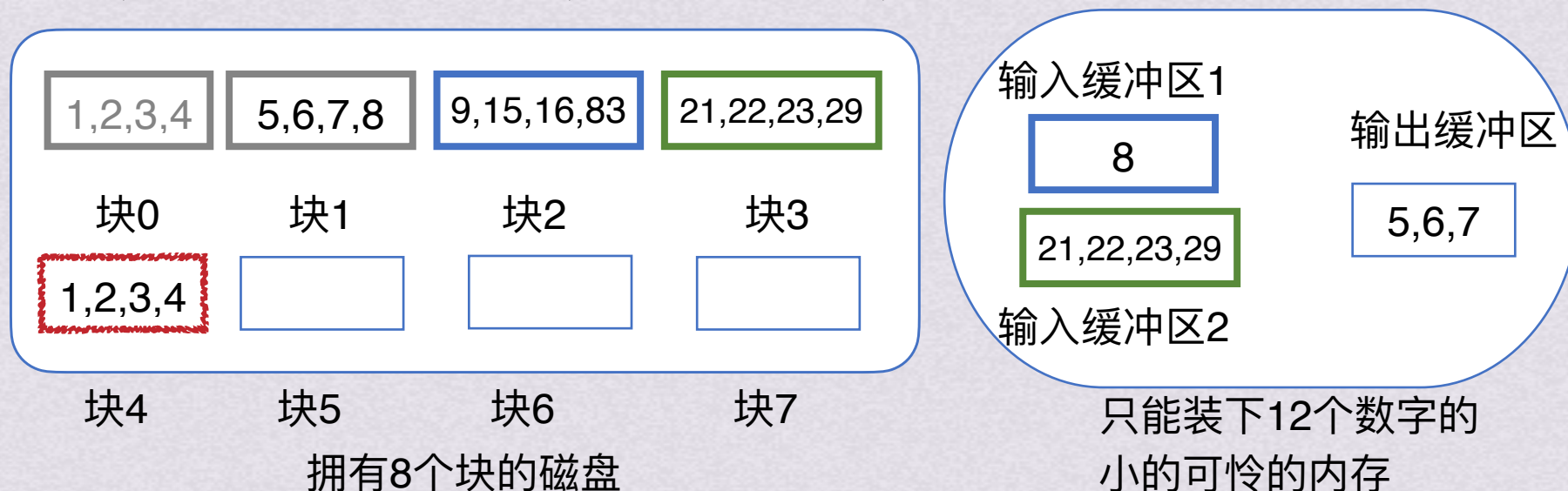
生成初始归并段 → 归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段

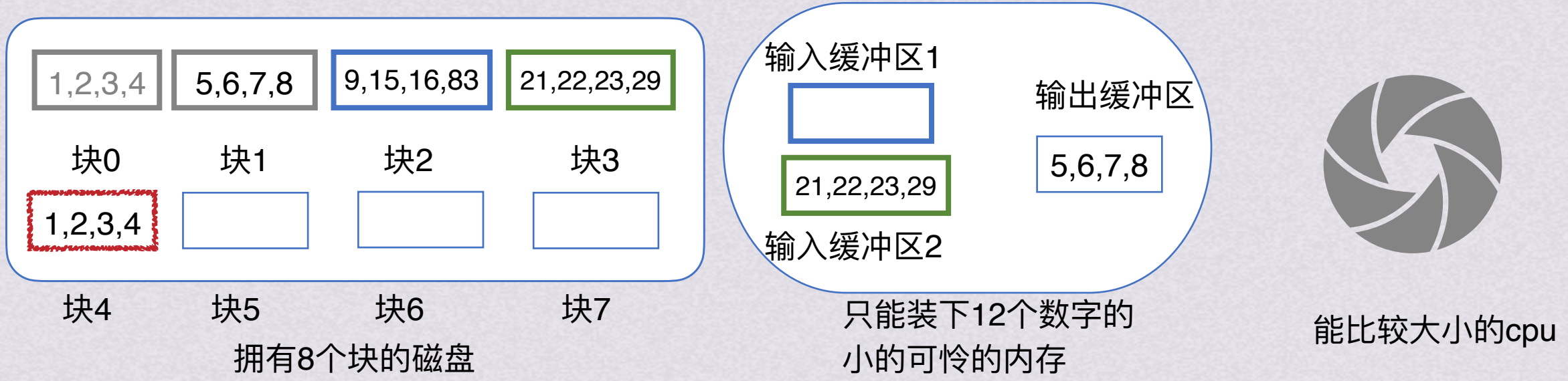


归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

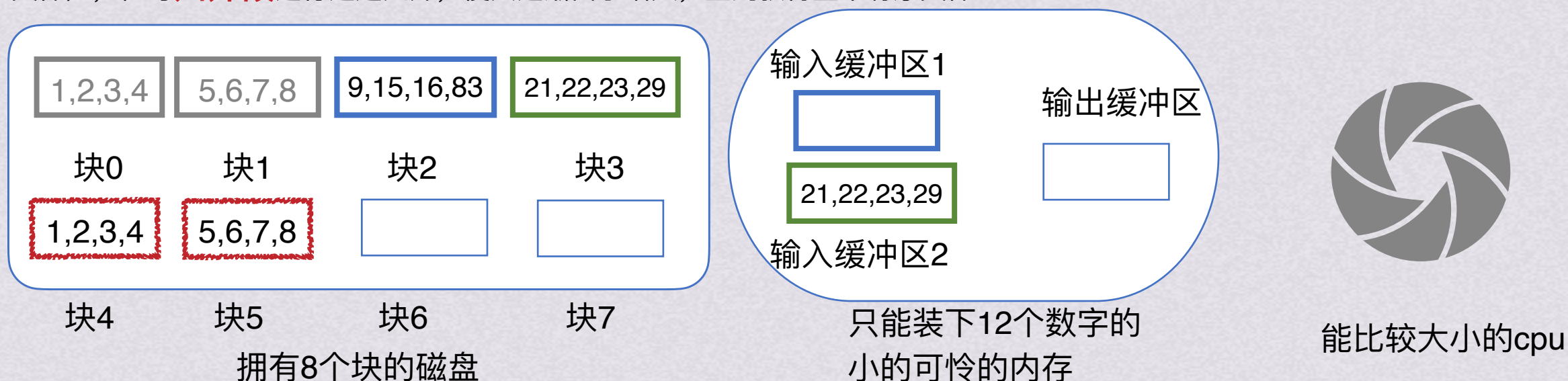
生成初始归并段 → 归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

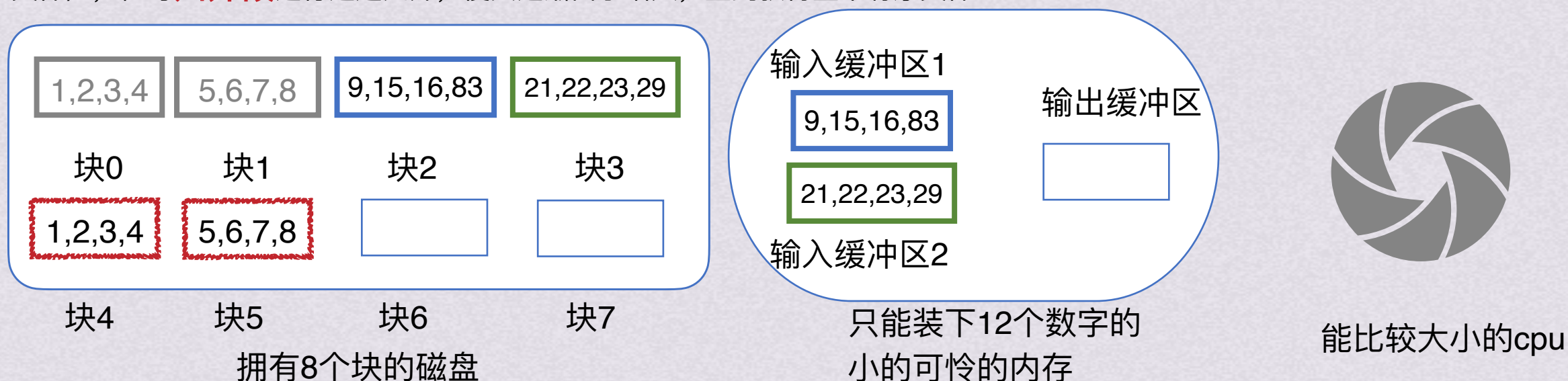
生成初始归并段 → 归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段



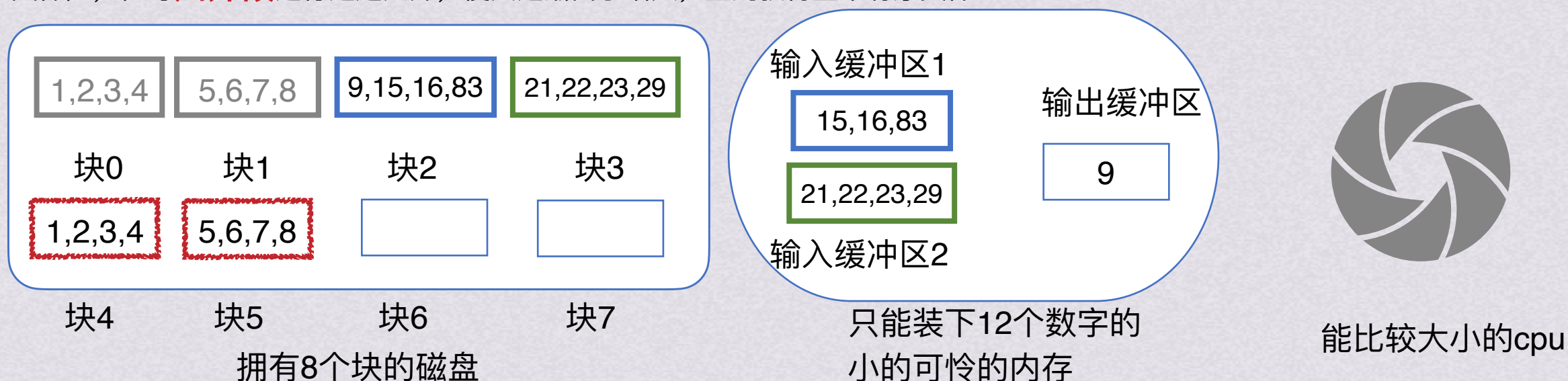
归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

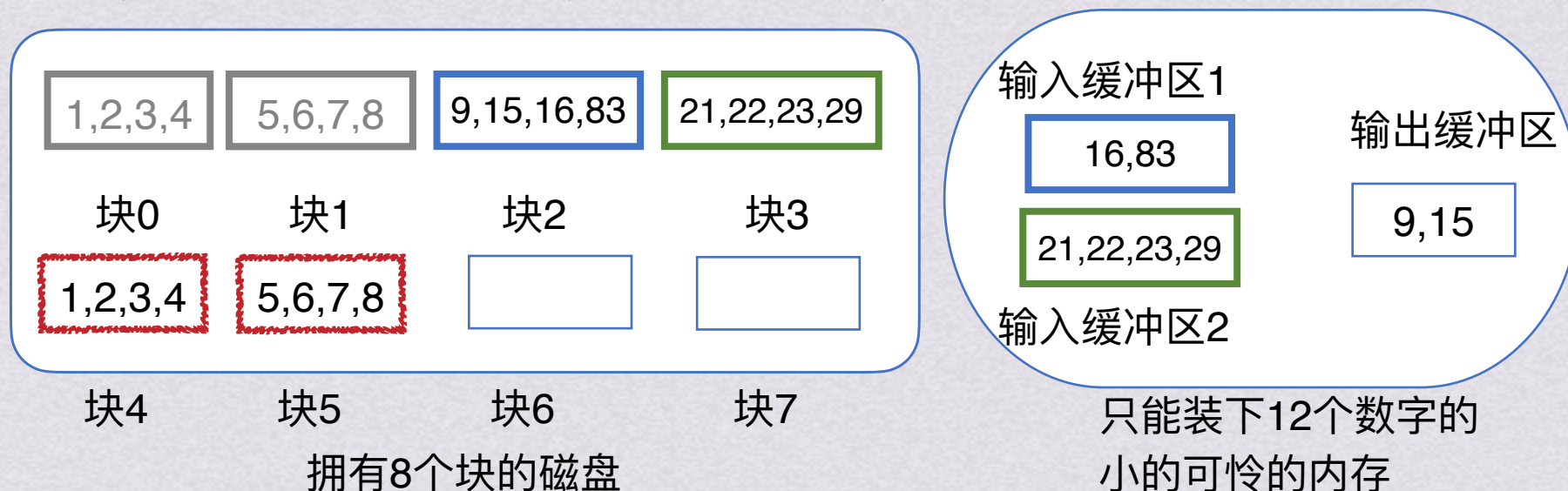
生成初始归并段 → 归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



能比较大小的cpu

1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段

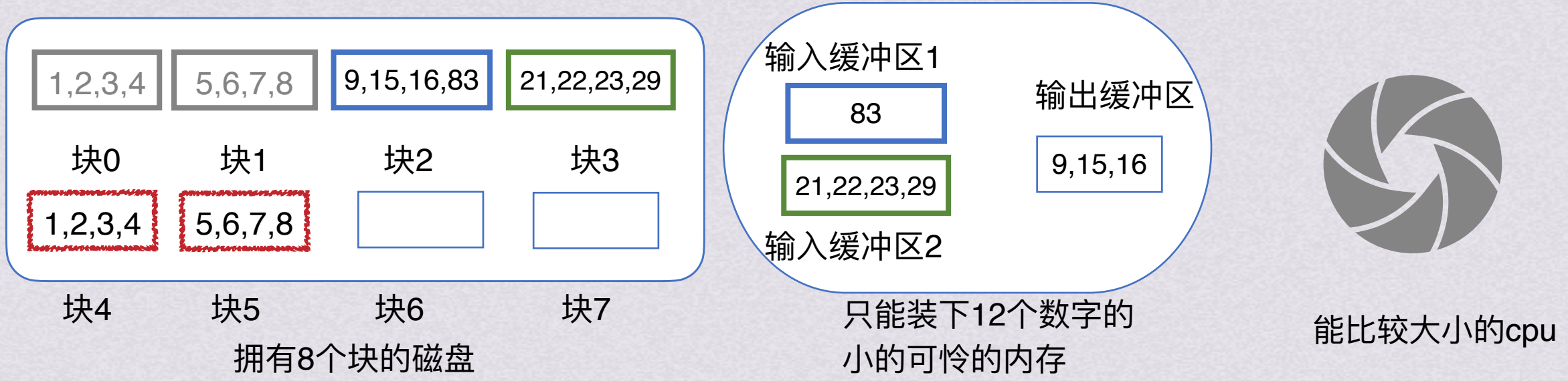


归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

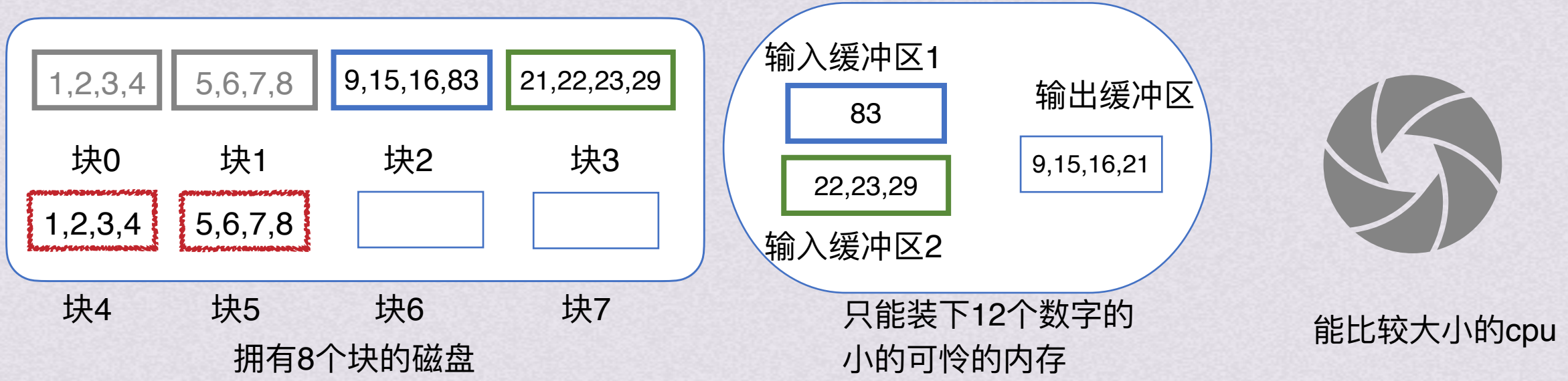
- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段 → 归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

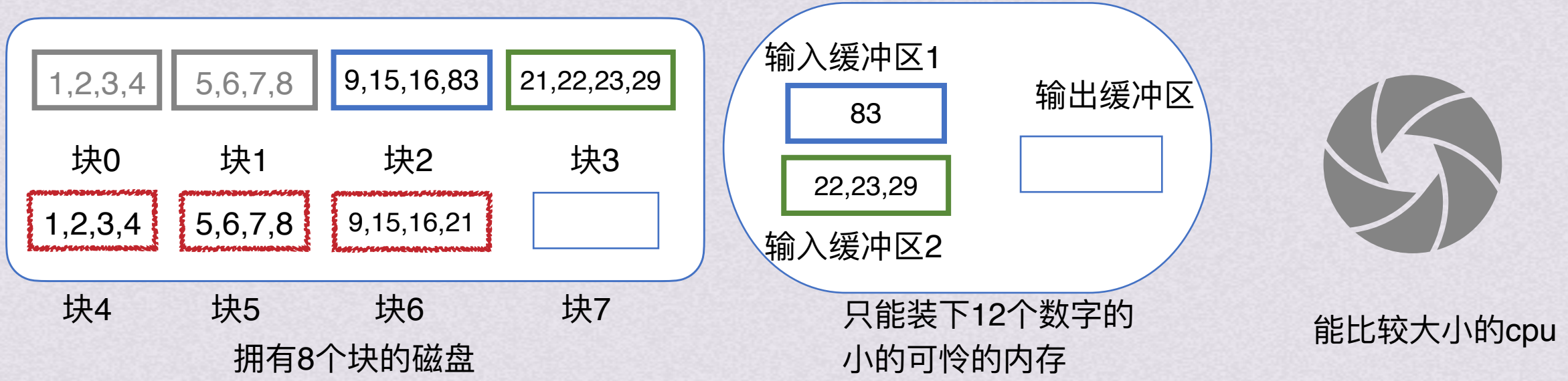
- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段 → 归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

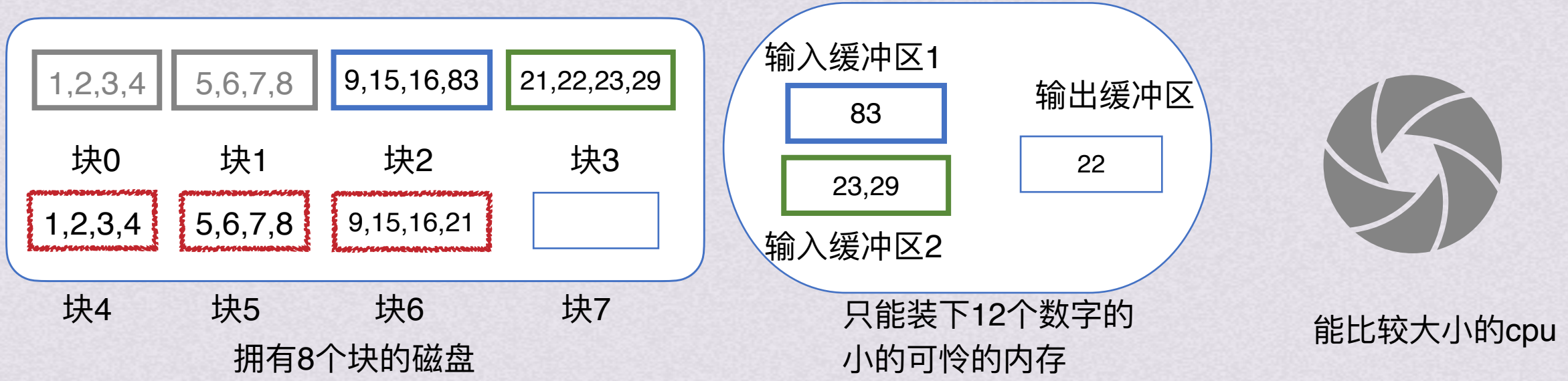
- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段 → 归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

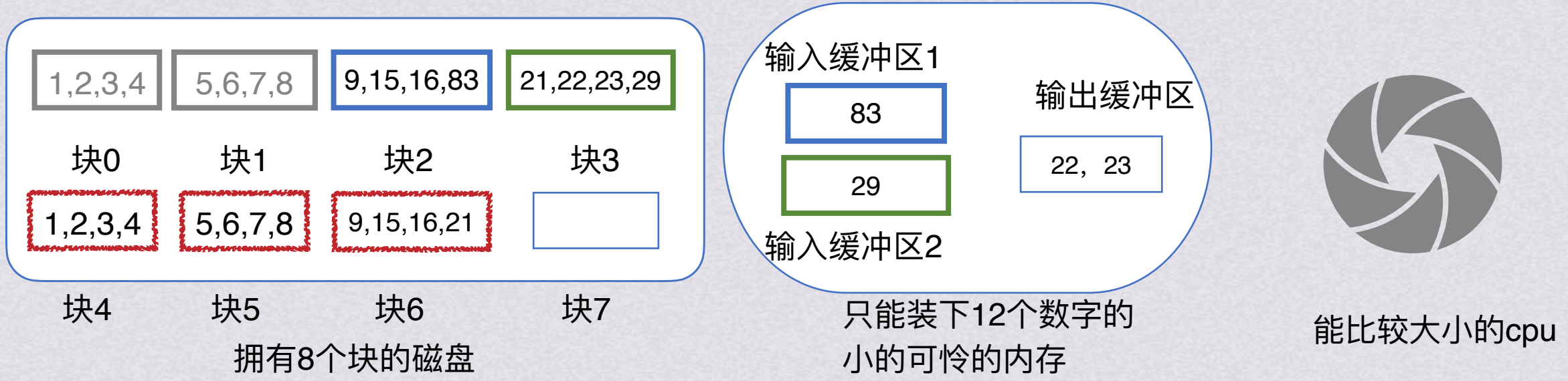
- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段 → 归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

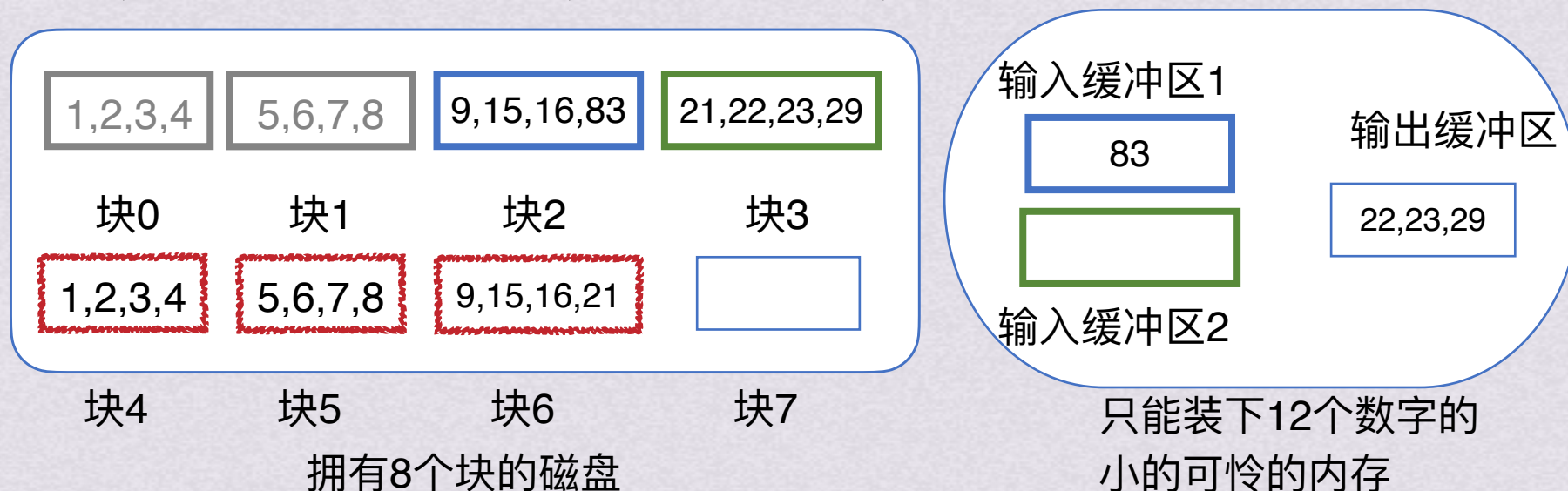
生成初始归并段 → 归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段



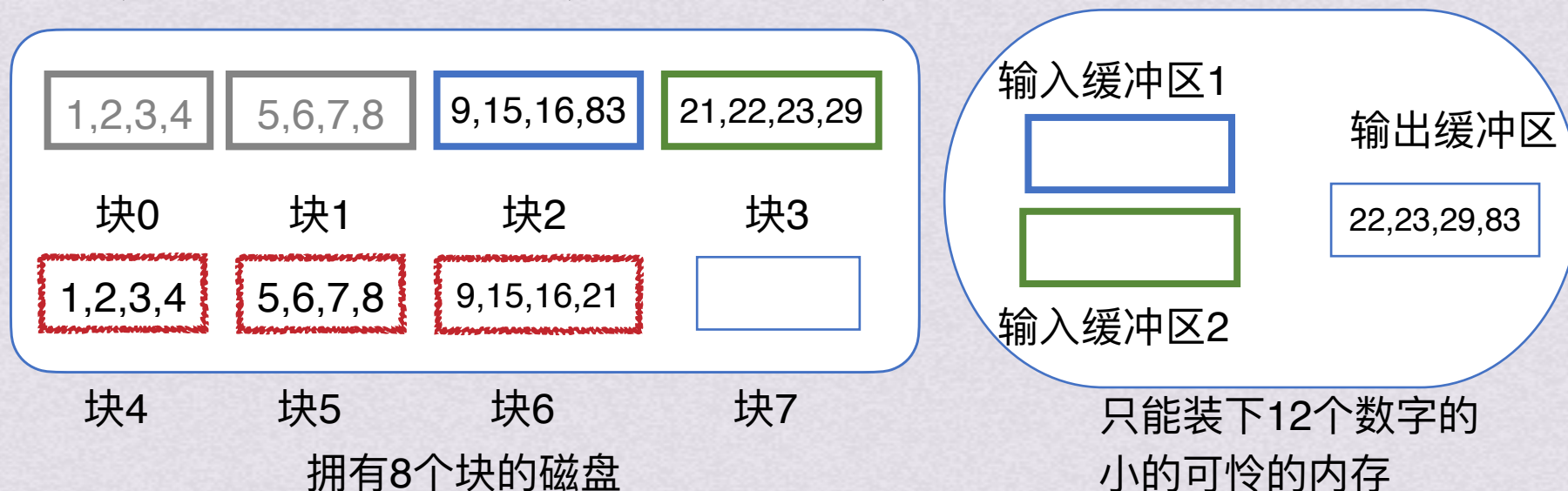
归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



1.cpu只能访问直接访问内存，不能访问外存

2.内存只能装得下12个数字

3.每次访问外存都需要大量时间

这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段

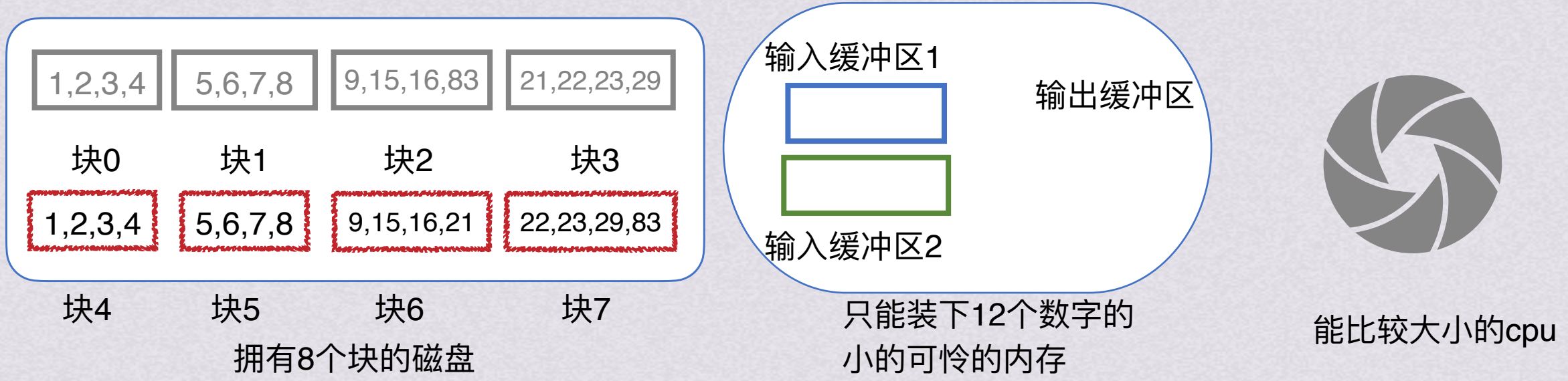


归并排序（假设二路归并）

外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



- 1.cpu只能访问直接访问内存，不能访问外存
 - 2.内存只能装得下12个数字
 - 3.每次访问外存都需要大量时间
- 这种情况下，该如何对16个数字排序？

- 1.把块放入内存
- 2.用之前学过的内部排序算法排序好
- 3.写回外存
- 4.对其余块一样处理

生成初始归并段 → 归并排序（假设二路归并）



外部排序

指待排序的记录数目很大，无法一次性调入内存，整个排序过程就必须借用外存分批调入内存才能完成。

按照可用内存的大小分批次将外存上的文件分成若干子文件，将其依次读入内存并利用有效的内部排序方法对它们进行排序得到**归并段**（有序的子文件），在对**归并段**进行逐趟归并，使其逐渐由小增大，直到获得整个有序文件



将内存无法一口气读入的数据量排序完成



外部排序

基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

假设一个含有10000个记录的文件
通过10次内部排序得到10个初始归并段R1~R10





外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

假设一个含有10000个记录的文件
通过10次内部排序得到10个初始归并段R1~R10





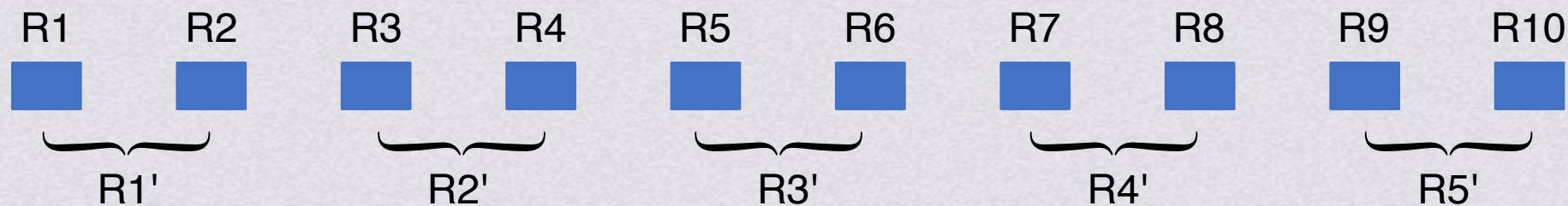
外部排序基本思想

1.通过内部排序获得初始归并段

2.将初始归并段进行归并

假设一个含有10000个记录的文件

通过10次内部排序得到10个初始归并段R1~R10





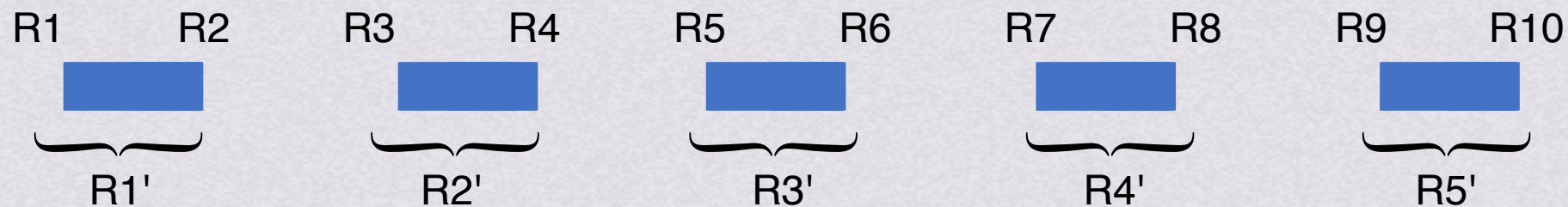
外部排序基本思想

1.通过内部排序获得初始归并段

2.将初始归并段进行归并

假设一个含有10000个记录的文件

通过10次内部排序得到10个初始归并段R1~R10

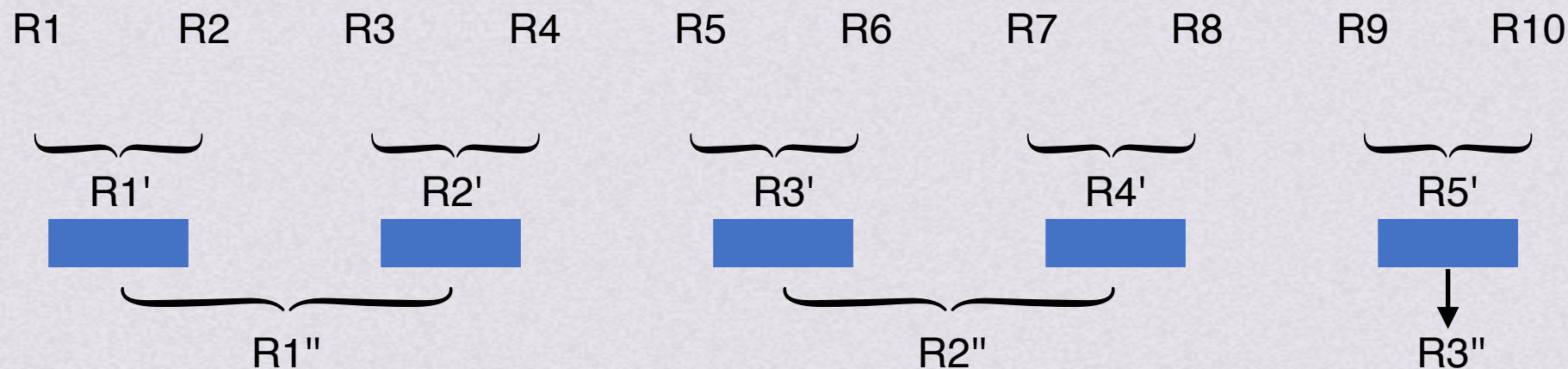




外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

假设一个含有10000个记录的文件
通过10次内部排序得到10个初始归并段R1~R10





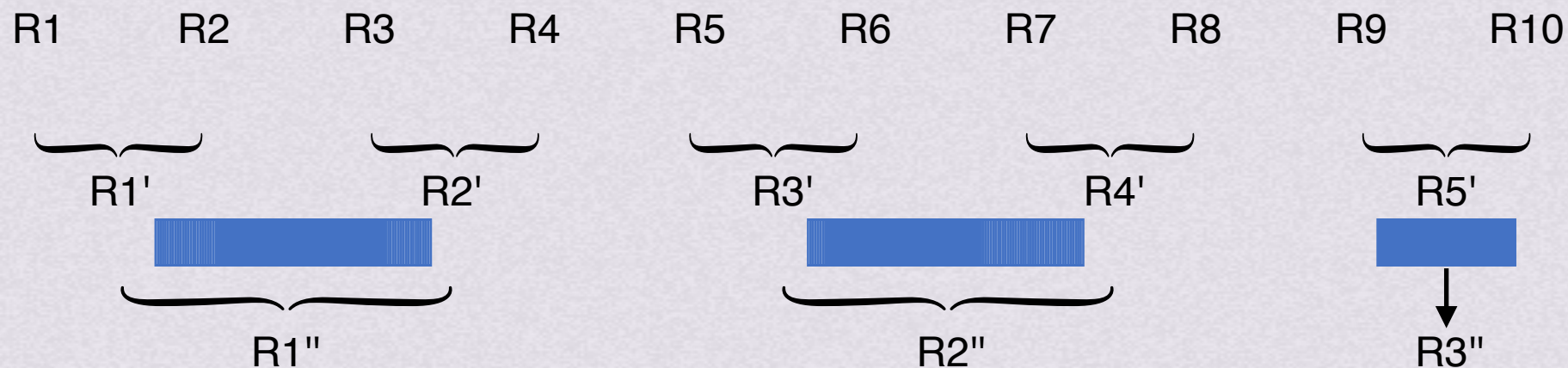
外部排序基本思想

1.通过内部排序获得初始归并段

2.将初始归并段进行归并

假设一个含有10000个记录的文件

通过10次内部排序得到10个初始归并段R1~R10





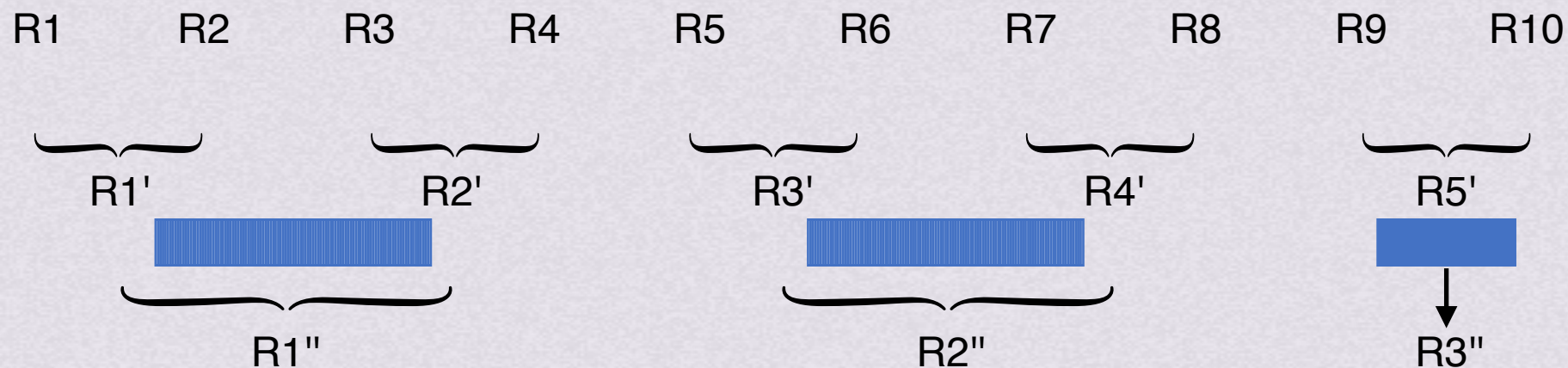
外部排序基本思想

1.通过内部排序获得初始归并段

2.将初始归并段进行归并

假设一个含有10000个记录的文件

通过10次内部排序得到10个初始归并段R1~R10





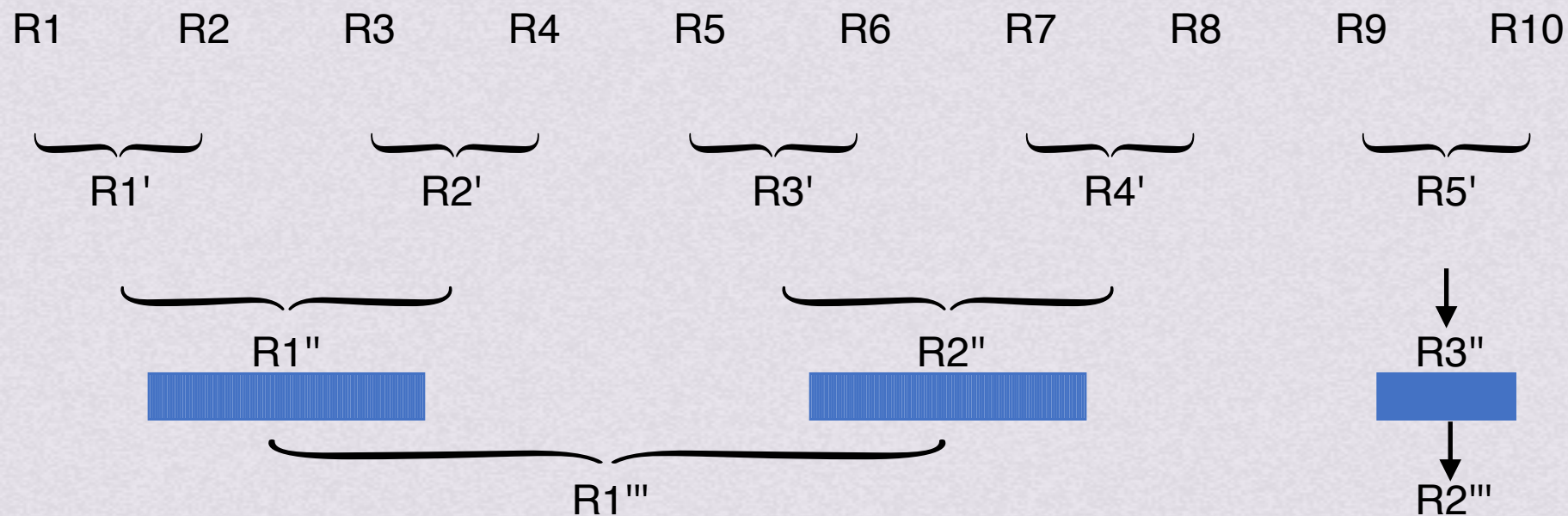
外部排序基本思想

1.通过内部排序获得初始归并段

2.将初始归并段进行归并

假设一个含有10000个记录的文件

通过10次内部排序得到10个初始归并段R1~R10





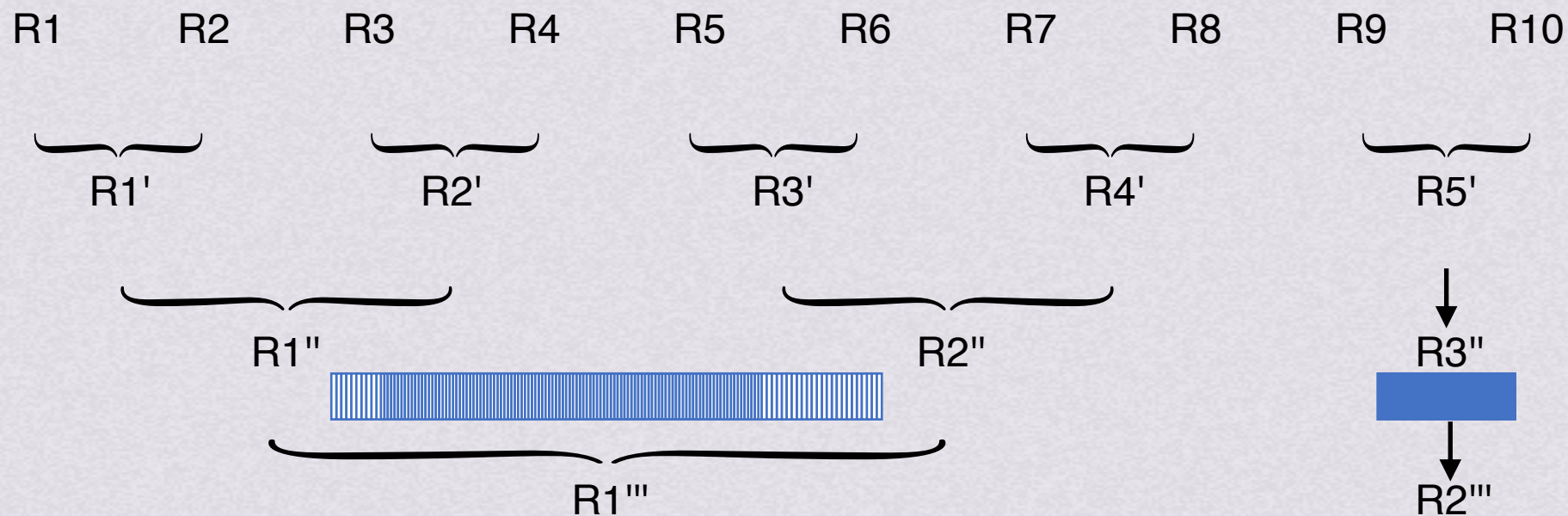
外部排序基本思想

1.通过内部排序获得初始归并段

2.将初始归并段进行归并

假设一个含有10000个记录的文件

通过10次内部排序得到10个初始归并段R1~R10

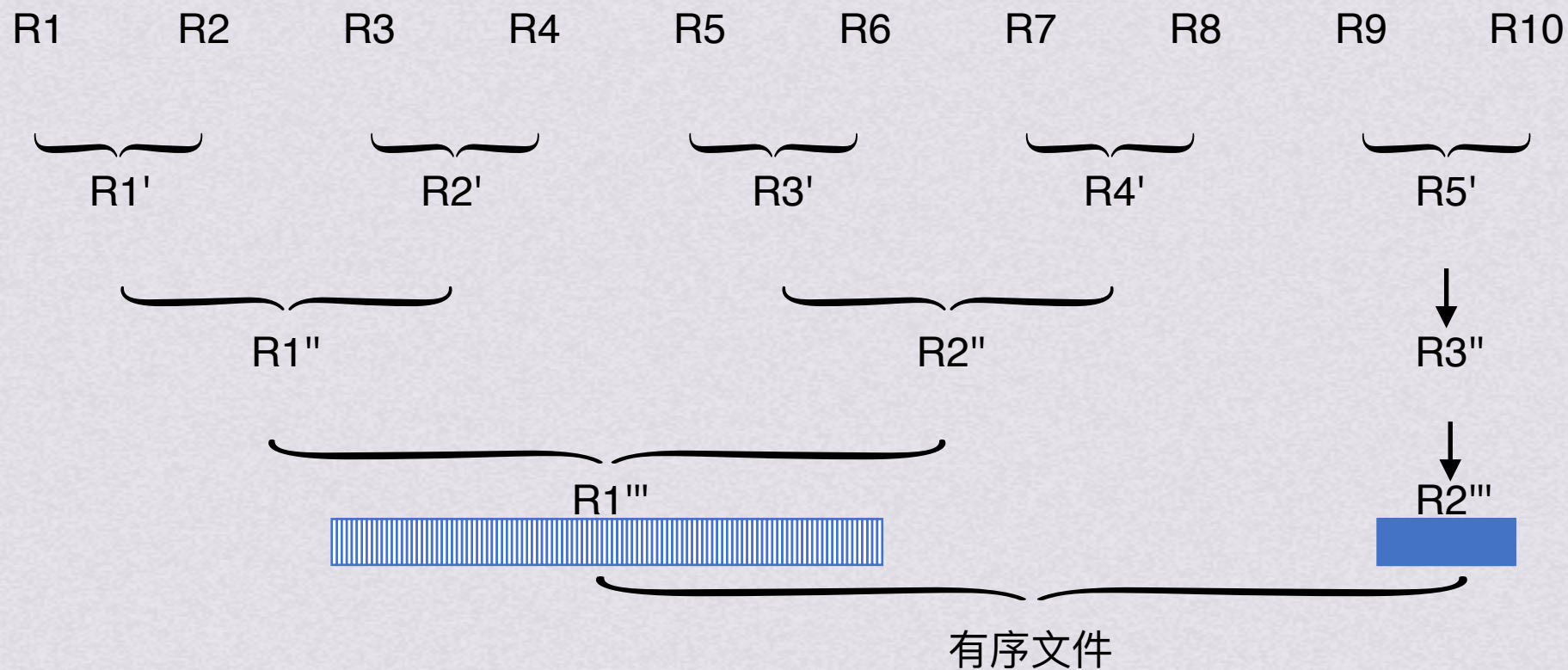




外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

假设一个含有10000个记录的文件
通过10次内部排序得到10个初始归并段R1~R10





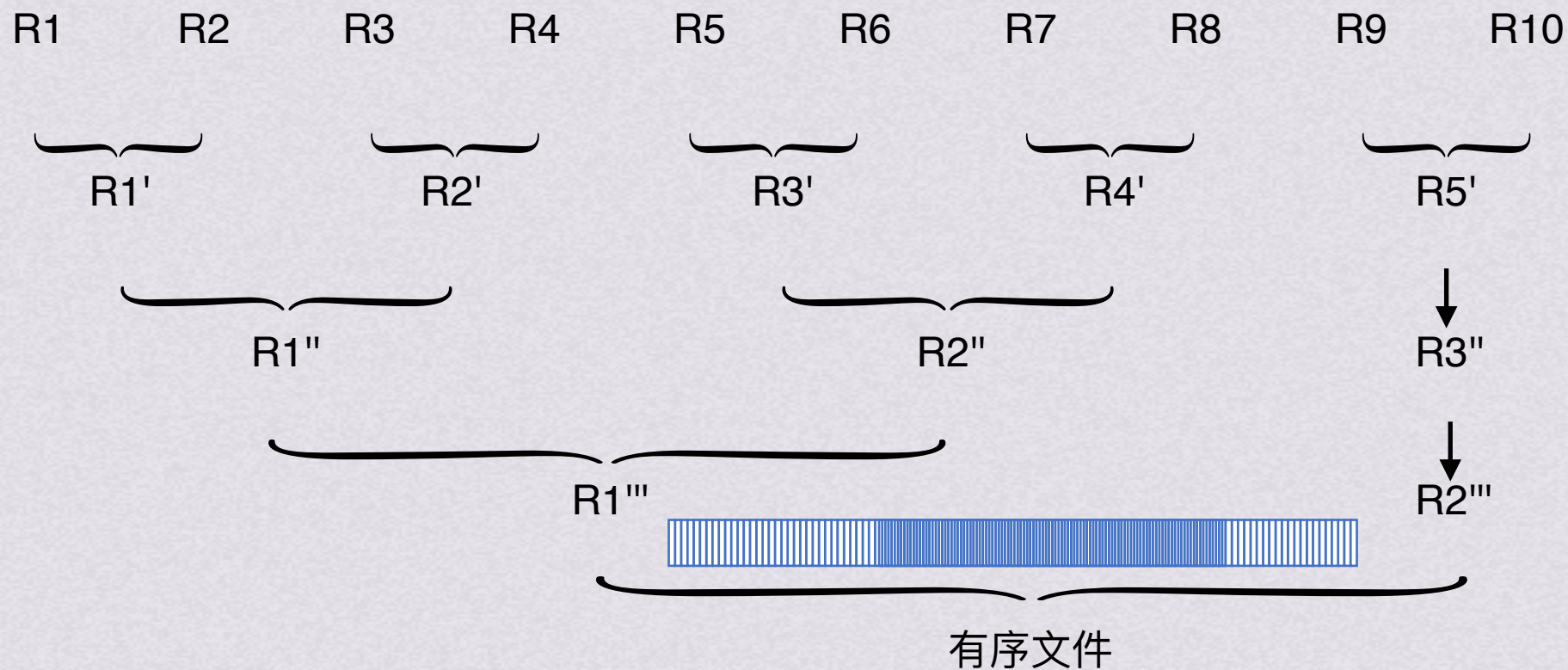
外部排序基本思想

1.通过内部排序获得初始归并段

2.将初始归并段进行归并

假设一个含有10000个记录的文件

通过10次内部排序得到10个初始归并段R1~R10





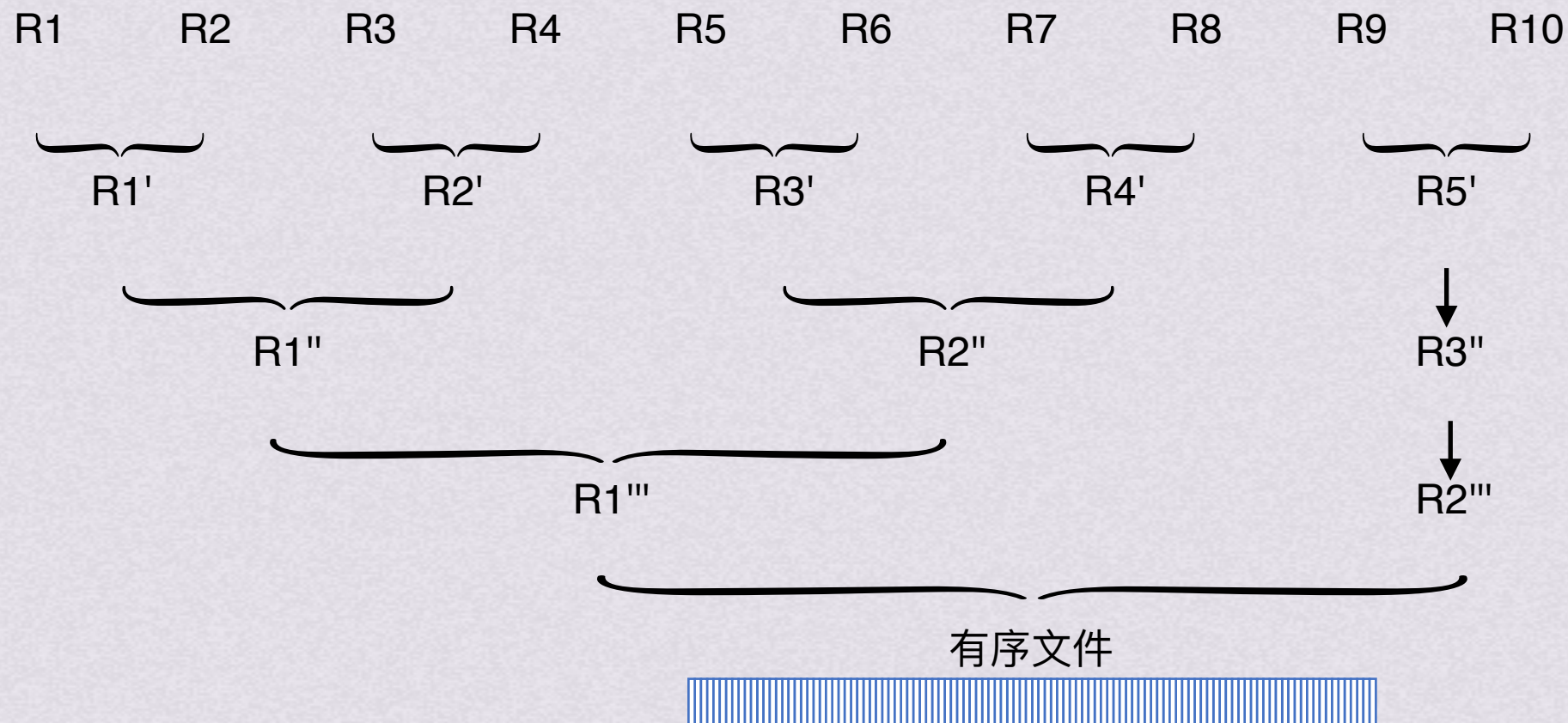
外部排序基本思想

1.通过内部排序获得初始归并段

2.将初始归并段进行归并

假设一个含有10000个记录的文件

通过10次内部排序得到10个初始归并段R1~R10

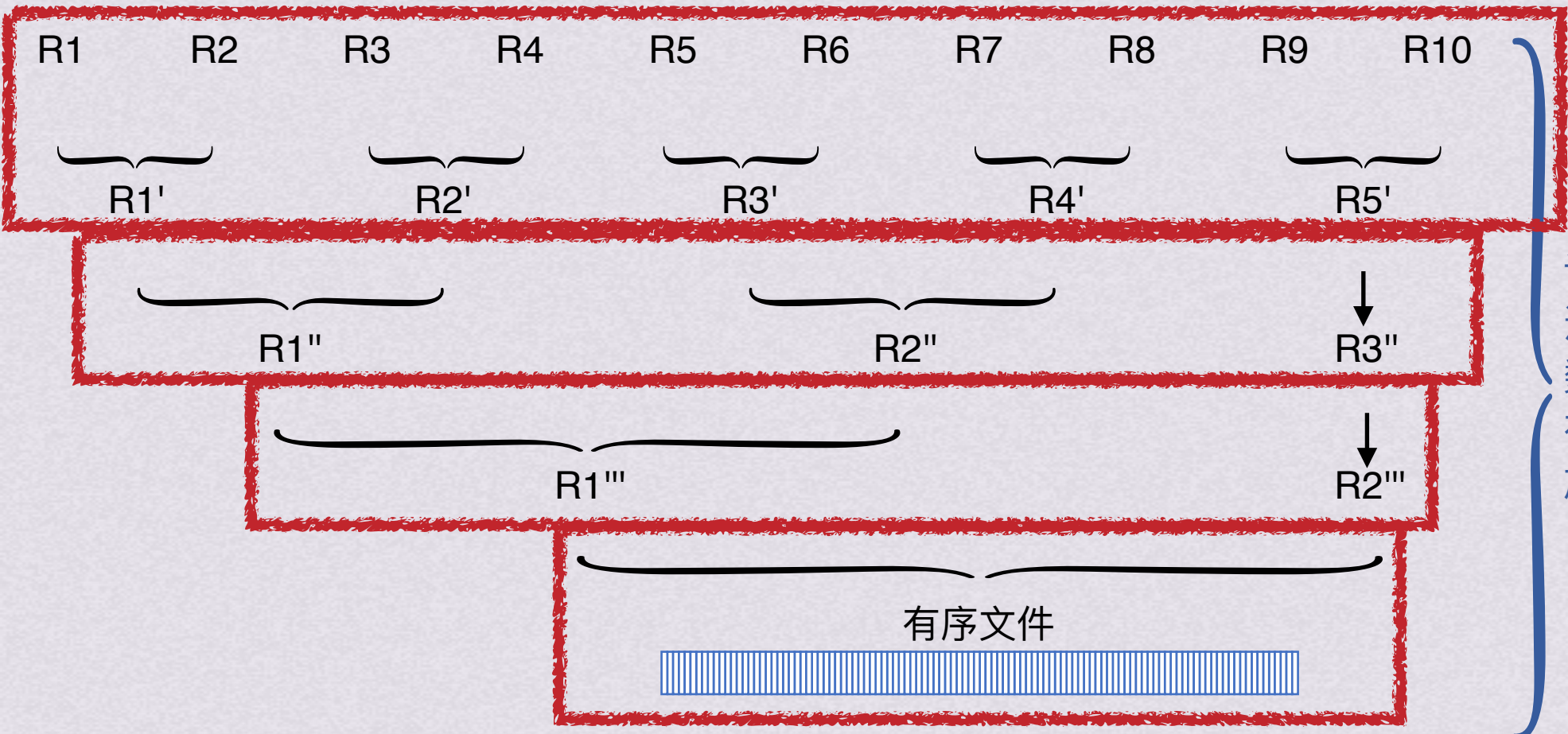


外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

假设一个含有10000个记录的文件
通过10次内部排序得到10个初始归并段R1~R10

对外存上的信息的读/写是以物理块为单位的
若假设每个物理块可容纳200个记录，一趟归并需50次读，50次写



该过程不仅需要归并排序
还涉及对外存的读/写，因为
数据量太大，不可能将两个
有序段及归并结果段同时存
放在内存中

4趟归并加上内部排序时所需进行的读/写使外部排序中总共需进行500次读/写



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

外部排序总时间=内部排序时间+外存信息读/写时间+内部归并时间

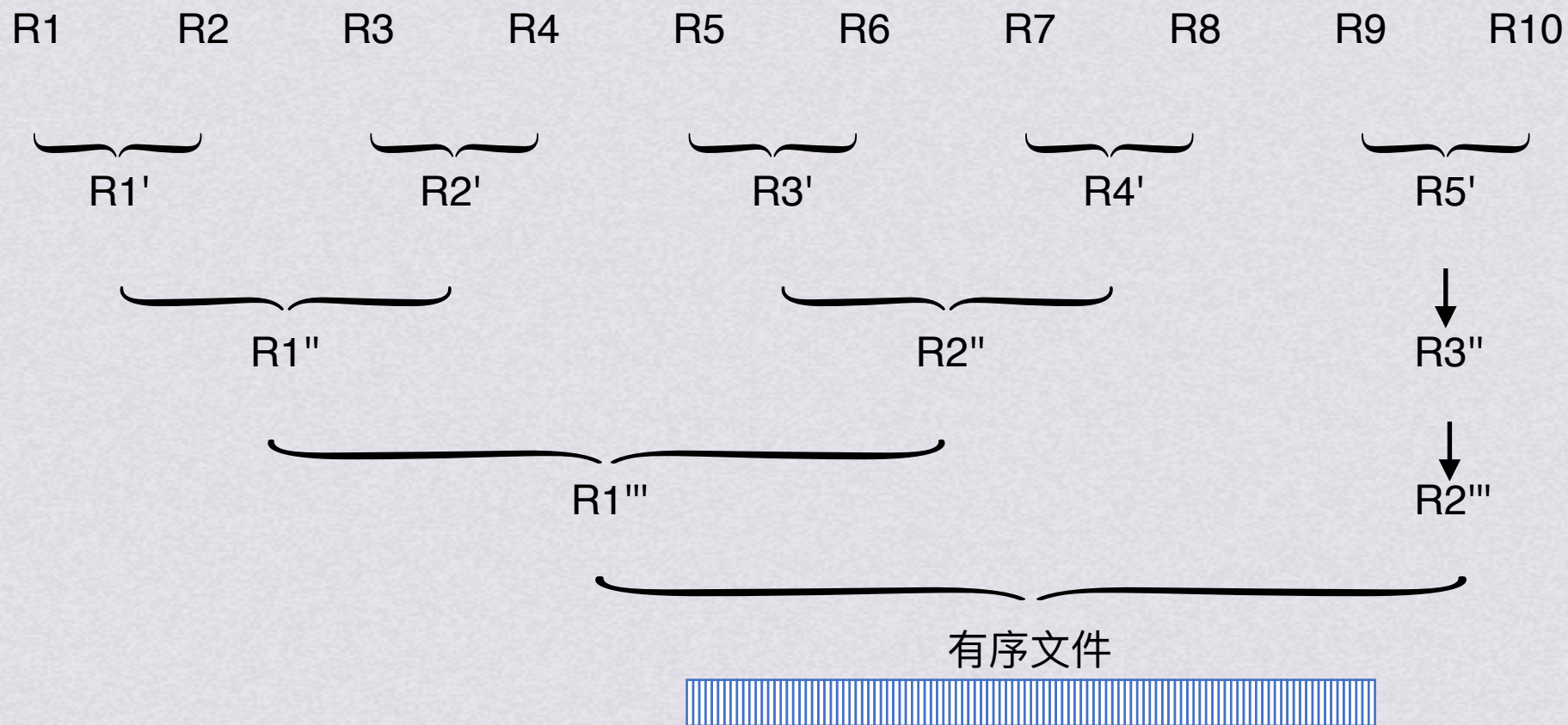
由于外存的读写时间远大于内部排序、内部归并时间，因此要减少io次数



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

若进行五路归并

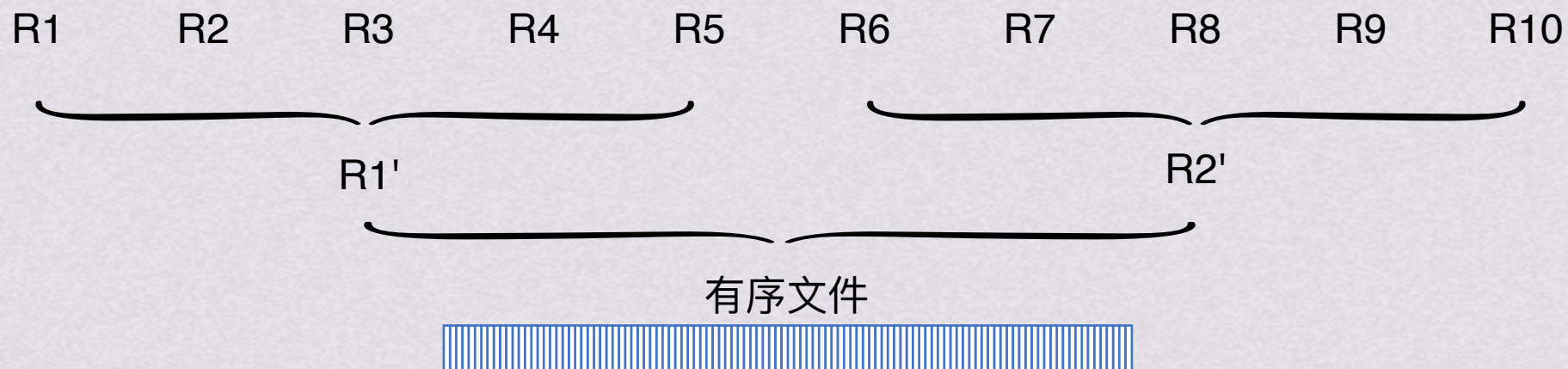




外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

若进行五路归并



只需要两趟归并即完成排序
io次数减少



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

一般情况下，对 m 个初始归并段进行 k 路平衡归并时，归并趟数

$$s = \lceil \log_k m \rceil$$

想减少归并趟数 s ，可以增加归并路数 k 或者减少初始归并段个数 m



外部排序基本思想

1.通过内部排序获得初始归并段

2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$

减少初始归并段个数 m

增加归并路数 k



归并路数过多会导致缓冲区太多

内部归并时间随 k 增长而增长，最终抵消由于增大 k 缩短的io时间收益

引入败者树

可使在 k 个记录中选出关键字最小的记录仅需进行 $\lceil \log_2 k \rceil$ 次比较



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$

减少初始归并段个数 m

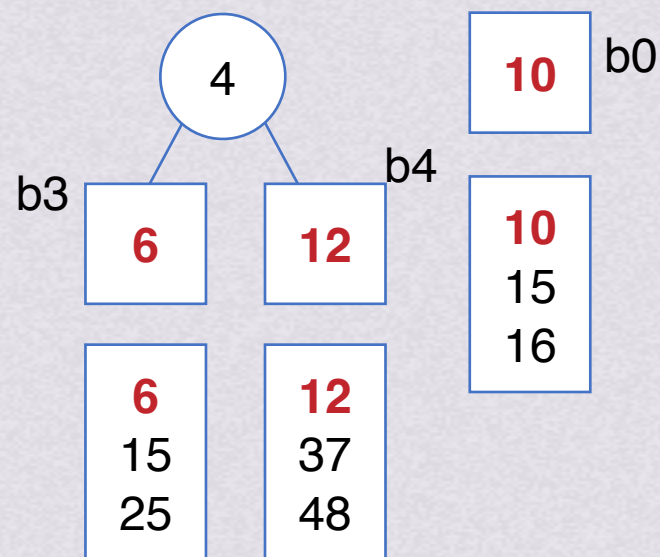
增加归并段个数 k

内部归并时间随 k 增长而增长

最终抵消由于增大 k 缩短 io 时间收益

引入败者树

败者树



胜者继续比较，败者留下姓名（组号）



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$

减少初始归并段个数 m

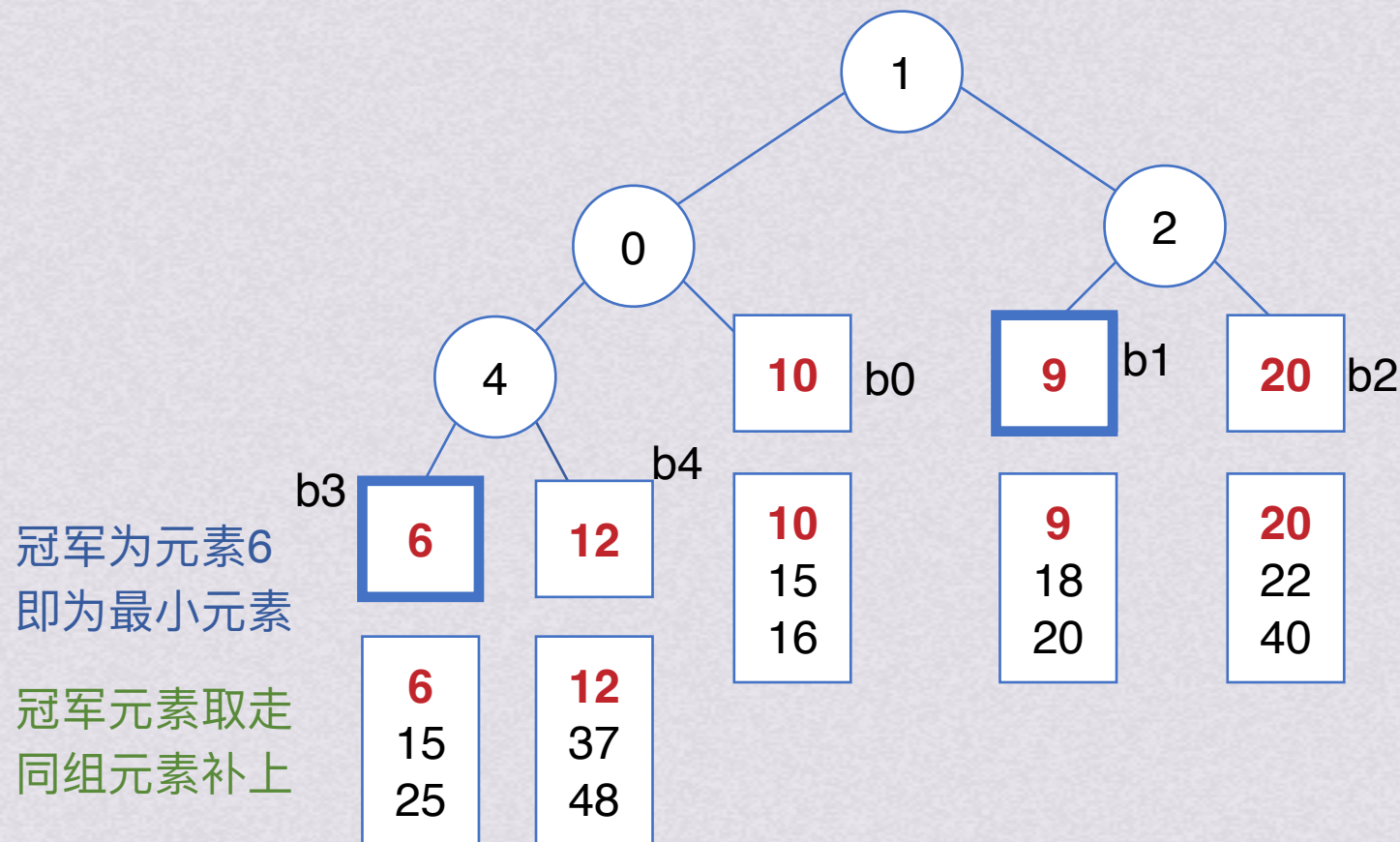
增加归并段个数 k

内部归并时间随 k 增长而增长

最终抵消由于增大 k 缩短 io 时间收益

引入败者树

败者树



胜者继续比较，败者留下姓名（组号）



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$

减少初始归并段个数 m

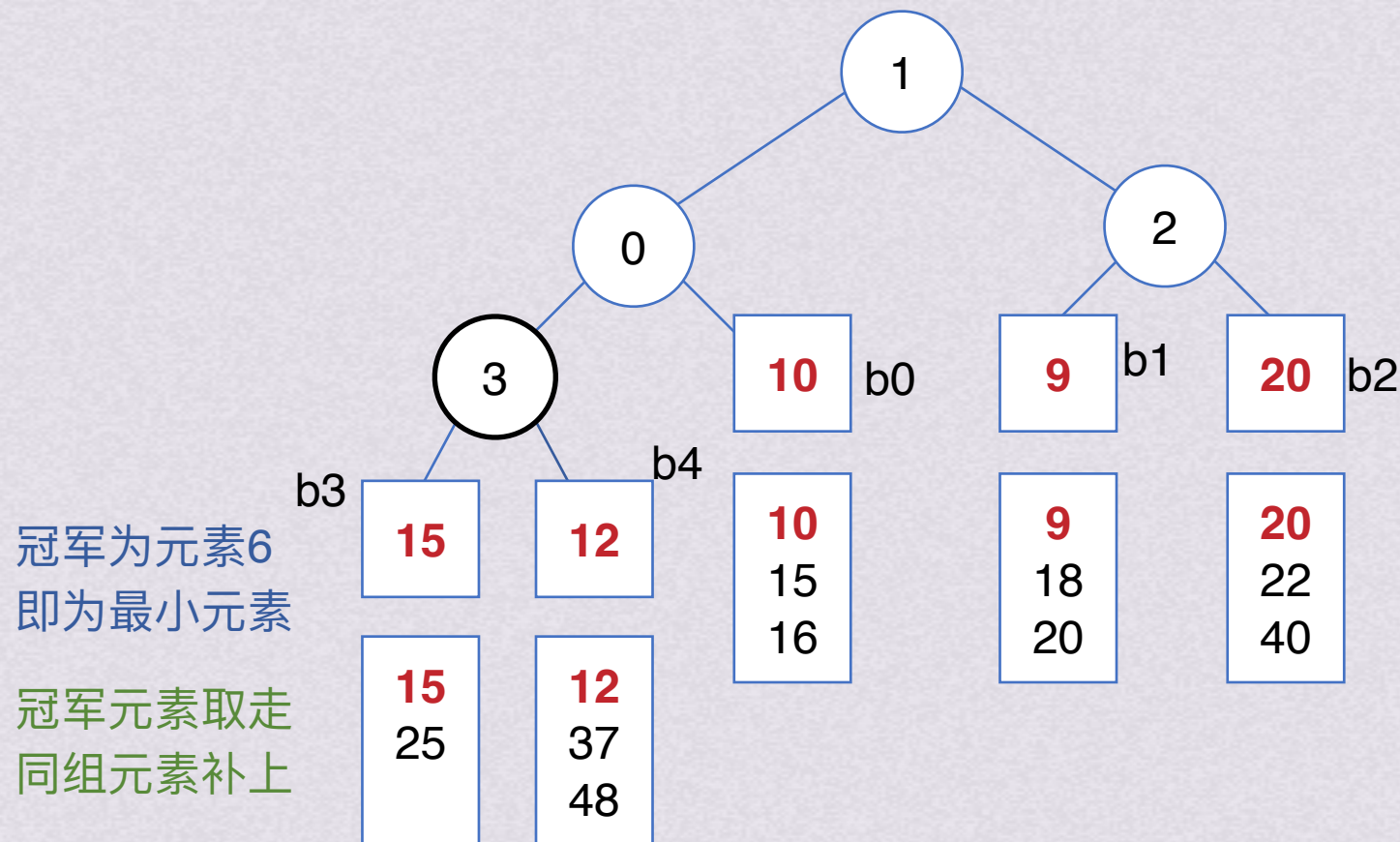
增加归并段个数 k

内部归并时间随 k 增长而增长

最终抵消由于增大 k 缩短 io 时间收益

引入败者树

败者树



胜者继续比较，败者留下姓名（组号）



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$

减少初始归并段个数 m

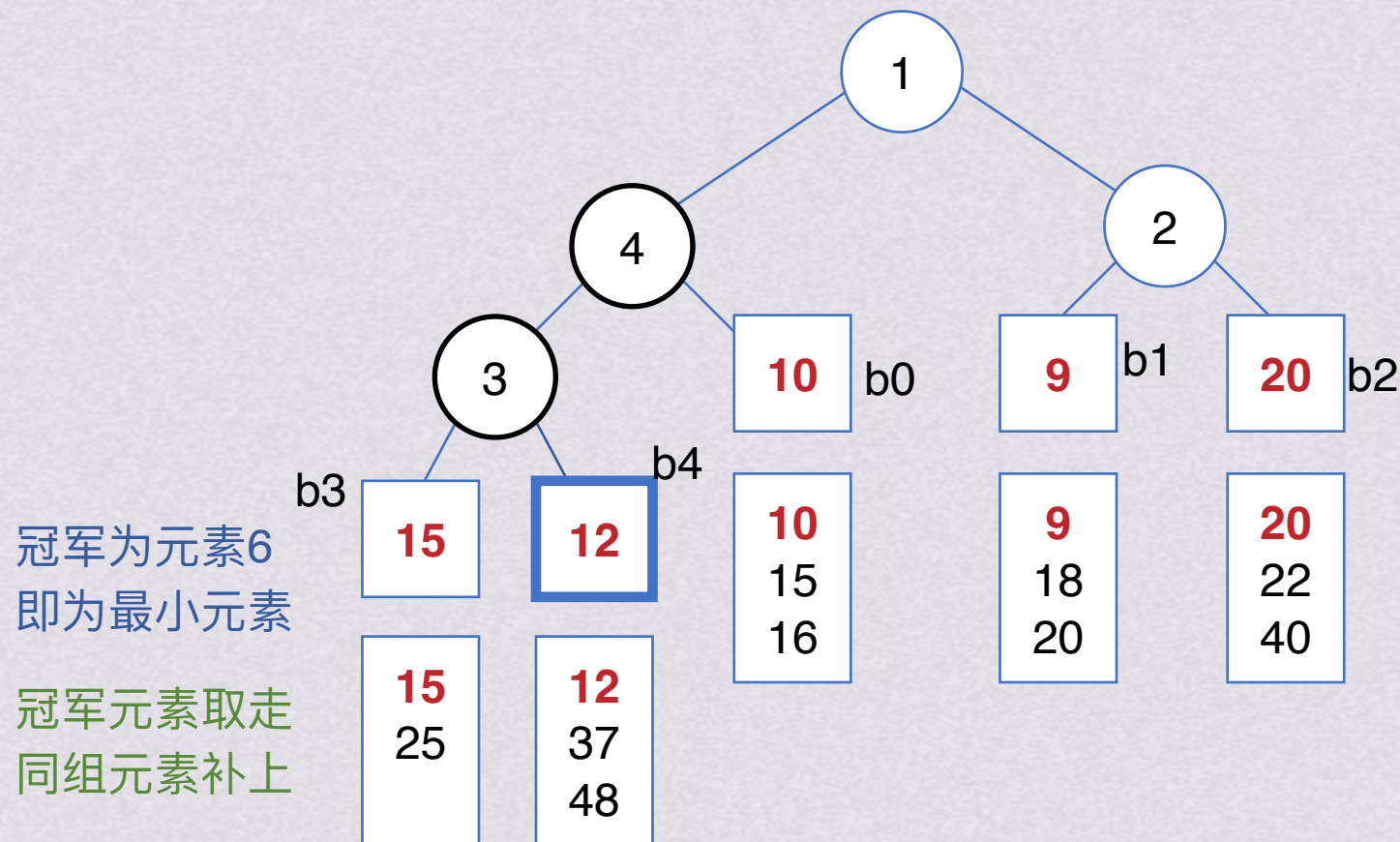
增加归并段个数 k

内部归并时间随 k 增长而增长

最终抵消由于增大 k 缩短 io 时间收益

引入败者树

败者树





外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$

减少初始归并段个数 m

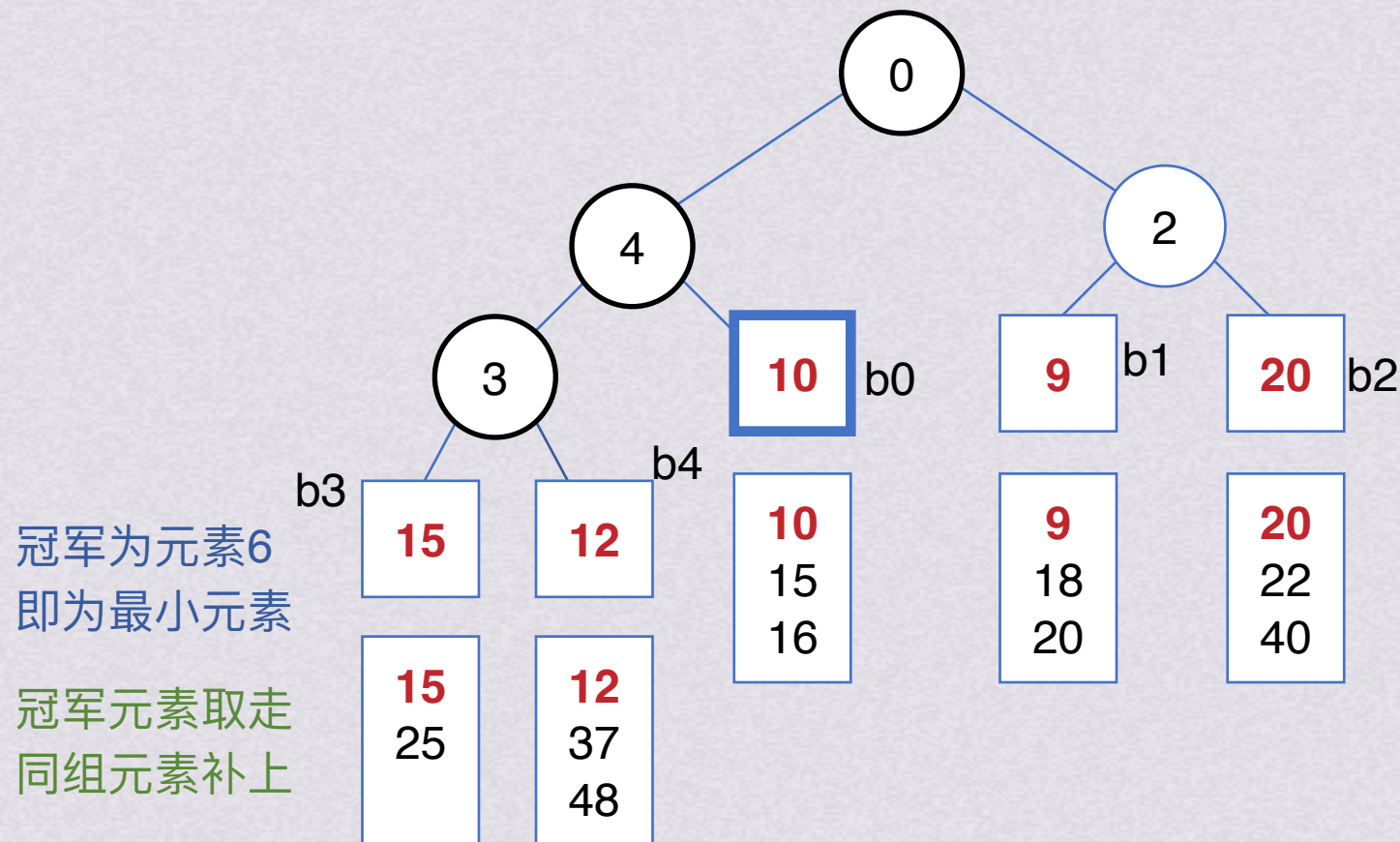
增加归并段个数 k

内部归并时间随 k 增长而增长

最终抵消由于增大 k 缩短 io 时间收益

引入败者树

败者树



冠军为元素6
即为最小元素

冠军元素取走
同组元素补上

胜者继续比较，败者留下姓名（组号）



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$

减少初始归并段个数 m

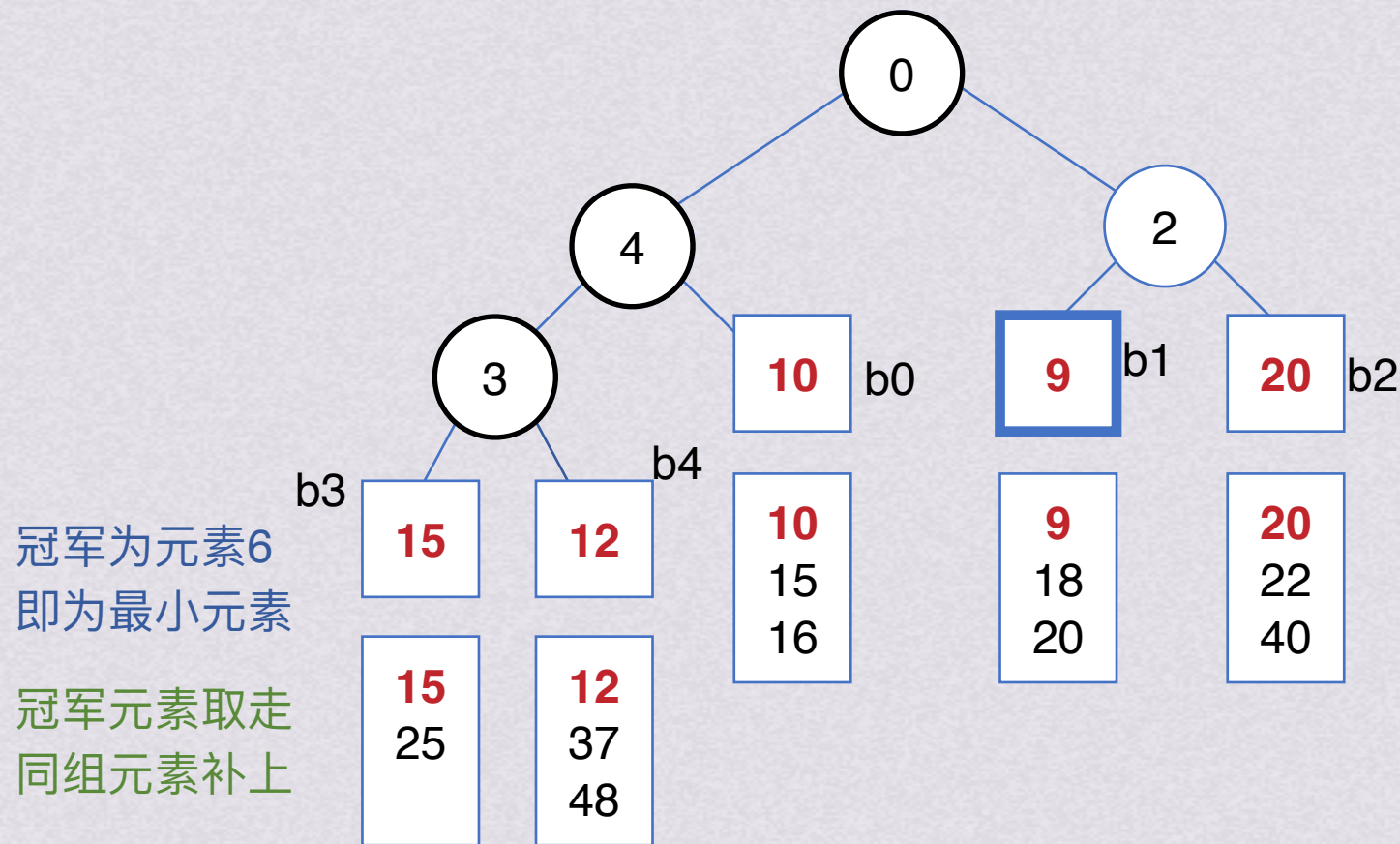
增加归并段个数 k

内部归并时间随 k 增长而增长

最终抵消由于增大 k 缩短 io 时间收益

引入败者树

败者树





外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$

减少初始归并段个数 m

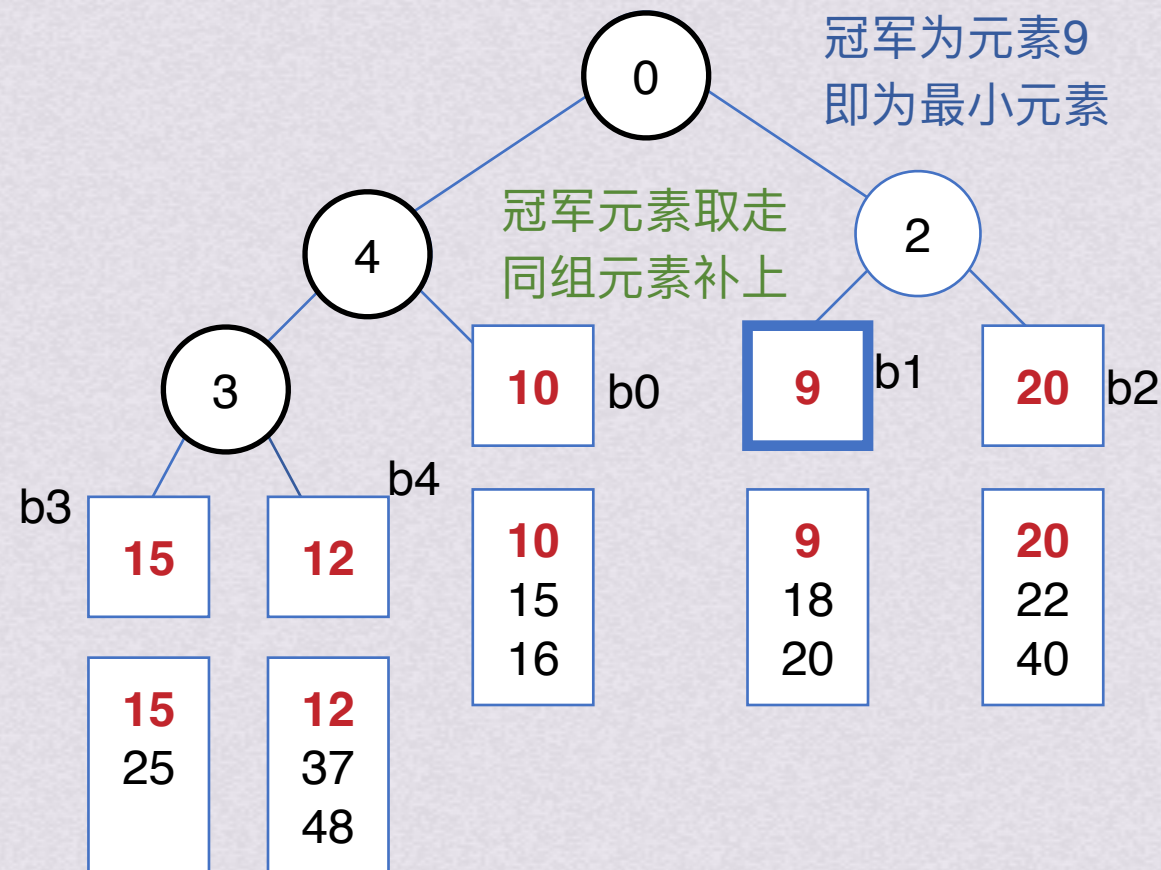
增加归并段个数 k

内部归并时间随 k 增长而增长

最终抵消由于增大 k 缩短 io 时间收益

引入败者树

败者树



胜者继续比较，败者留下姓名（组号）



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$

减少初始归并段个数 m

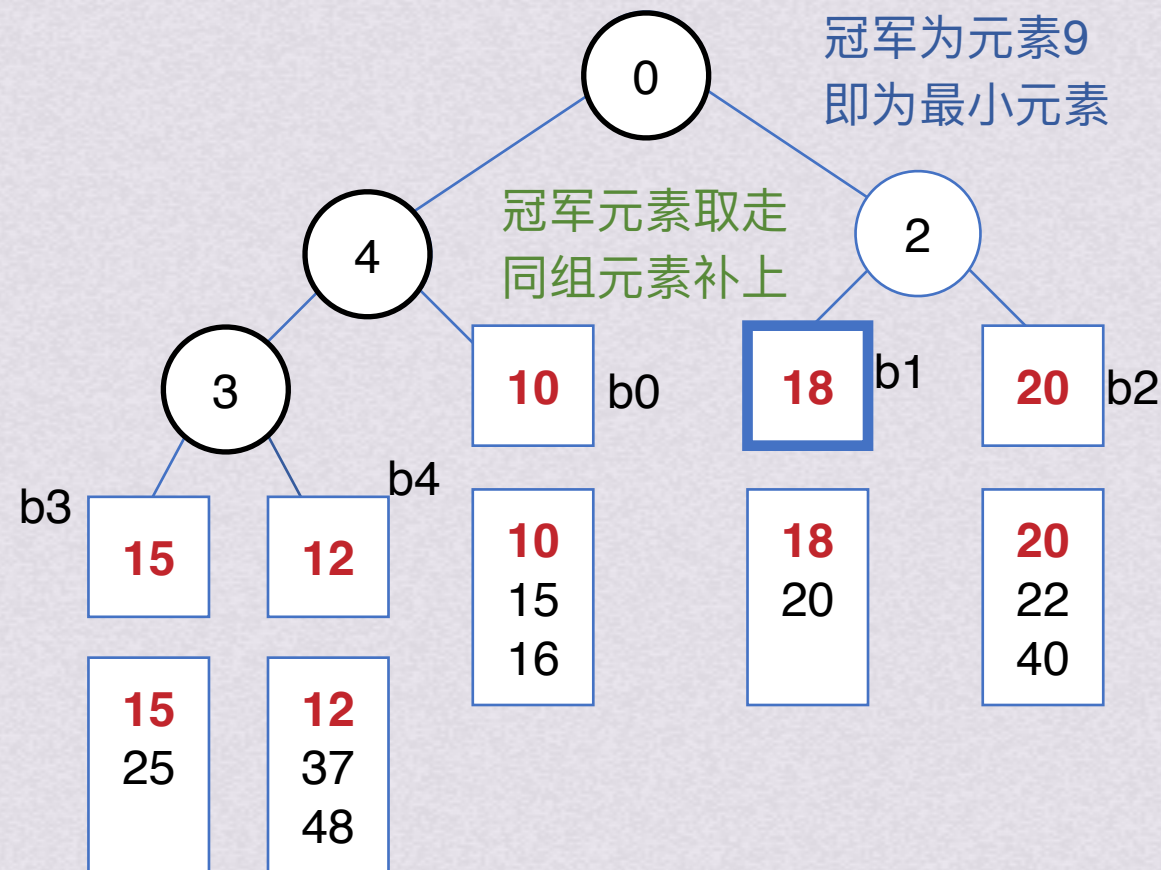
增加归并段个数 k

内部归并时间随 k 增长而增长

最终抵消由于增大 k 缩短 io 时间收益

引入败者树

败者树



胜者继续比较，败者留下姓名（组号）



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$ 减少初始归并段个数 m

成功解决掉了一半的问题，另一半怎么办？

增加归并段个数 k

内部归并时间随 k 增长而增长
最终抵消由于增大 k 缩短 io 时间收益
引入败者树



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$ 减少初始归并段个数 m

成功解决掉了一半的问题，另一半怎么办？

归并段个数 = 总数据量（外部文件中记录数） / 初始归并段中数据量（归并段中记录数）

初始归并段中记录数依赖于内存大小，所以必须探索新的办法，于是发明了**置换-选择排序**



外部排序基本思想

- 1.通过内部排序获得初始归并段
- 2.将初始归并段进行归并

归并趟数 $s = \lceil \log_k m \rceil$

减少初始归并段个数m

依赖于内存

置换-选择排序生成初始归并段

置换-选择排序

- 步骤：
- 1.从待排序文件FI输入记录到工作区WA
 - 2.从工作区WA中选出其中关键字取最小值的记录，记为MINIMAX
 - 3.将MINIMAX记录输出到输出文件FO
 - 4.若FI不空，从FI输入下一个记录到WA
 - 5.从WA中所有关键字比MINIMAX记录的关键字大的记录中选出最小关键字记录，作为新的MINIMAX
 - 6.重复3~5，直到WA中选不出新的MINIMAX，由此得到一个初始归并段，输出一个归并段的结束标志到FO
 - 7.重复2~6，直到WA空，得到全部初始归并段



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

51	49	39	46	38	29	14	61	15	30	1	48	52	3	63	27	4	13	89	24	46	58	33	76
----	----	----	----	----	----	----	----	----	----	---	----	----	---	----	----	---	----	----	----	----	----	----	----



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

工作区WA

输入文件FI

51	49	39	46	38	29	14	61	15	30
----	----	----	----	----	----	----	----	----	----



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

工作区WA

输入文件FI

51	49	39	46	38	29
----	----	----	----	----	----

14	61	15	30	1	48	52	3	63	27
----	----	----	----	---	----	----	---	----	----

在内存中选择最小的



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29

工作区WA

51 49 39 46 38

在内存中选择最小的

输入文件FI

14 61 15 30 1 48 52 3 63 27

外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO	工作区WA	输入文件FI
29	51 49 39 46 38 14 在内存中选择最小的 但要比29大	61 15 30 1 48 52 3 63 27 4



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29 38

工作区WA

51 49 39 46 14

输入文件FI

61 15 30 1 48 52 3 63 27 4

在内存中选择最小的
但要比29大



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29 38

工作区WA

51 49 39 46 61 14

输入文件FI

15 30 1 48 52 3 63 27 4 13 8

在内存中选择最小的
但要比38大



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29 38 39

工作区WA

51 49 46 61 14

输入文件FI

15 30 1 48 52 3 63 27 4 13 8

在内存中选择最小的
但要比38大



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29 38 39

工作区WA

51 49 15 46 61 14

输入文件FI

30 1 48 52 3 63 27 4 13 89

在内存中选择最小的
但要比39大



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29 38 39 46

工作区WA

51 49 15 61 14

输入文件FI

30 1 48 52 3 63 27 4 13 89

在内存中选择最小的
但要比39大



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29 38 39 46

工作区WA

51 49 15 30 61 14

输入文件FI

1 48 52 3 63 27 4 13 89 24 .

在内存中选择最小的
但要比46大



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29 38 39 46 49

工作区WA

51 15 30 61 14

输入文件FI

1 48 52 3 63 27 4 13 89 24 .

在内存中选择最小的
但要比46大



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29 38 39 46 49

工作区WA

51 1 15 30 61 14

输入文件FI

48 52 3 63 27 4 13 89 24 46

在内存中选择最小的
但要比49大



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29	38	39	46	49	51
----	----	----	----	----	----

工作区WA

1	15	30	61	14
---	----	----	----	----

在内存中选择最小的
但要比49大

输入文件FI

48	52	3	63	27	4	13	89	24	46
----	----	---	----	----	---	----	----	----	----



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29	38	39	46	49	51
----	----	----	----	----	----

工作区WA

48	1	15	30	61	14
----	---	----	----	----	----

在内存中选择最小的
但要比51大

输入文件FI

52	3	63	27	4	13	89	24	46	58
----	---	----	----	---	----	----	----	----	----



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29 38 39 46 49 51 61

工作区WA

48 1 15 30 14

在内存中选择最小的
但要比51大

输入文件FI

52 3 63 27 4 13 89 24 46 58



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段

输出文件FO

29	38	39	46	49	51	61
----	----	----	----	----	----	----

工作区WA

48	1	15	30	52	14
----	---	----	----	----	----

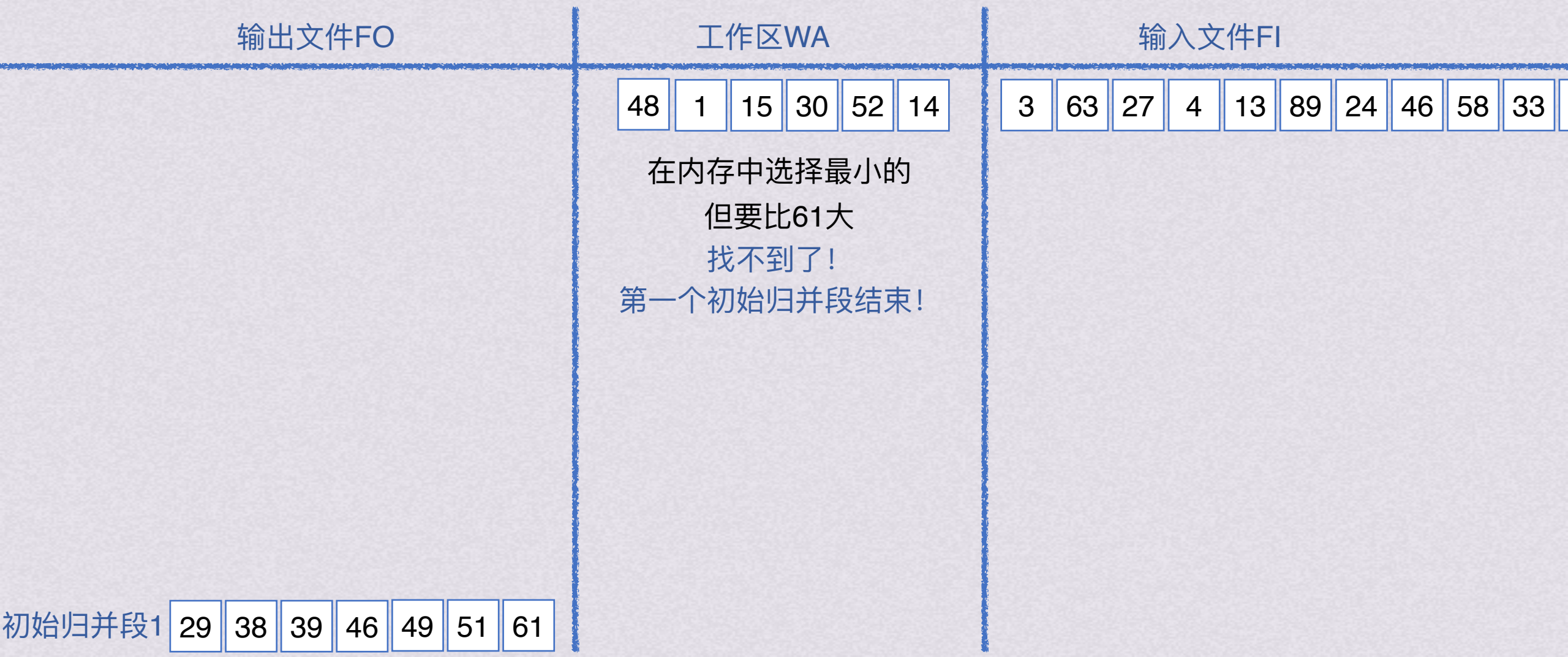
输入文件FI

3	63	27	4	13	89	24	46	58	33
---	----	----	---	----	----	----	----	----	----

在内存中选择最小的
但要比61大
找不到了！
第一个初始归并段结束！

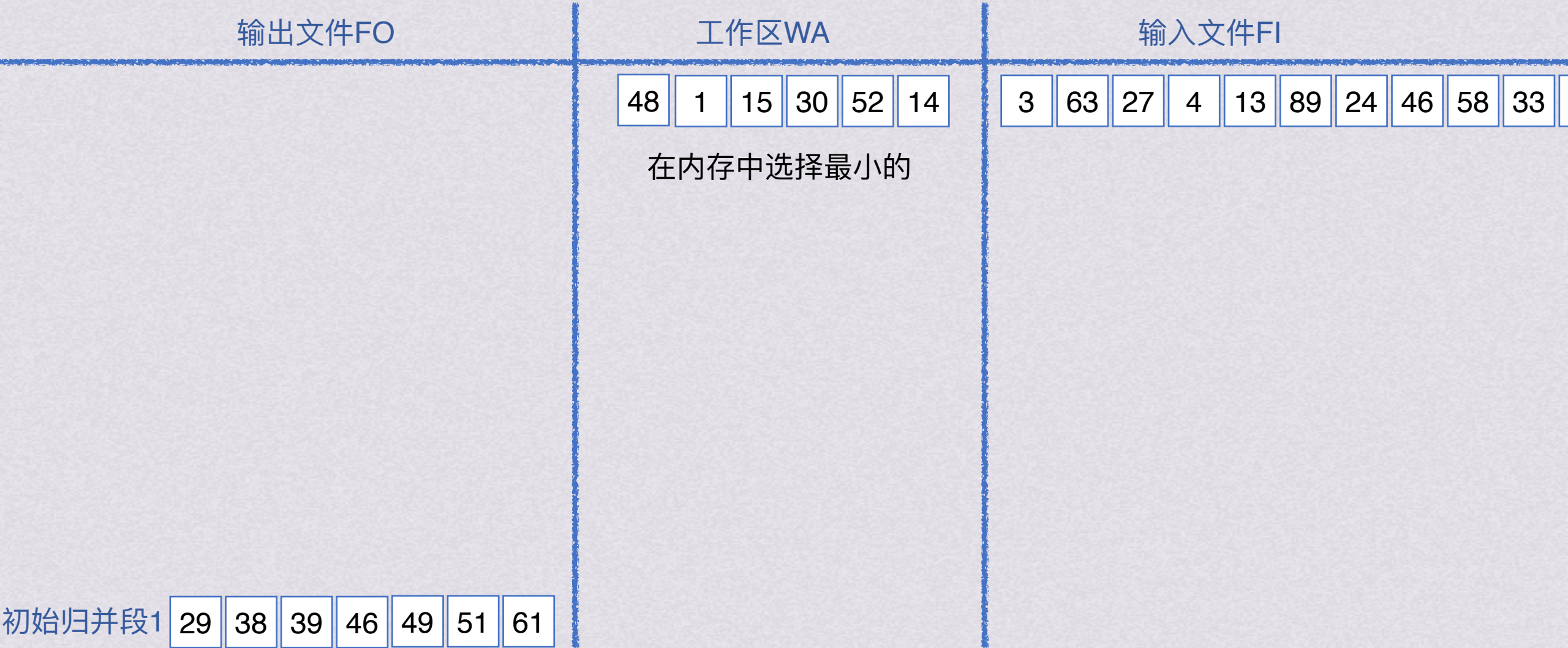
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



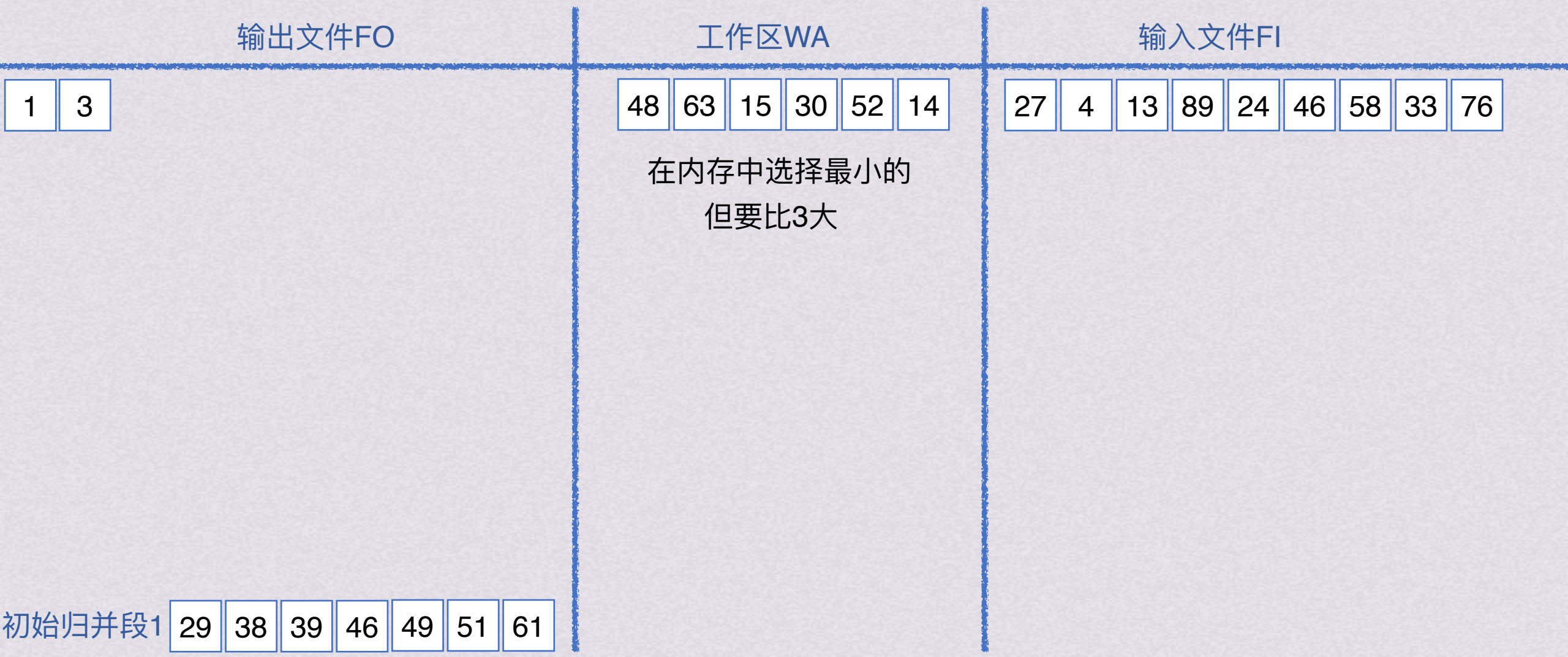
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



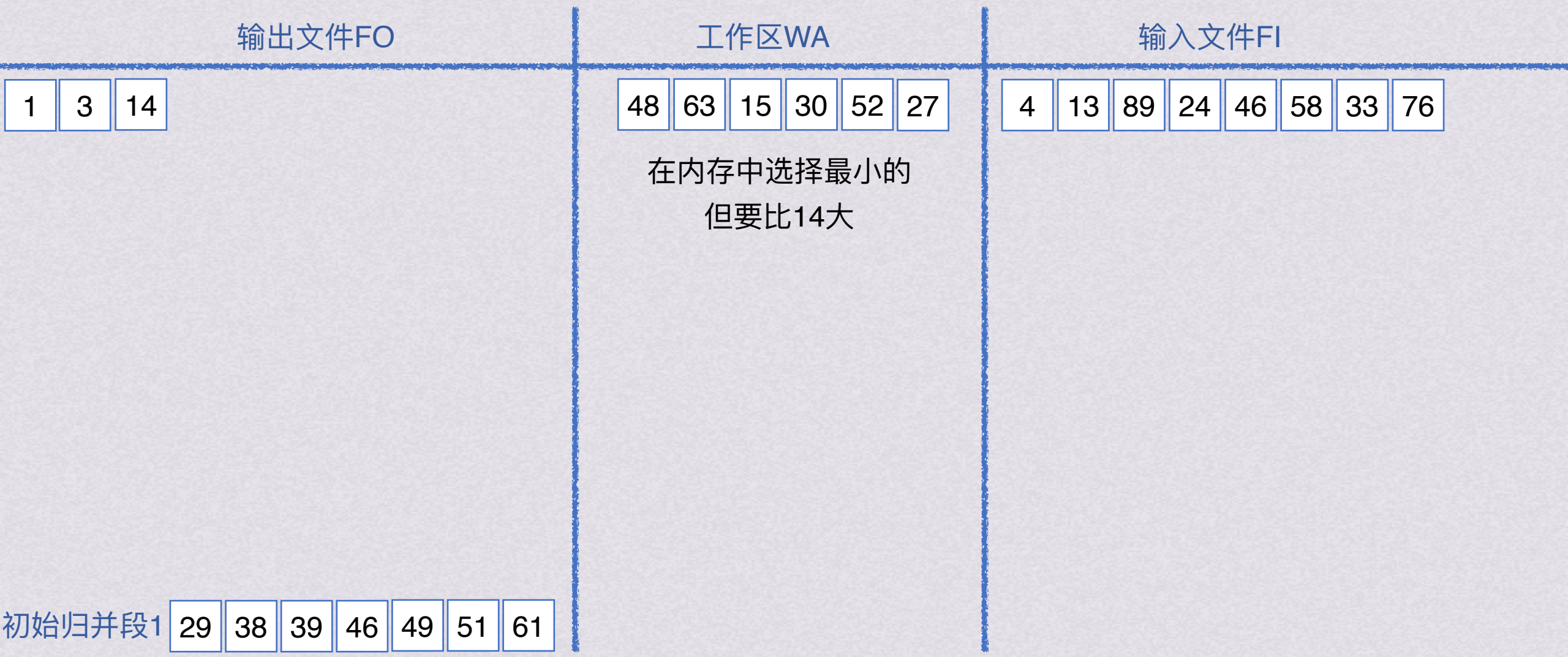
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



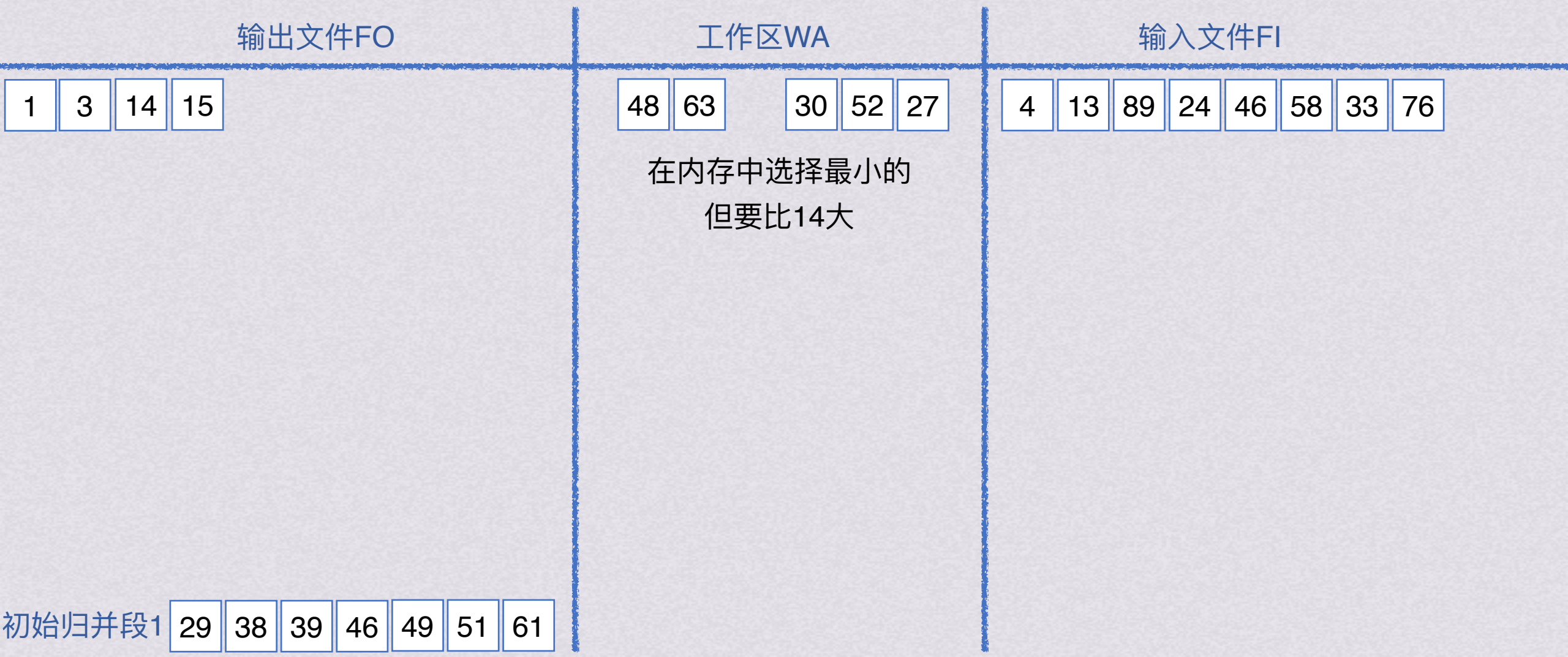
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



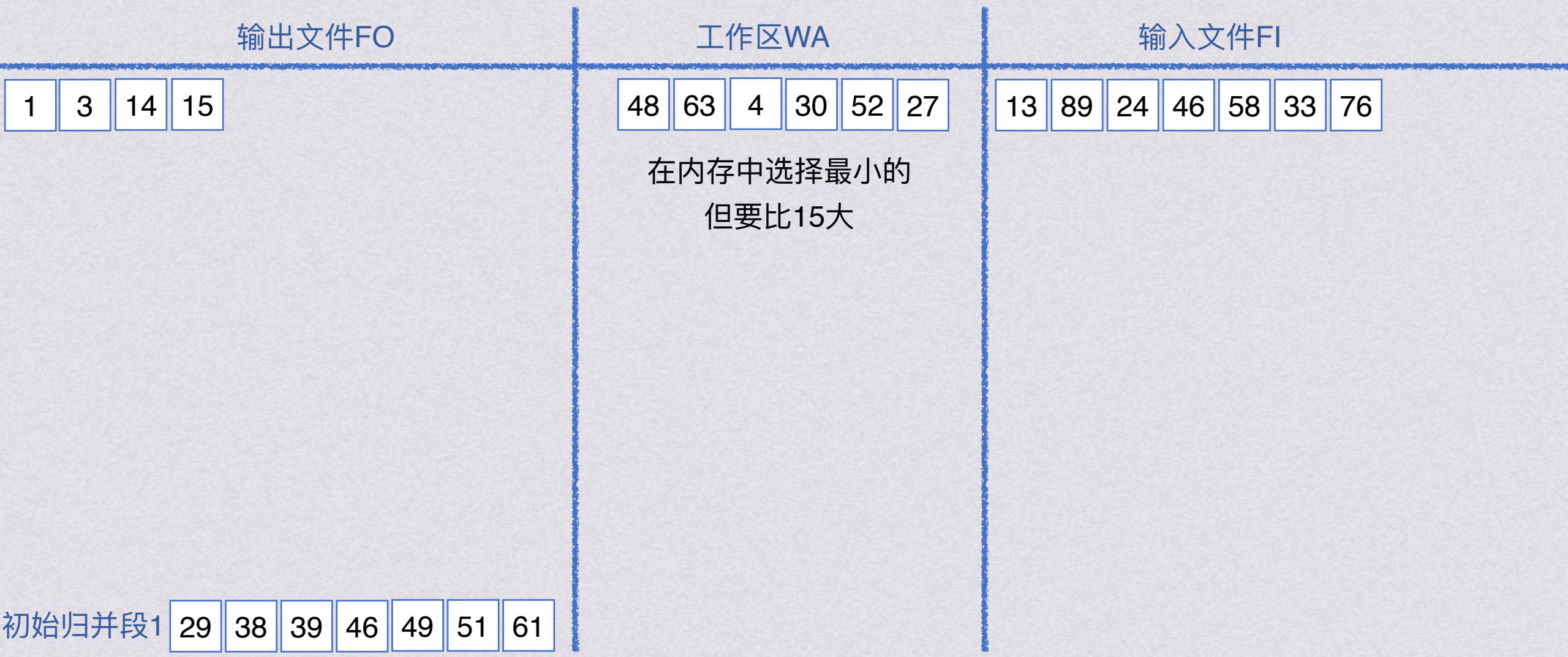
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



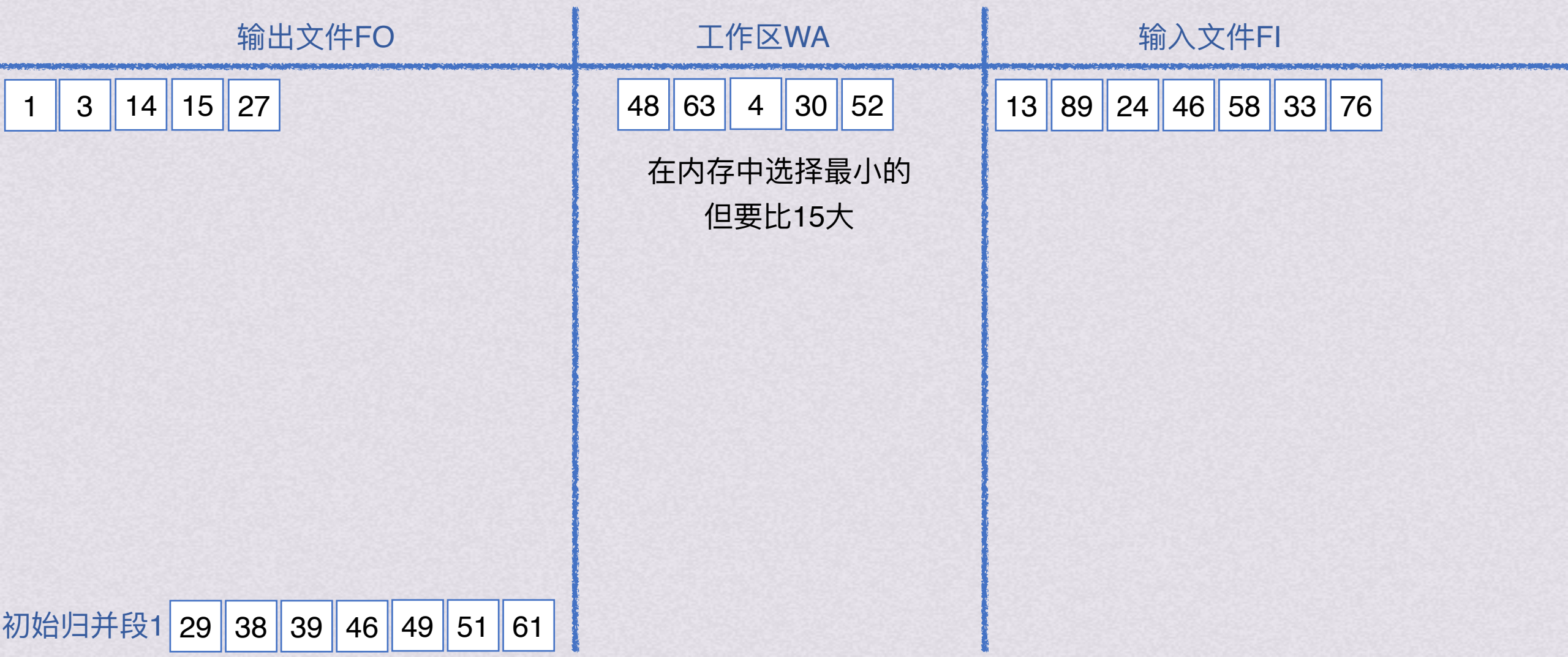
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



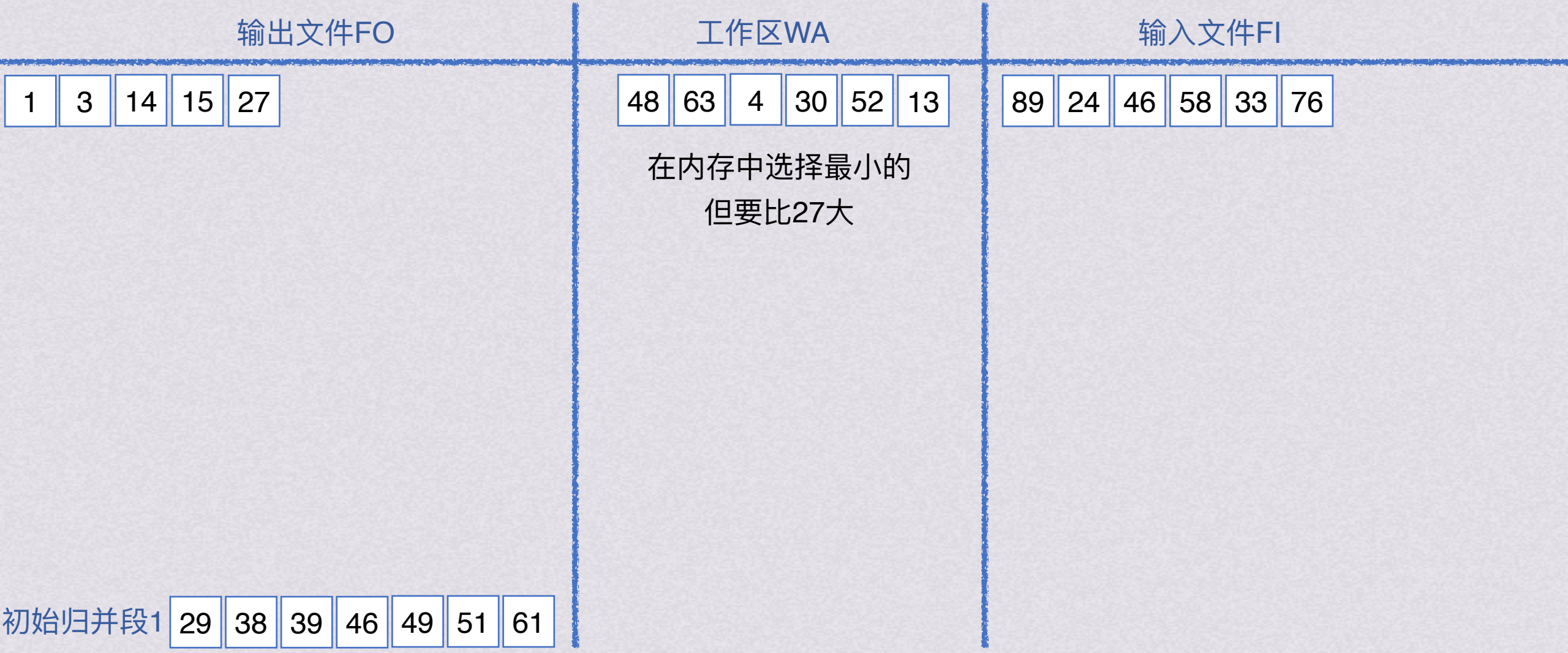
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



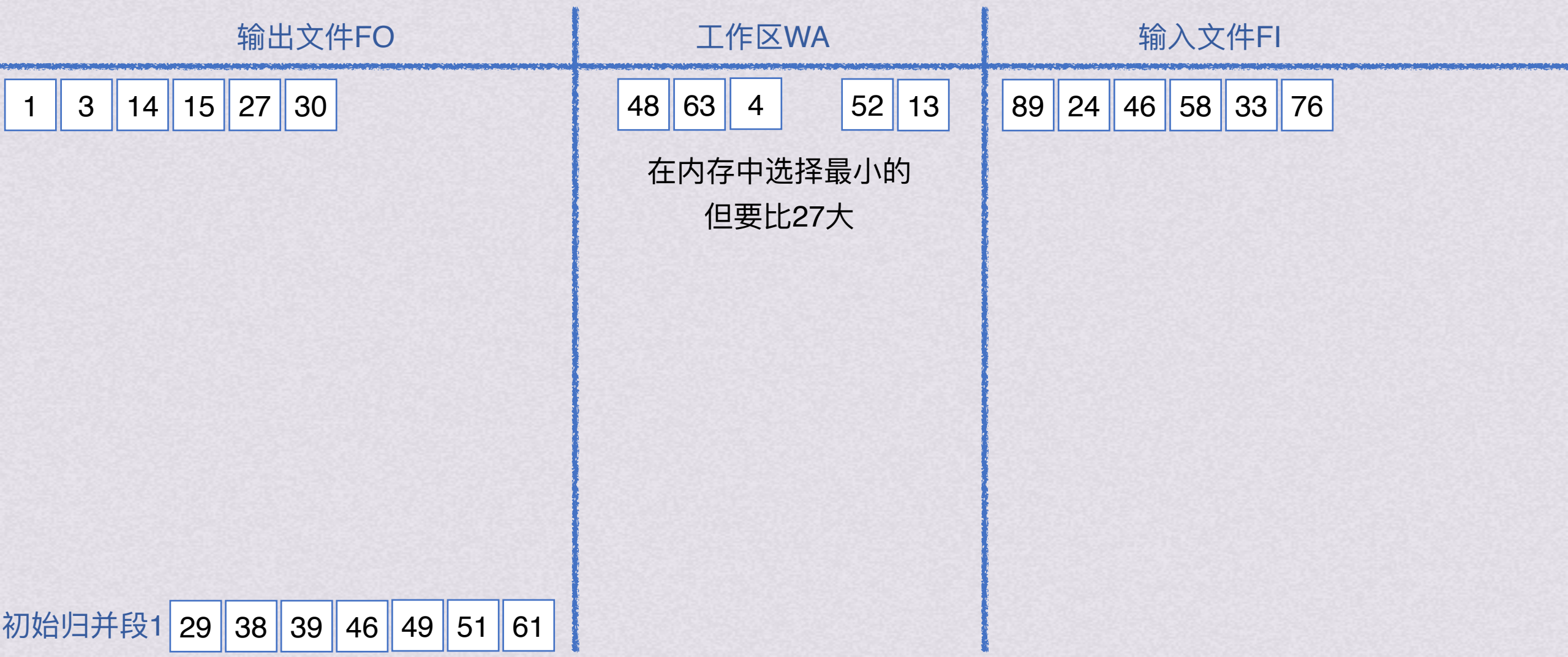
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



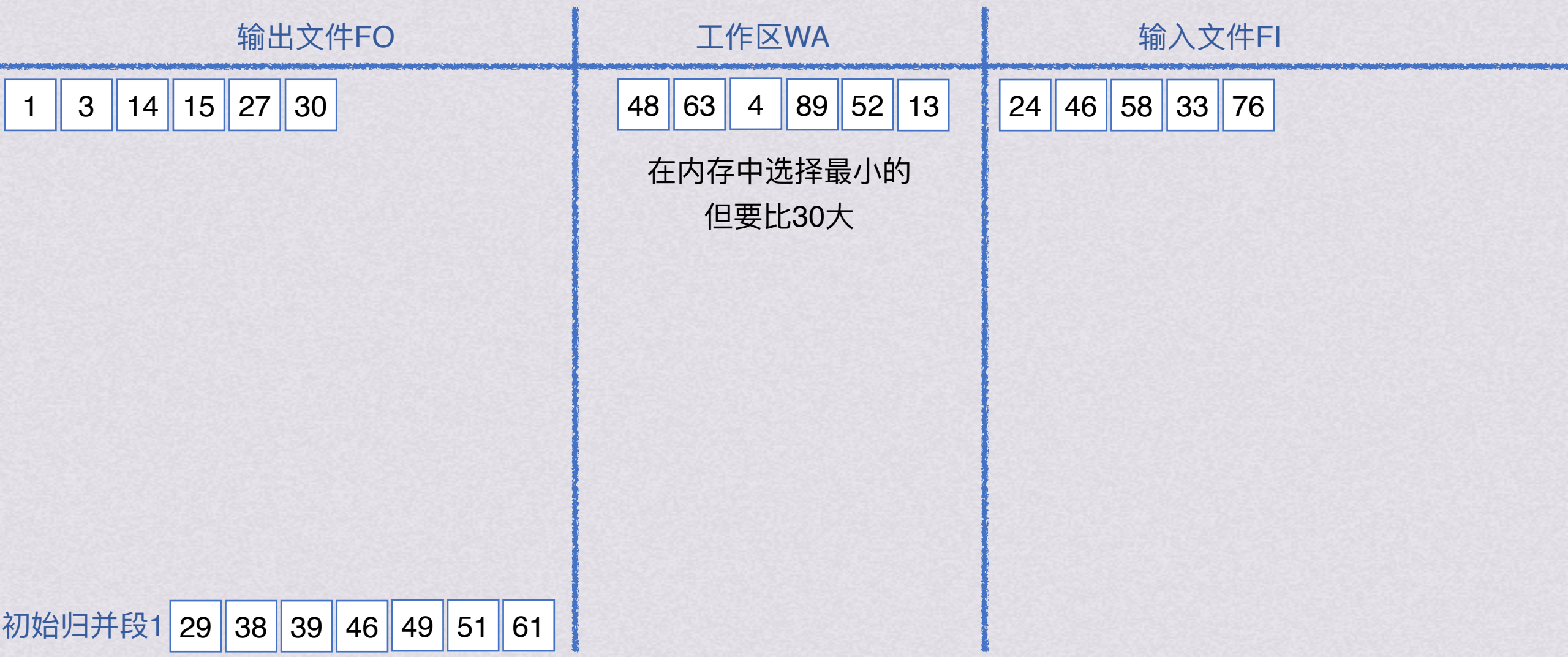
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



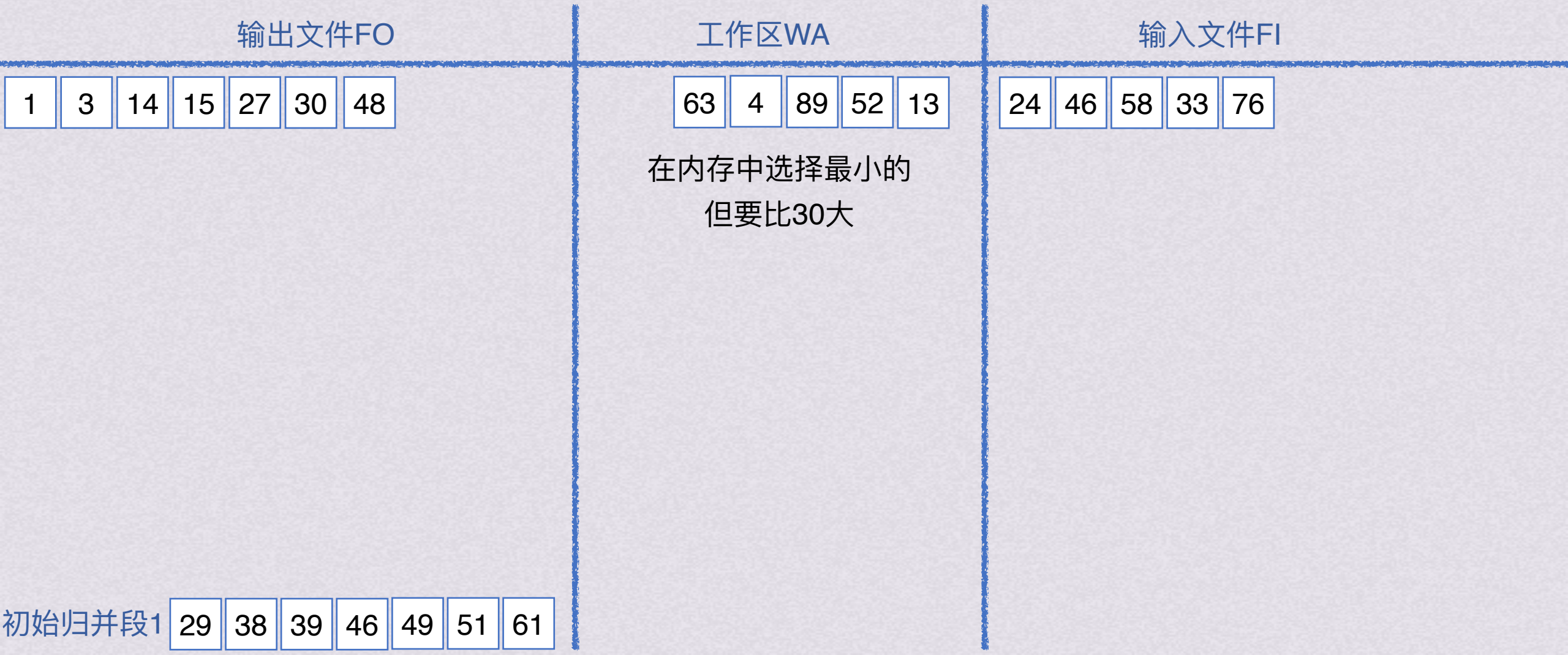
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



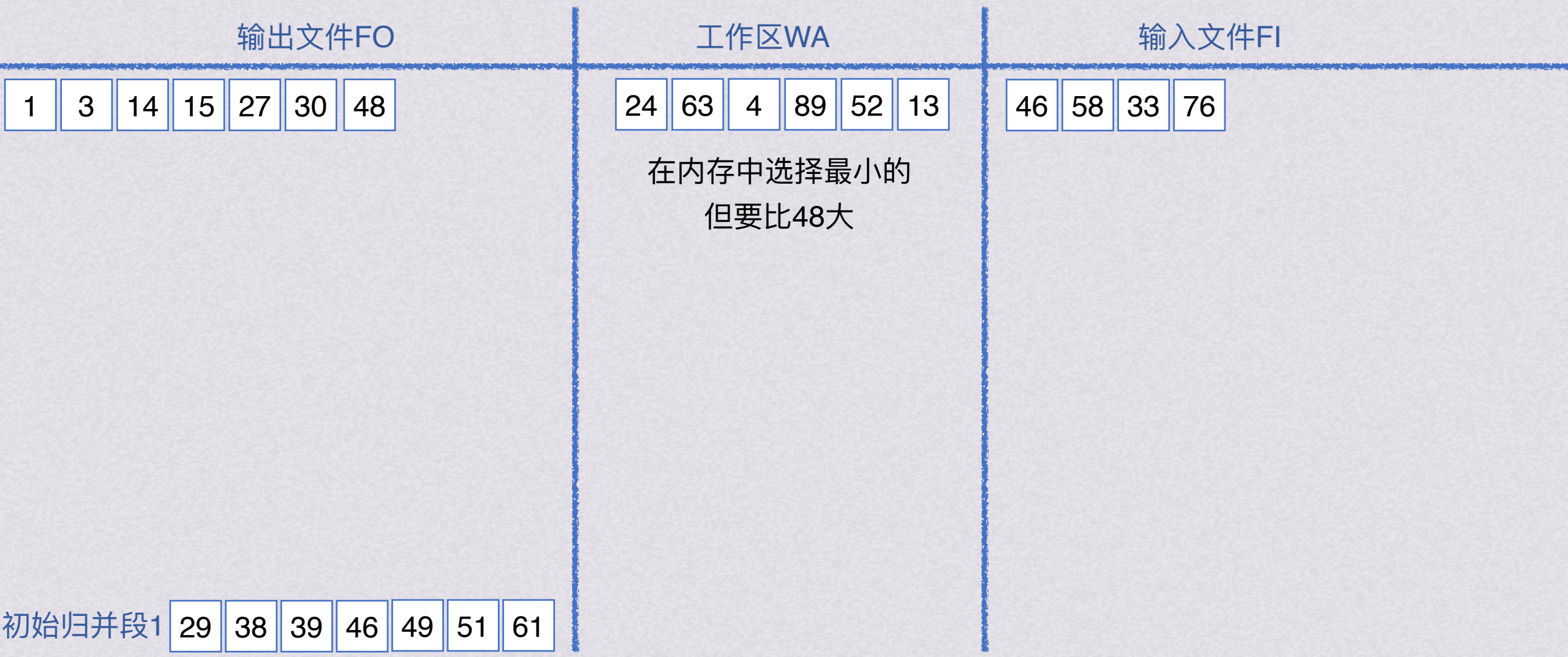
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



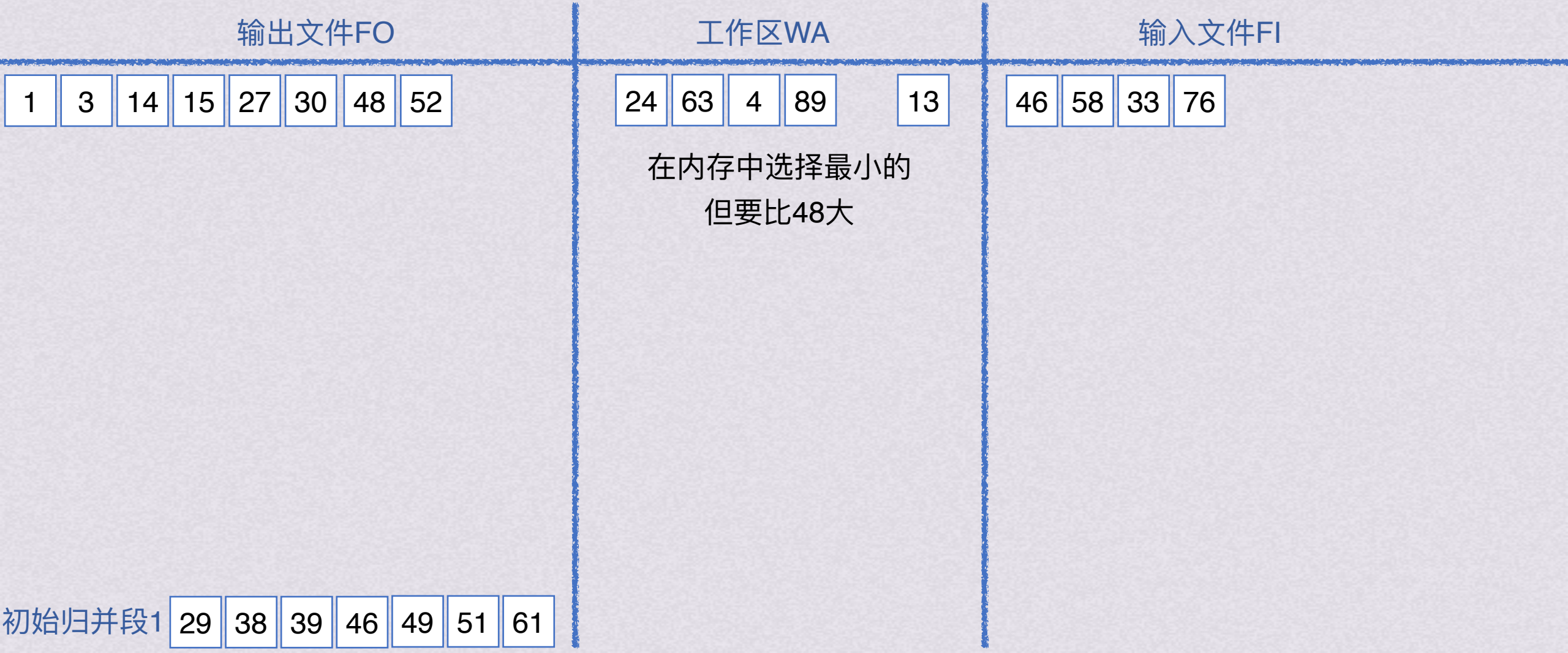
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



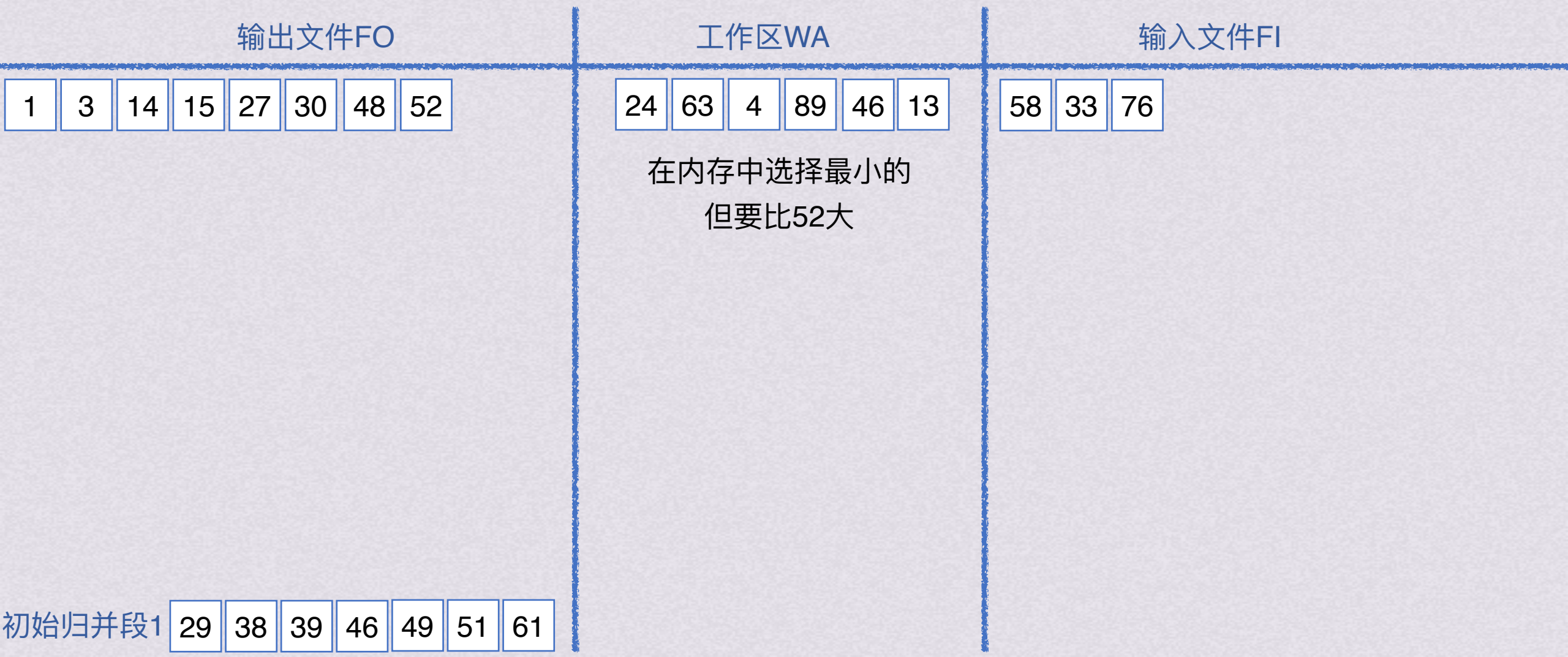
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



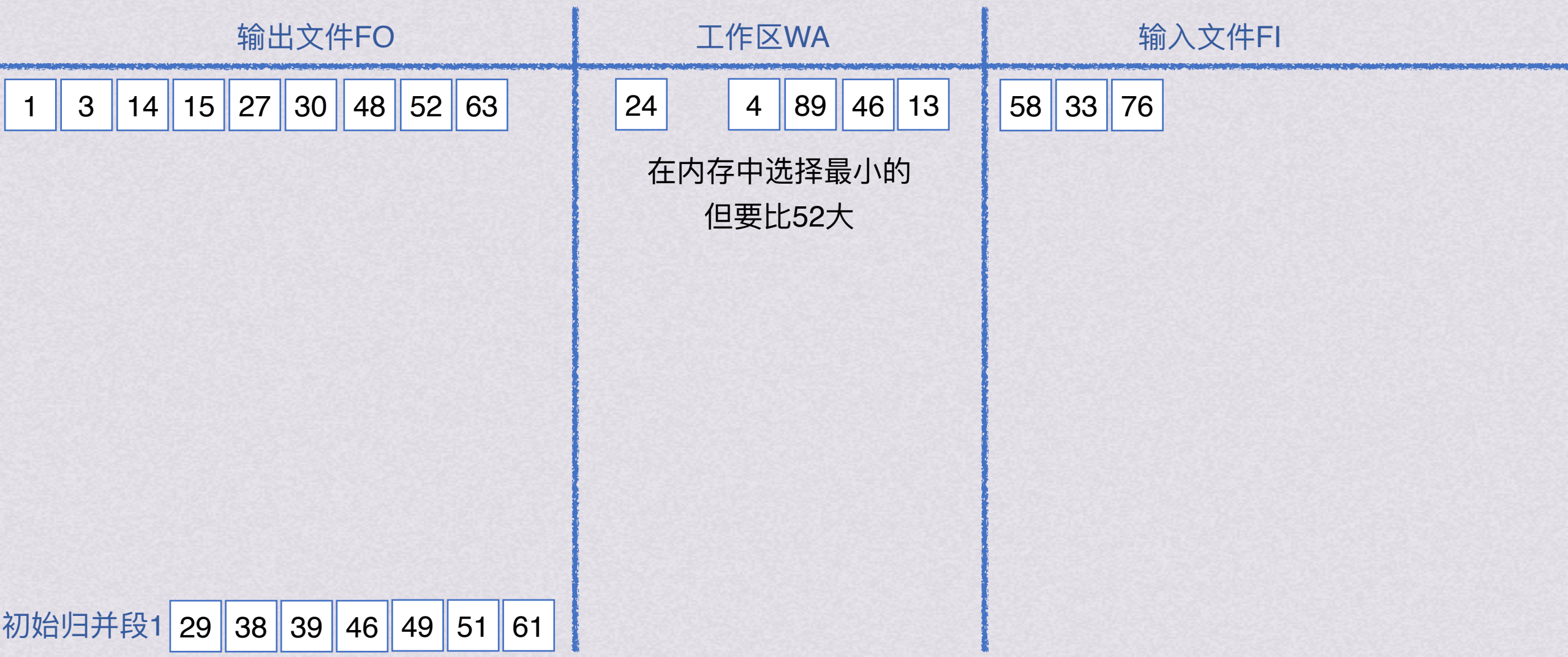
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



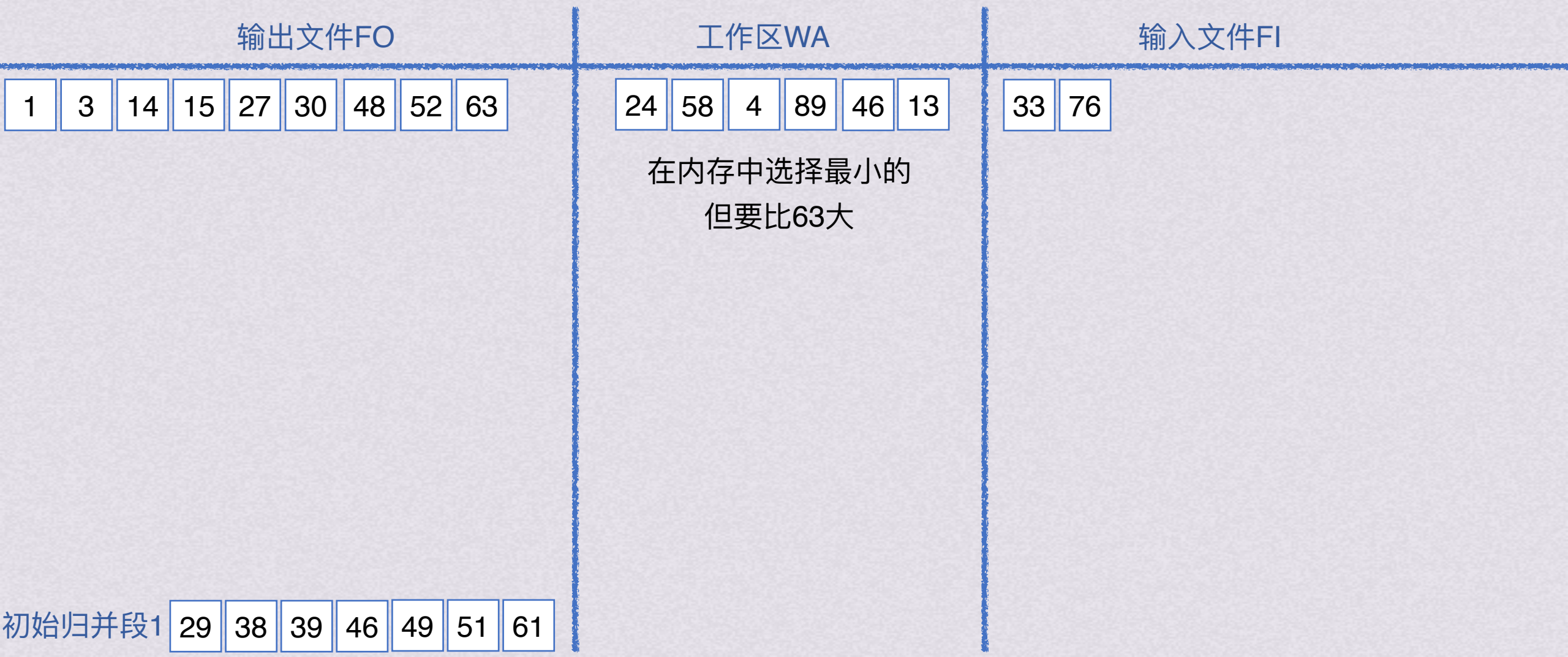
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



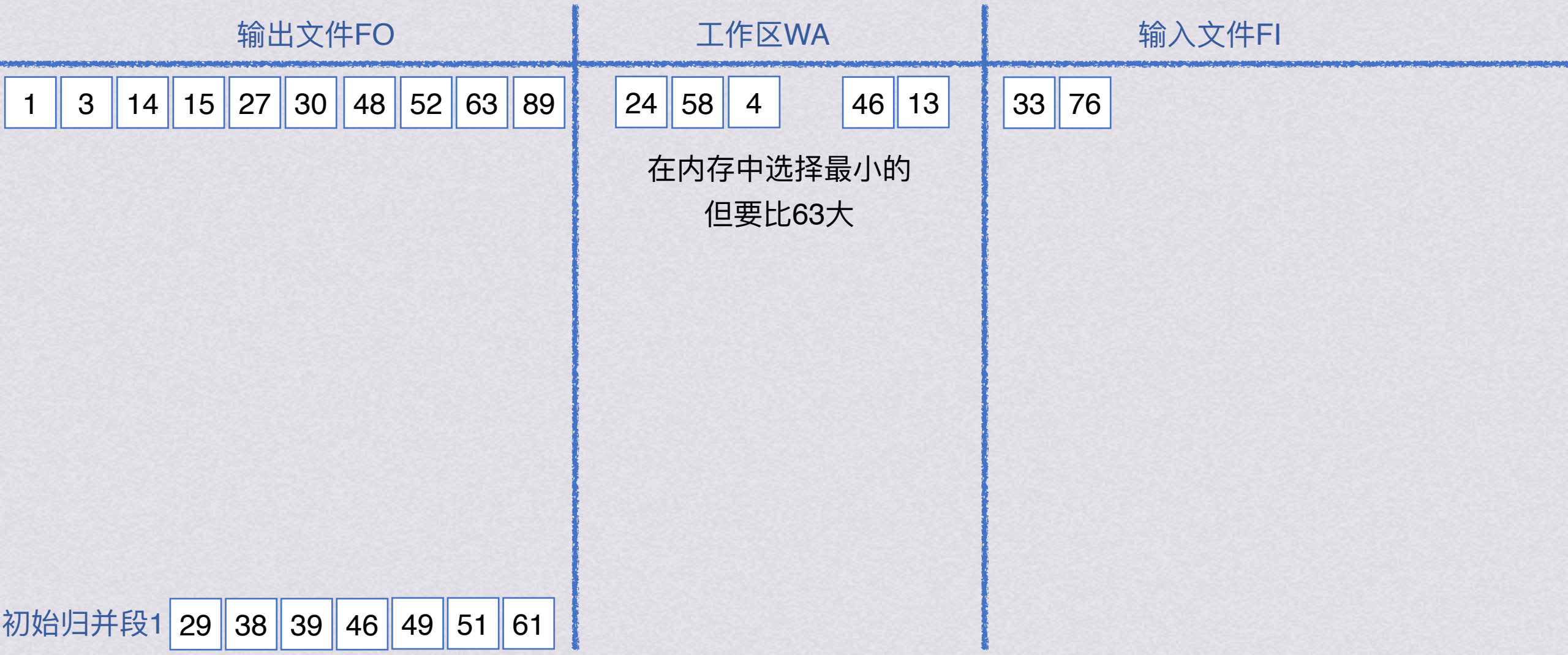
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



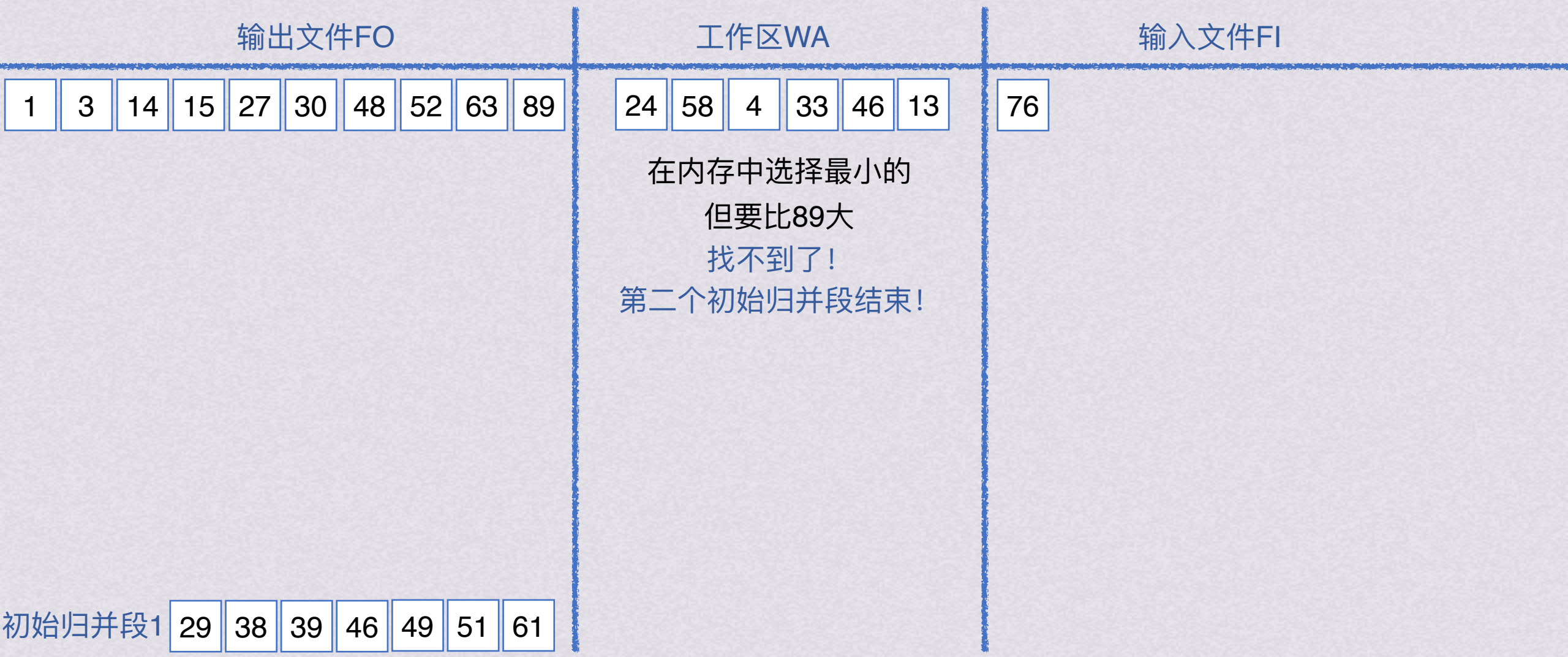
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



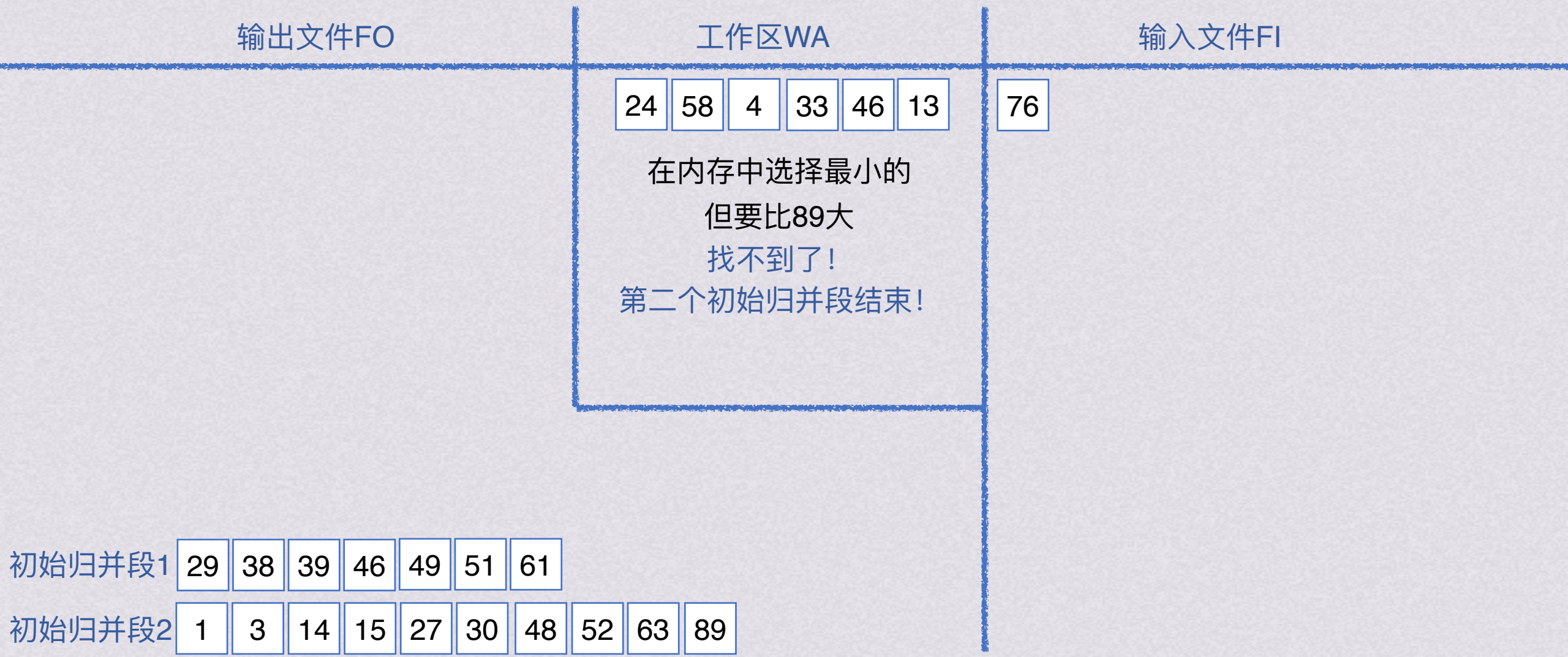
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



外部排序 置换-选择排序

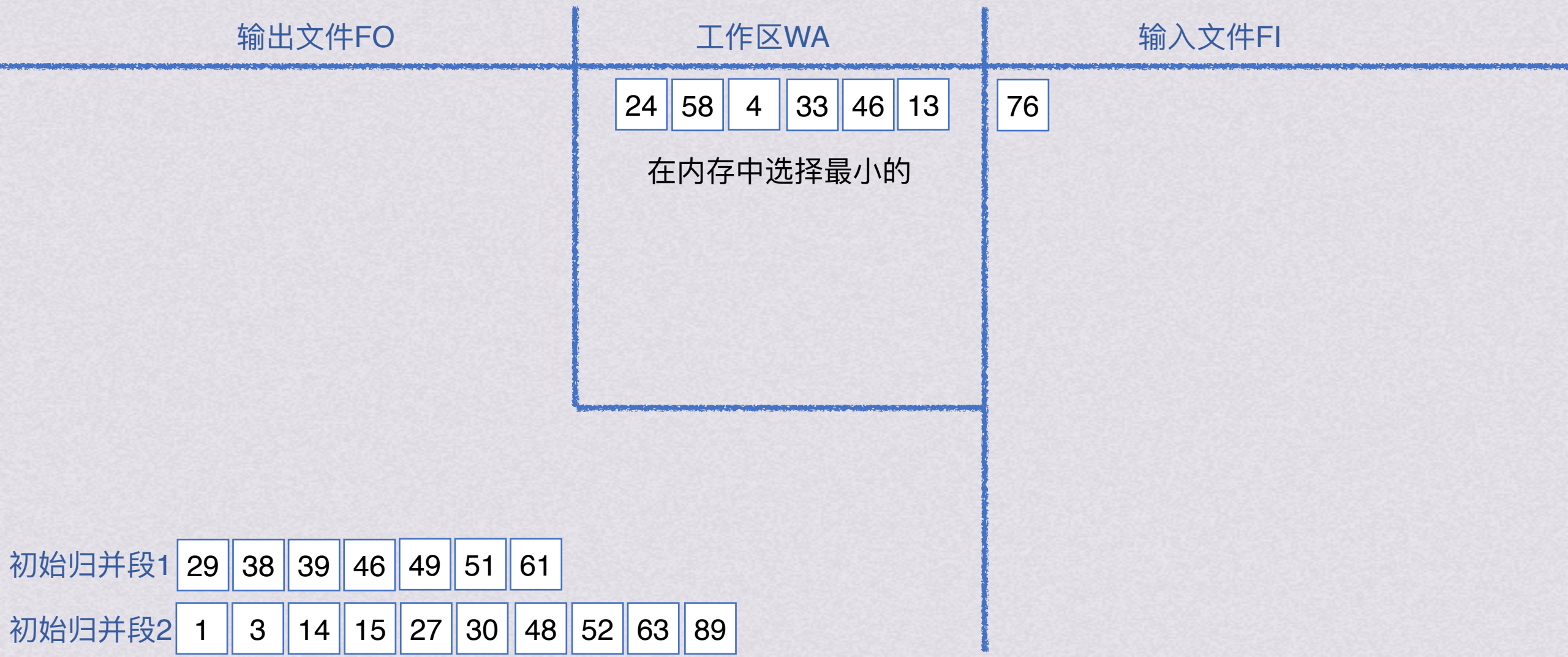
例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段





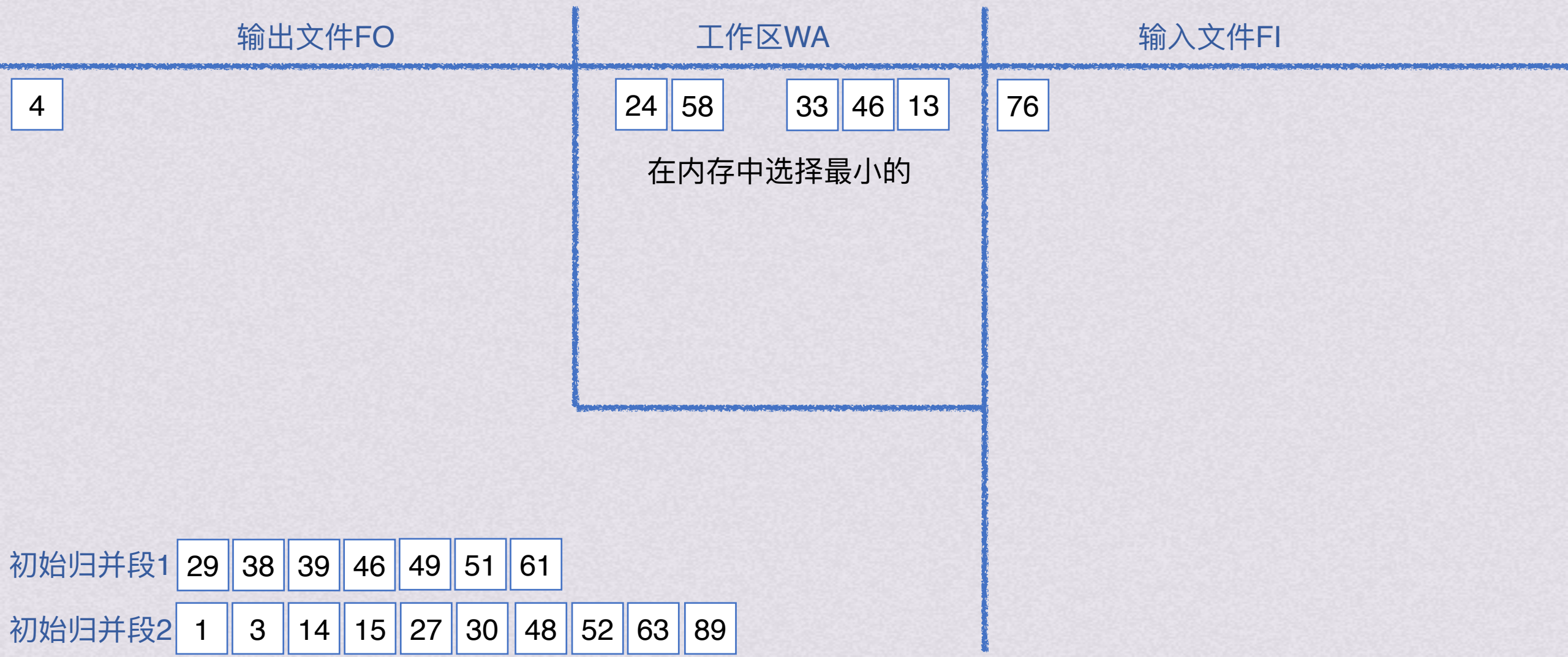
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



外部排序 置换-选择排序

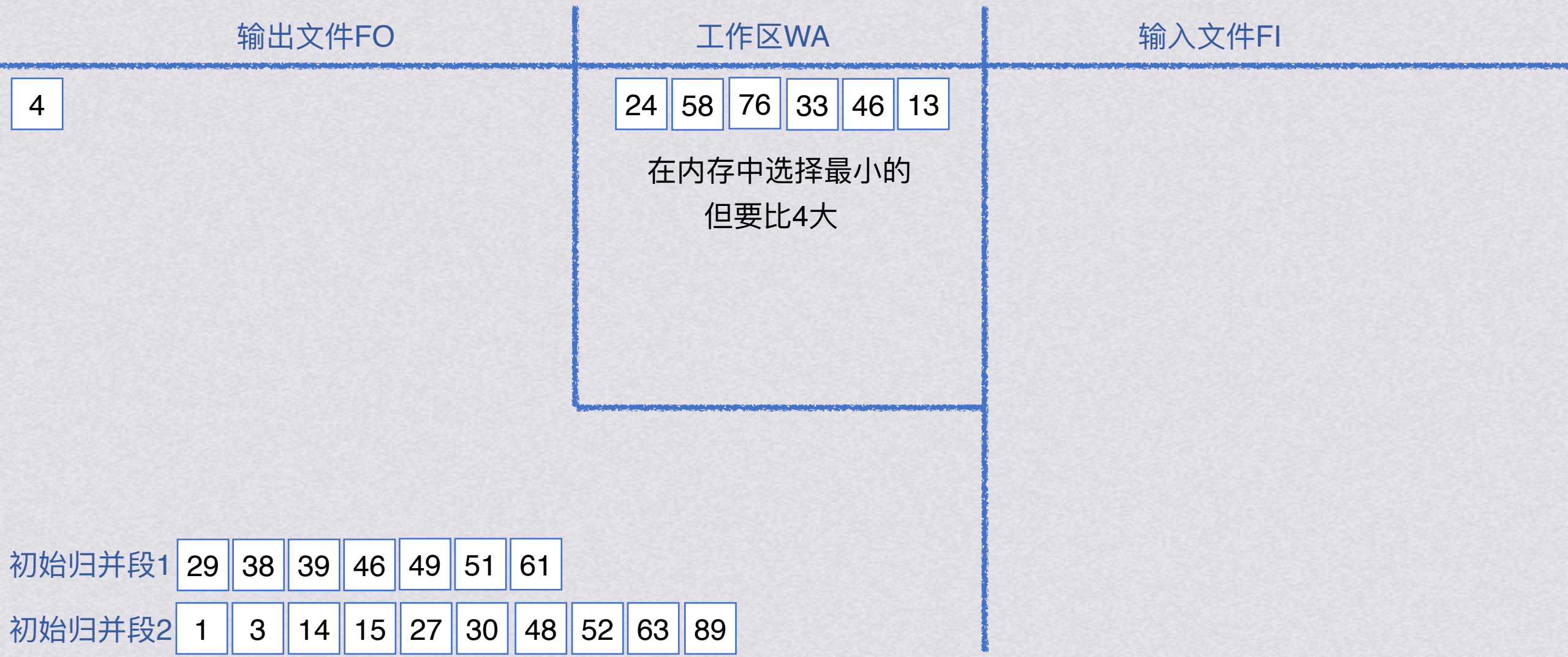
例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段





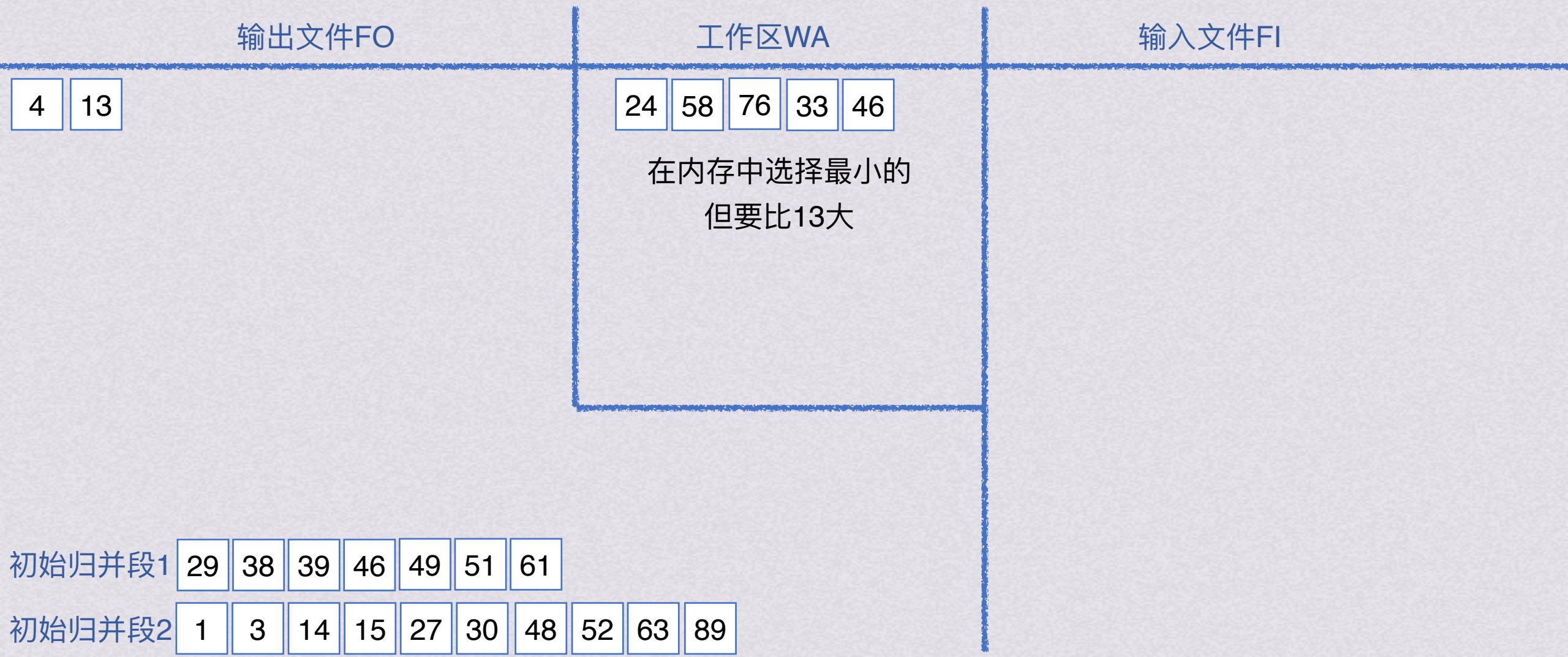
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



外部排序 置换-选择排序

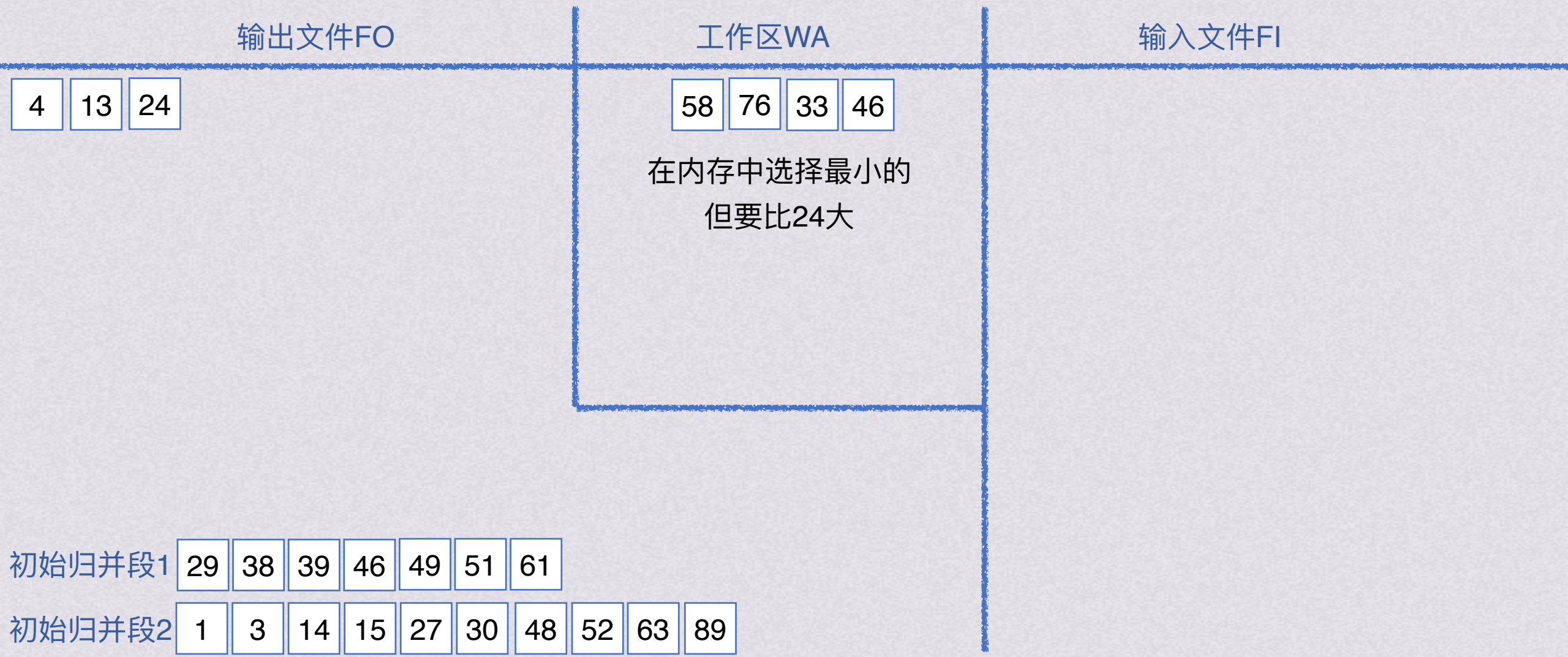
例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段





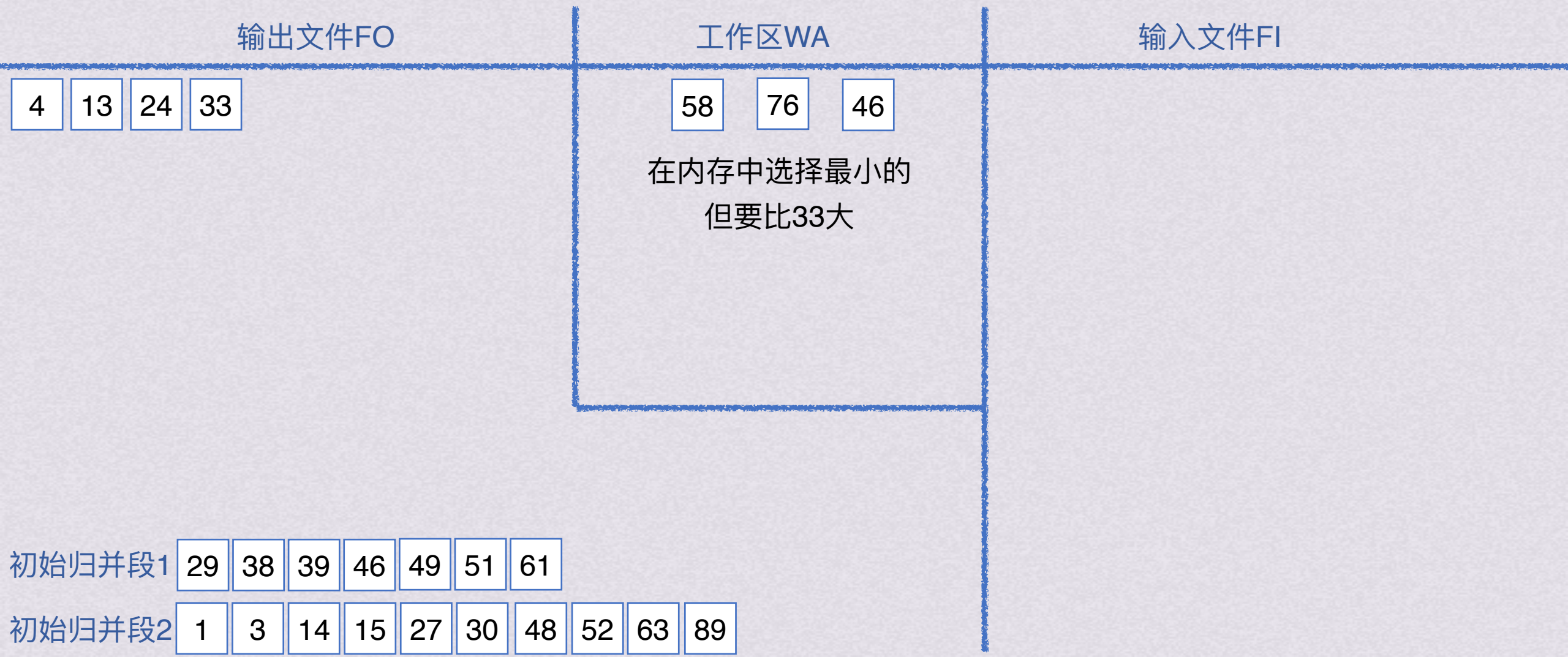
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



外部排序 置换-选择排序

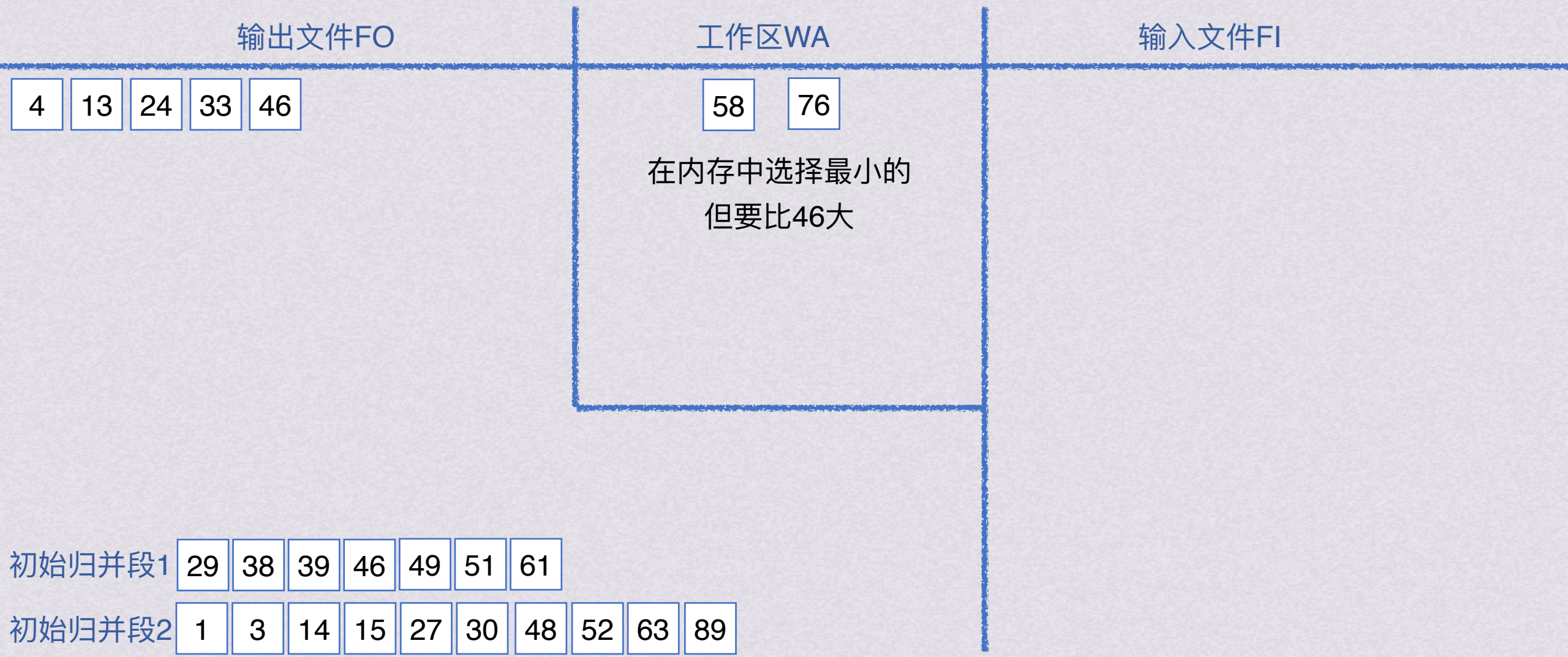
例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段





外部排序 置换-选择排序

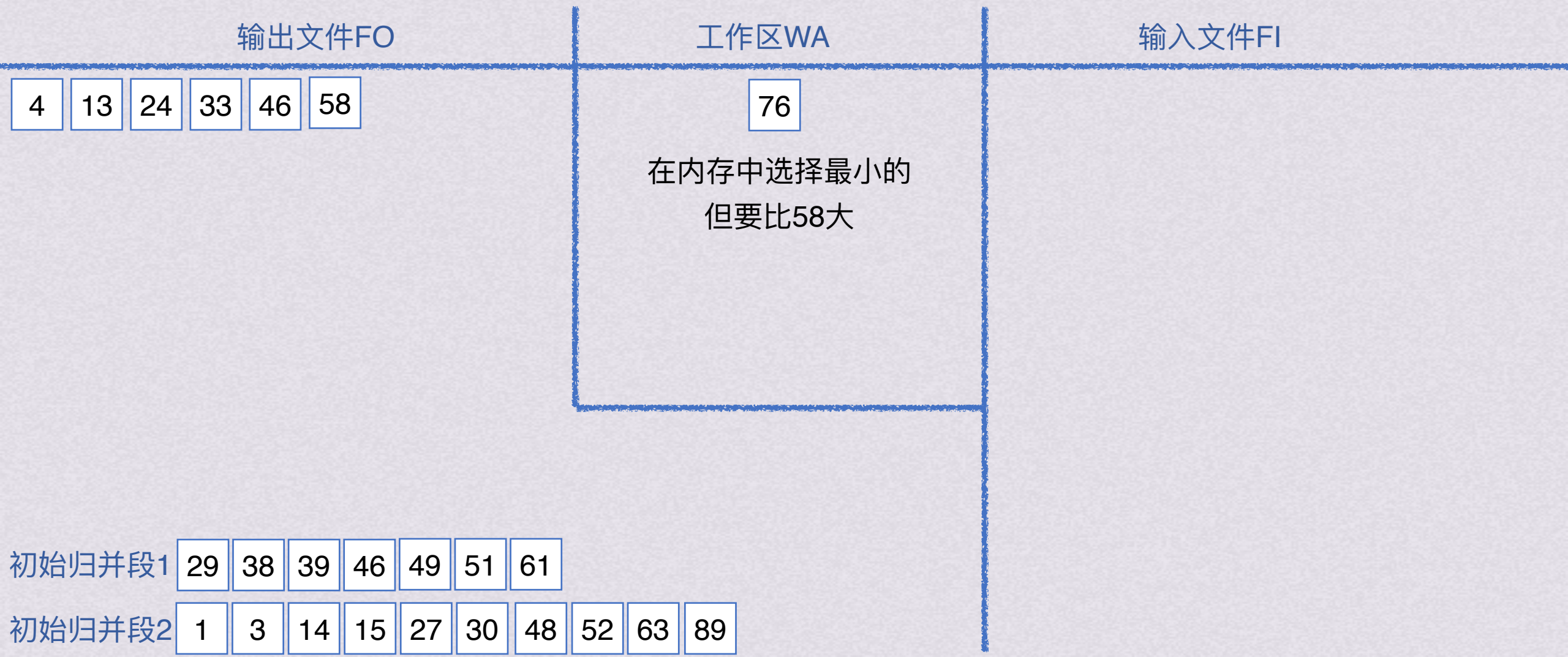
例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段





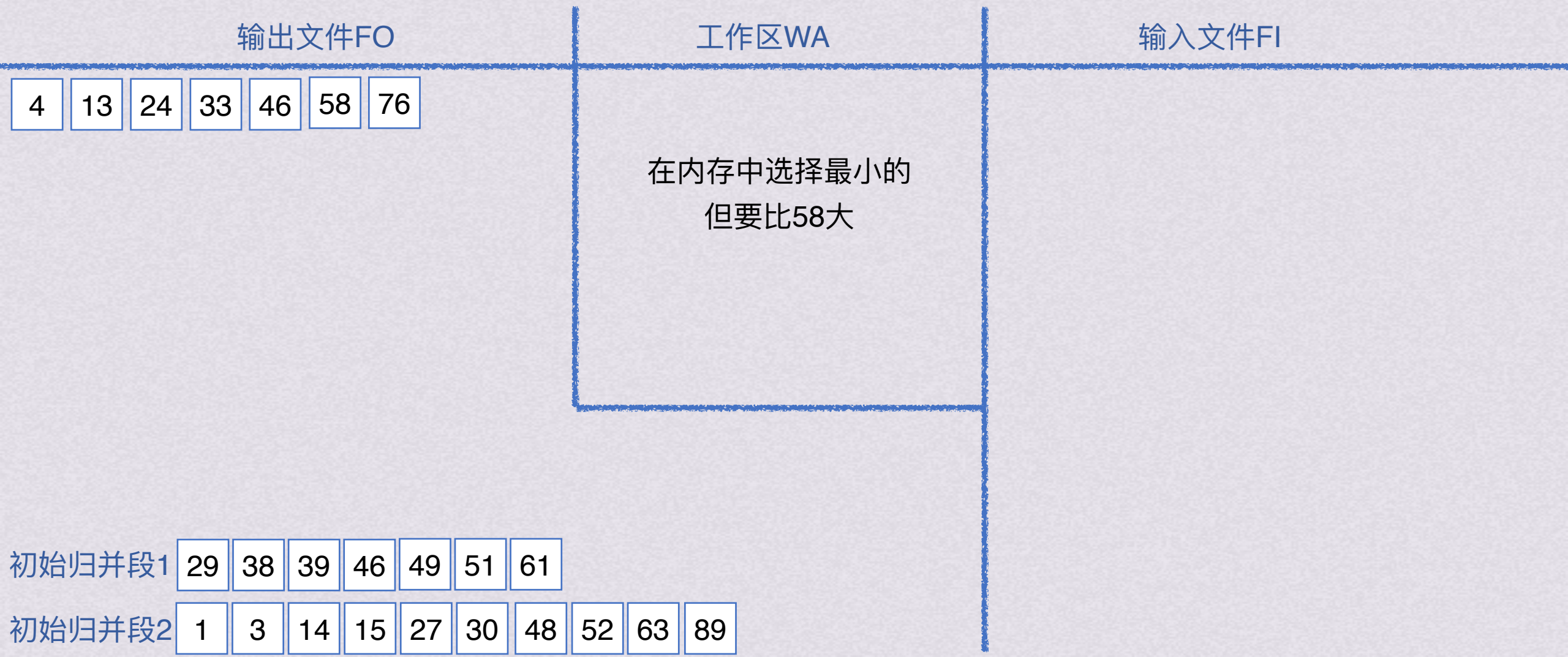
外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段



外部排序 置换-选择排序

例：假设内存工作区可容纳6记录，请用置换-选择排序，生成以下24个记录对应的初始归并段





外部排序 置换-选择排序

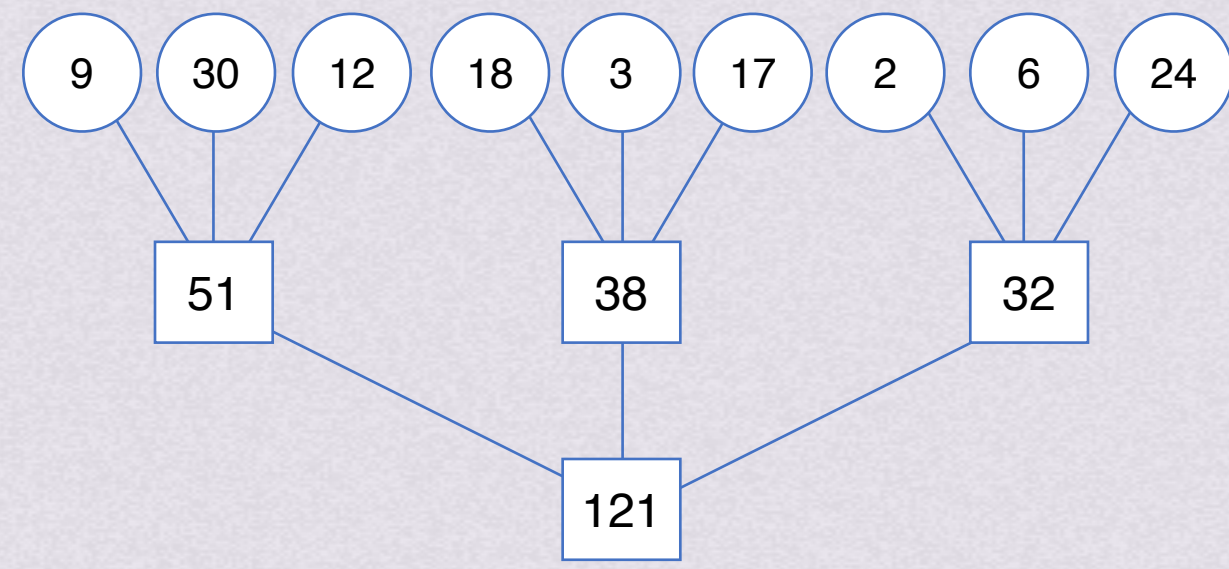
初始归并段1	29	38	39	46	49	51	61			
初始归并段2	1	3	14	15	27	30	48	52	63	89
初始归并段3	4	13	24	33	46	58	76			

最佳归并树——>组织长短不等的初始归并段归并排序，使io次数最少

外部排序

置换-选择排序 最佳归并树

假设由置换-选择得到9个初始归并段，长度分别为9,30,12,8,3,17,2,6,24，现对其进行3路平衡归并，其归并树如图所示



若每个记录占一个物理块，两趟归并所需对外存读写次数总共为 $(9+30+12+18+3+17+2+6+24)*2*2=484$

若能对其应用哈夫曼树知识内容，是不是可以降低io次数？

初始归并段1

29	38	39	46	49	51	61
----	----	----	----	----	----	----

初始归并段2

1	3	14	15	27	30	48	52	63	89
---	---	----	----	----	----	----	----	----	----

初始归并段3

4	13	24	33	46	58	76
---	----	----	----	----	----	----

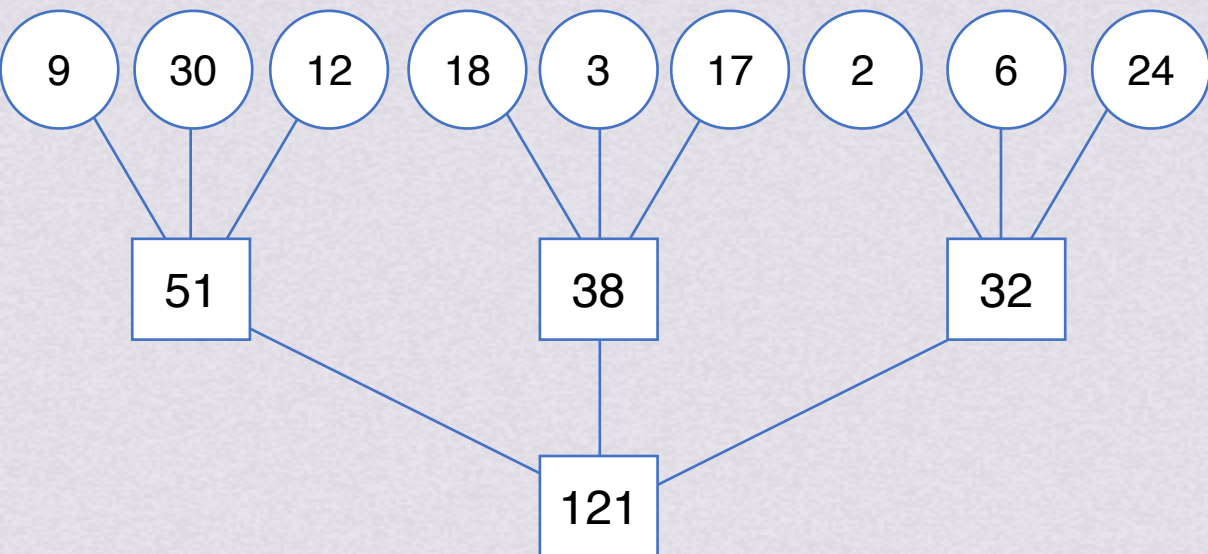
(本例归并段数量太少，舍弃)



外部排序

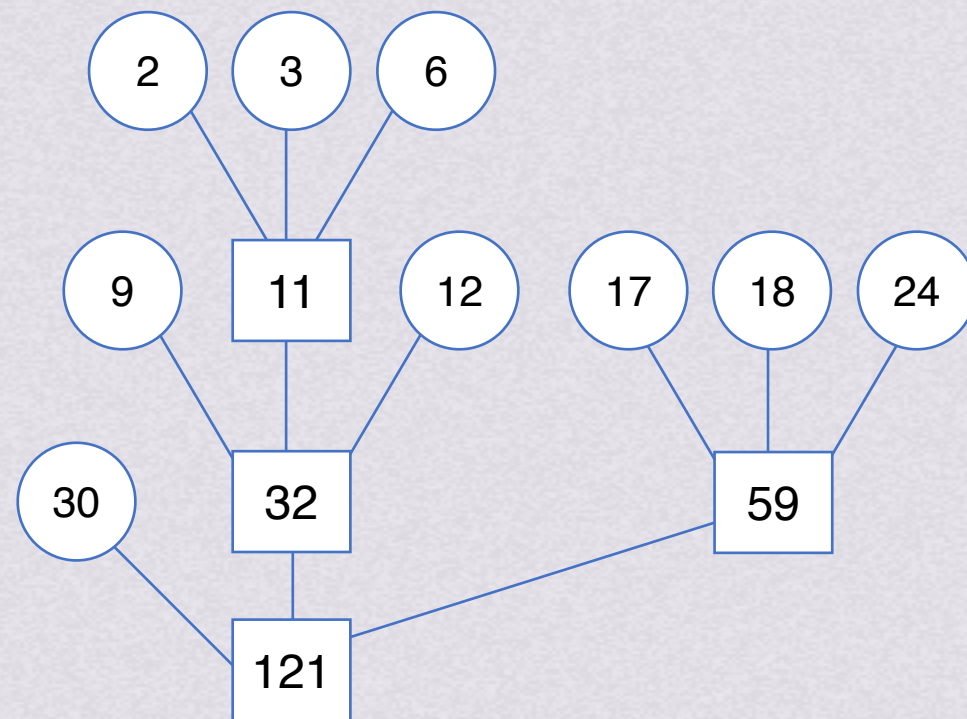
置换-选择排序 最佳归并树

假设由置换-选择得到9个初始归并段，长度分别为9,30,12,8,3,17,2,6,24，现对其进行3路平衡归并，其归并树如图所示



若每个记录占一个物理块，
两趟归并所需对外存读写次数总共为
 $(9+30+12+18+3+17+2+6+24)*2*2=484$

若能对其应用哈夫曼树知识内容，是不是可以降低io次数？



读写次数仅为446

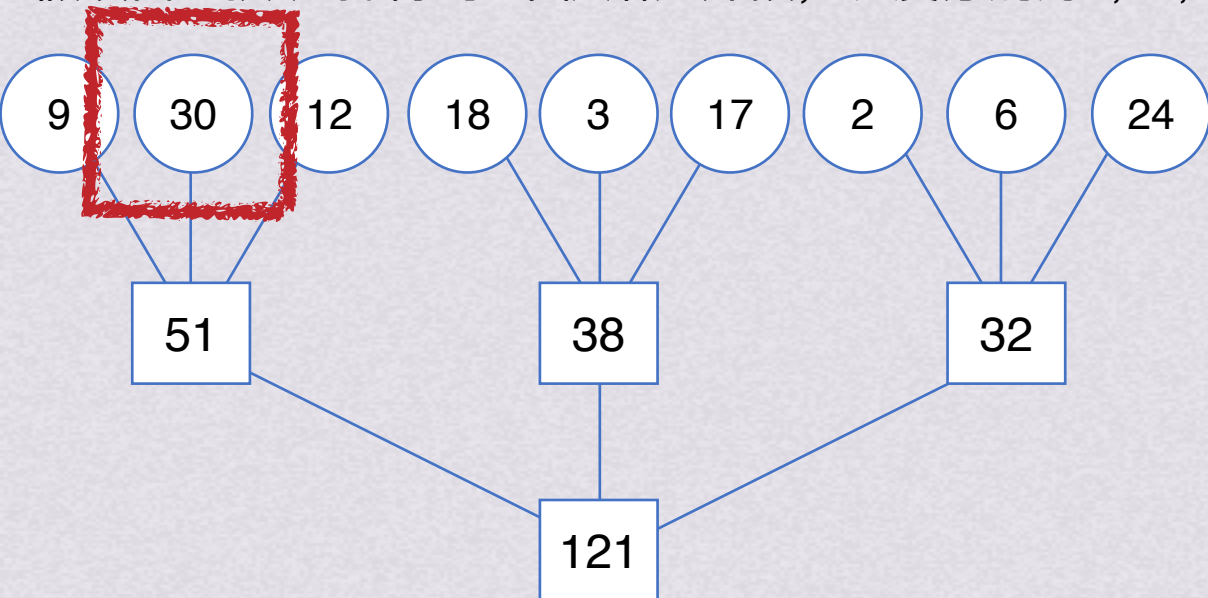
最佳归并树



外部排序

置换-选择排序 最佳归并树

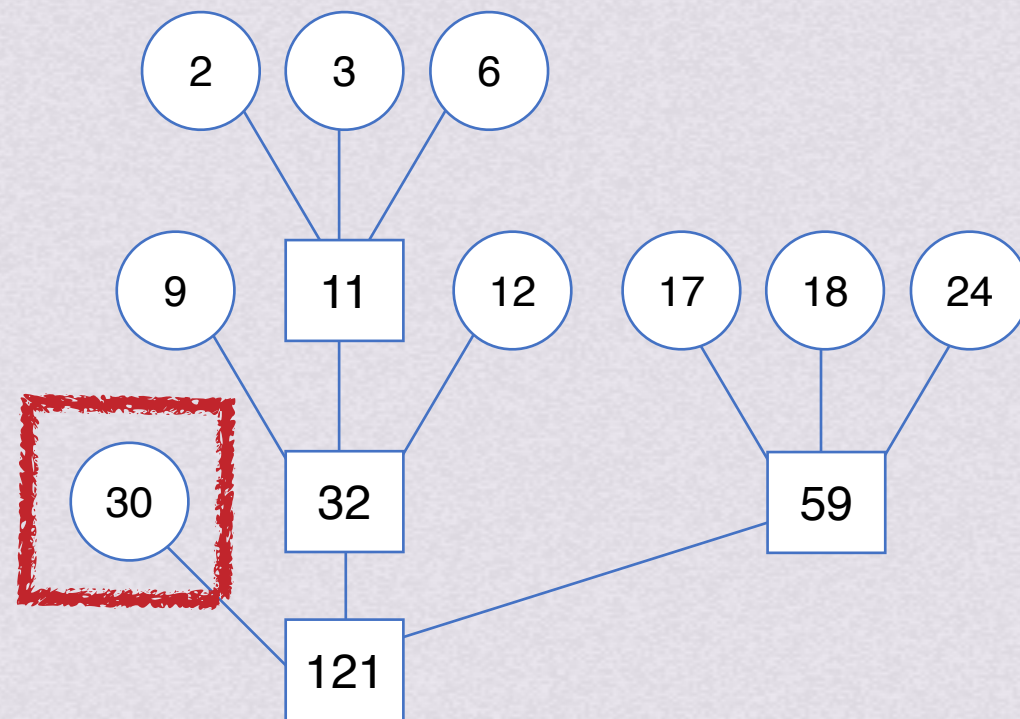
假设由置换-选择得到9个初始归并段，长度分别为9,30,12,8,3,17,2,6,24，现对其进行3路平衡归并，其归并树如图所示



若每个记录占一个物理块，
两趟归并所需对外存读写次数总共为
 $(9+30+12+18+3+17+2+6+24)*2*2=484$

去掉一个节点以后，还怎么组成严格k叉树呢？

若能对其应用哈夫曼树知识内容，是不是可以降低io次数？



读写次数仅为446
最佳归并树

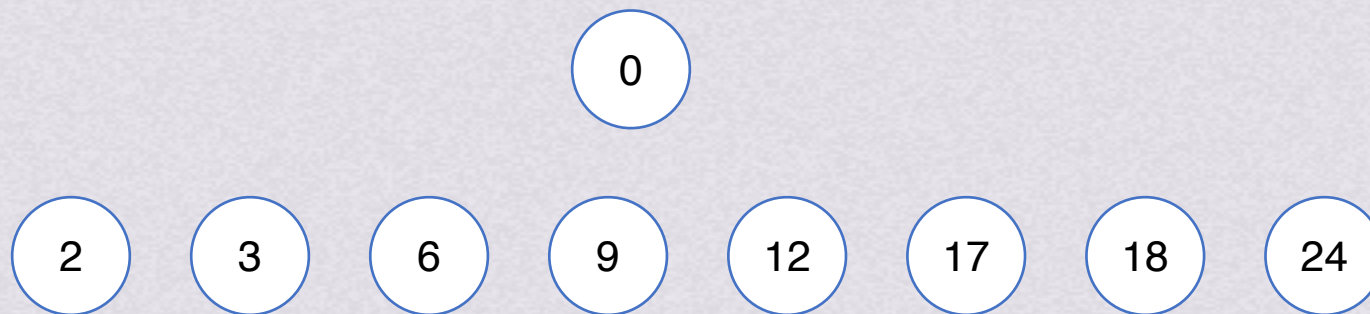


外部排序

置换-选择排序 最佳归并树

假设由置换-选择得到9个初始归并段，长度分别为9,30,12,8,3,17,2,6,24，现对其进行3路平衡归并，其归并树如图所示

去掉一个节点以后，还怎么组成严格k叉树呢？ 添加长度为0的“虚段”



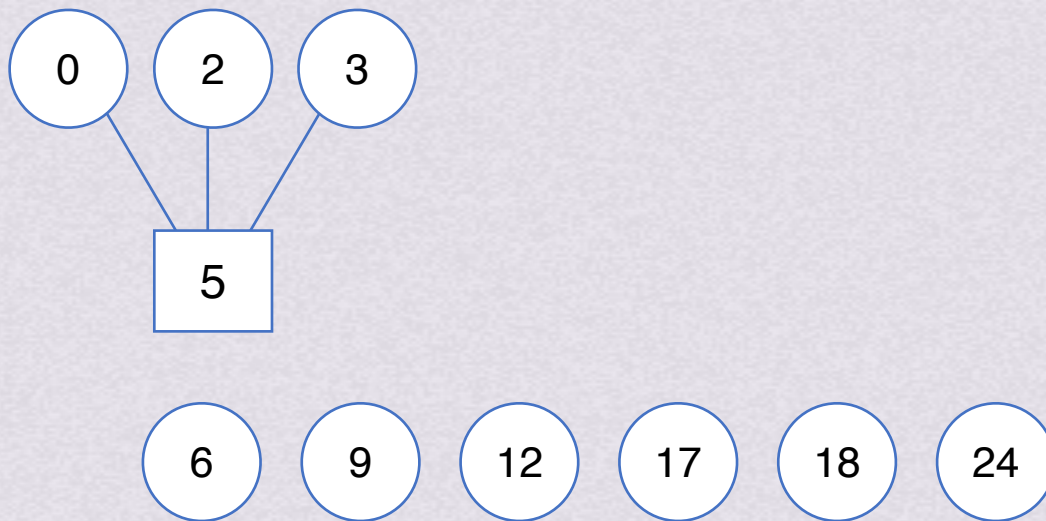


外部排序

置换-选择排序 最佳归并树

假设由置换-选择得到9个初始归并段，长度分别为9,30,12,8,3,17,2,6,24，现对其进行3路平衡归并，其归并树如图所示

去掉一个节点以后，还怎么组成严格k叉树呢？ 添加长度为0的“虚段”



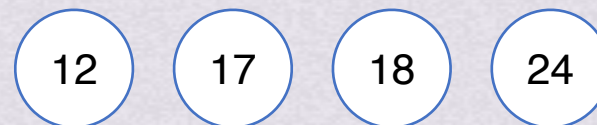
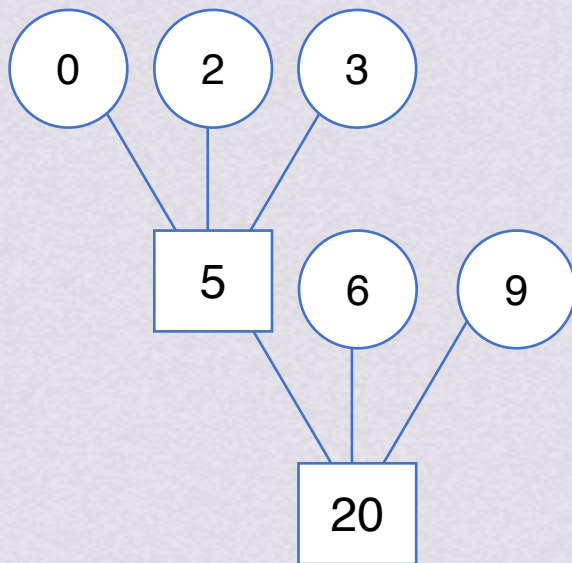


外部排序

置换-选择排序 最佳归并树

假设由置换-选择得到9个初始归并段，长度分别为9,30,12,8,3,17,2,6,24，现对其进行3路平衡归并，其归并树如图所示

去掉一个节点以后，还怎么组成严格k叉树呢？ 添加长度为0的“虚段”



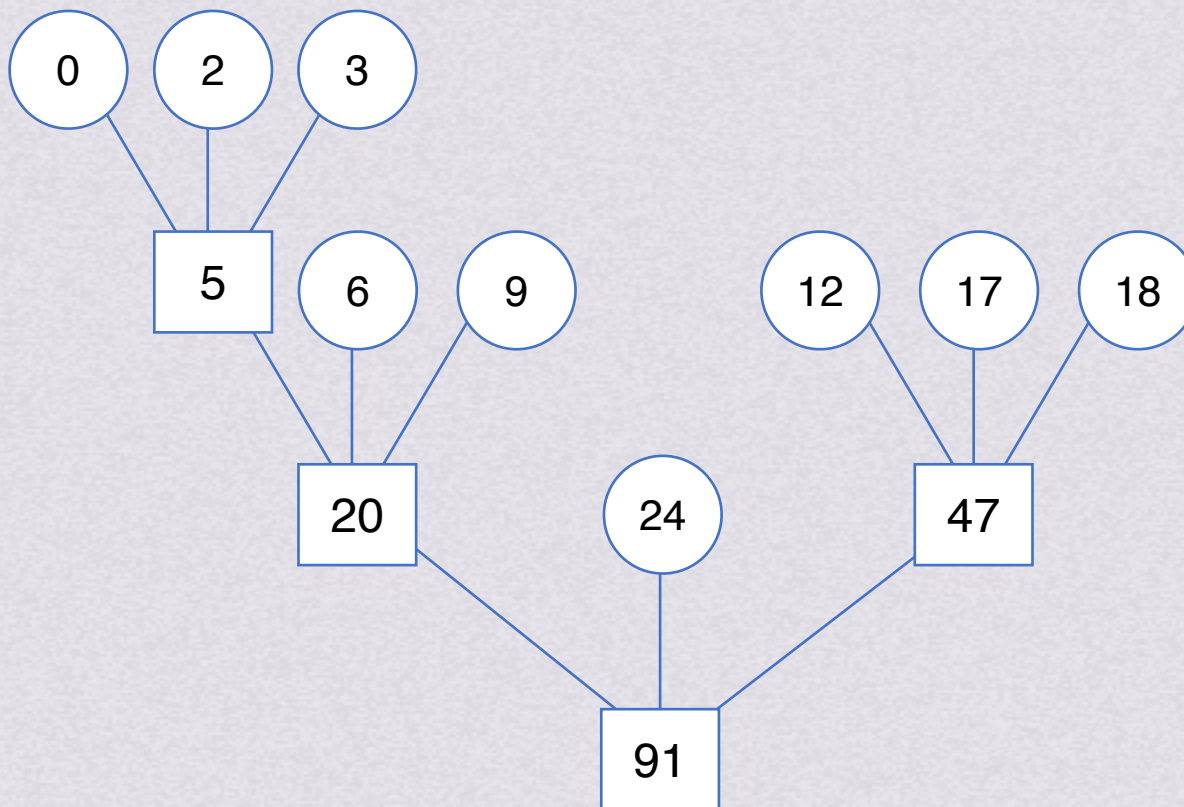


外部排序

置换-选择排序 最佳归并树

假设由置换-选择得到9个初始归并段，长度分别为9,30,12,8,3,17,2,6,24，现对其进行3路平衡归并，其归并树如图所示

去掉一个节点以后，还怎么组成严格k叉树呢？ 添加长度为0的“虚段”





外部排序

置换-选择排序 最佳归并树

假设由置换-选择得到9个初始归并段，长度分别为9,30,12,8,3,17,2,6,24，现对其进行3路平衡归并，其归并树如图所示

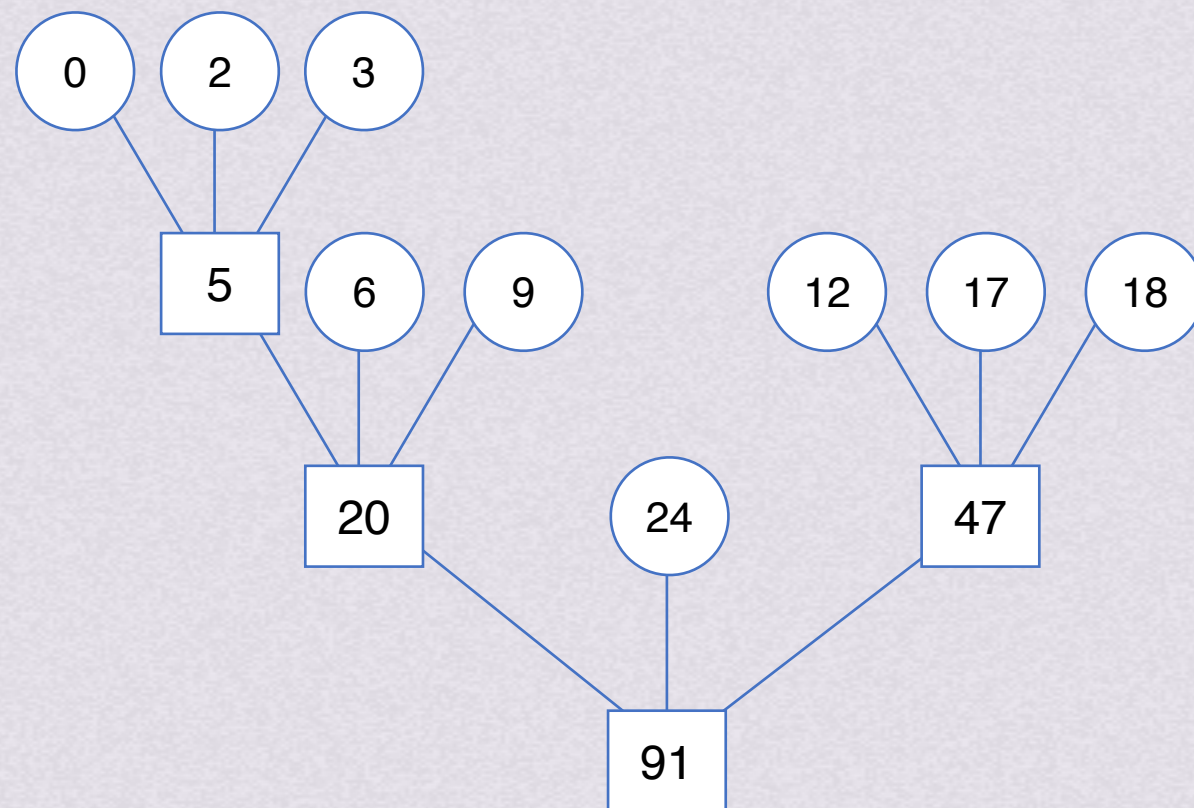
去掉一个节点以后，还怎么组成严格k叉树呢？ 添加长度为0的“虚段”

对严格k叉树有 $n_0 = (k-1)n_k + 1$

$$\text{故 } n_k = \frac{n_0 - 1}{k - 1}$$

若 $(n_0 - 1) \% (k - 1) = 0$ ，则不需要添加虚段

若 $(n_0 - 1) \% (k - 1) = u \neq 0$ ，则需再加 $k - u - 1$ 个虚段



严格三叉树/最佳归并树



小试🐮刀

(前文例题) 有8个初始归并段，进行3路归并，想要做出最佳归并树，需要添加几个虚段？

因为是3路归并的最佳归并树

所以该最佳归并树只能有度为3和度为0的节点

运用【度数和=节点数-1】的树的性质即可解决

假设需要添加k个虚段， $n_0=8+k$

严格三叉树的度数和= $3*n_3+0*n_0$

节点数量和= n_3+n_0

度数和=节点数-1

$$3*n_3+0*n_0=n_3+n_0-1$$

$$2*n_3=8+k-1=7+k$$

n_3 一定是整数

所以7+k是2的倍数->k最小为1



小试🐮刀

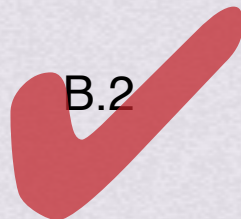
【2019】 设外存上有120个初始归并段，进行12路归并时，为了实现最佳归并，需要补充的虚段个数是

A.1

B.2

C.3

D.4



因为是12路归并的最佳归并树

所以该最佳归并树只能有度为12和度为0的节点

运用【度数和=节点数-1】的树的性质即可解决

度数和=节点数-1

$$12 \cdot n_{12} + 0 \cdot n_0 = n_{12} + n_0 - 1$$

$$11 \cdot n_{12} = 120 + k - 1 = 119 + k$$

n_{12} 一定是整数

所以119+k是11的倍数->k最小为2

假设需要添加k个虚段， $n_0 = 120 + k$

严格十二叉树的度数和= $12 \cdot n_{12} + 0 \cdot n_0$

节点数量和= $n_{12} + n_0$